Accelerating metabolic engineering through multimodal phenotype prediction with TorchCell and the Cell Graph Transformer (14/15 words)

Michael Volk[1,2,3*†], Aurosish Sharma[1,2,3†] and Huimin Zhao[1,2,3,4,5,6*]

[1]DOE Center for Advanced Bioenergy and Bioproducts Innovation, University of Illinois Urbana-Champaign, Urbana, IL, 61801, USA.

[2]Department of Chemical and Biomolecular Engineering, University of Illinois Urbana-Champaign, Urbana, IL, 61801, USA.

[3]Carl R. Woese Institute for Genomic Biology, University of Illinois Urbana-Champaign, Urbana, IL, 61801, USA.

[4]NSF iBioFoundry, University of Illinois Urbana-Champaign, Urbana, IL, 61801, USA.

[5]Center for Biophysics and Quantitative Biology, University of Illinois Urbana-Champaign, Urbana, IL, 61801, USA.

[6]NSF Molecule Maker Lab Institute, University of Illinois Urbana-Champaign, Urbana, IL, 61801, USA.

*Corresponding author(s). E-mail(s): mjvolk3@illinois.edu; zhao5@illinois.edu;

Contributing authors: ;

†These authors contributed equally to this work.

Abstract (≤150 words) [140/150]

Deep learning has advanced protein structure prediction, yet metabolic engineering lags because phenotype data are scattered across heterogeneous studies lacking a shared ontology. We present TorchCell, an open-source framework that annotates literature and public datasets with a shared ontology and ingests them into a Neo4j knowledge graph, then builds deep-learning-ready datasets in which each strain is a perturbation operator over a shared wildtype reference cell combining genome sequence, gene networks, metabolism, and environment. We develop the Cell Graph Transformer (CGT), a virtual-cell architecture in which biological interaction graphs constrain multi-head attention and an equivariant perturbation operator feeds multitask decoders. In *Saccharomyces cerevisiae*, CGT predicts trigenic gene-gene interactions (Pearson $\boldsymbol{r = 0.454}$), outperforming DANGO, DCell, and Yeast9 flux balance analysis, and generalizes to knockout expression ($\boldsymbol{r = 0.543}$) and morphology ($\boldsymbol{r = 0.619}$). CGT recommends gene deletions for beta-carotene and betaxanthin production, pairing with autonomous biofoundries for AI-guided strain engineering.

Keywords: metabolic engineering, foundation models, gene interactions, virtual cell, knowledge graph, strain design

Manuscript outline & status (editing view only) ✓ ready ■ author-reviewed, keep stable ✗ not done, agents may edit freely

Convention: each Results section = 3–4 paragraphs, high-level paragraph (P1) first; one main figure per section.

Main text

- ■ **Introduction**
- ✗ **Results** *(six sections, one per figure)*
  - ✗ **R1 – Shared ontology and knowledge graph unify data**
    - ■ Fig 1 – TorchCell + CGT overview (data a–c; model d–h)
    - ✗ P1 *(high level):* data fragmentation → one shared schema
    - ✗ P2: Biolink ontology, ingest-time validation, Neo4j KG, datasets-as-queries
    - ✗ P3: reference-cell + perturbation factorization; join/aggregation shrinks labels
    - ✗ P4: information budget (10^{10} vs 10^{8} bits); cut at H; baselines saturate fitness
  - ✗ **R2 – CGT predicts trigenic gene–gene interactions (state of the art)**
    - ✗ Fig 2 – gene–gene interactions + iBioFoundry design loop
    - ✗ P1 *(high level):* interactions are the hard structured label; CGT in one line
    - ✗ P2: trigenic interaction defined (peel off lower-order effects)
    - ✗ P3: results – SOTA vs Yeast9 FBA / DCell / DANGO (r=0.454)
    - ✗ P4: data/model scaling, graph-reg ablation, accuracy vs $|\tau|$
  - ✗ **R3 – One embedding, many phenotypes**
    - ✗ Fig 3 – expression, morphology, fitness
    - ✗ P1 *(high level):* one latent embedding generalizes across phenotypes
    - ✗ P2: expression prediction (r≈0.54) + diagnostics
    - ✗ P3: single-knockout morphology (r≈0.62)
    - ✗ P4: fitness + multitask sharing; why one embedding helps
  - ✗ **R4 – Natural genetic variation vs. model-designed perturbations**
    - ✗ Fig 4 – natural genetic variation
    - ✗ P1 *(high level):* natural variation vs designed perturbations
    - ✗ P2: natural strain variation across genome + environment
    - ✗ P3: model-designed (in-silico) gene perturbations
    - ✗ P4: comparison; what design adds beyond the natural range
  - ✗ **R5 – Drug exposure and representation robustness**
    - ✗ Fig 5 – drug exposure + robustness *(paper tag `r-robust`)*
    - ✗ P1 *(high level):* operator extends to drug/env; representation is robust
    - ✗ P2: drug-exposure prediction
    - ✗ P3: robustness – stable across splits; embedding + graph ablations
    - ✗ P4: graceful degradation as supervision shrinks
  - ✗ **R6 – Metabolism and strain design**
    - ✗ Fig 6 – metabolism + metabolic engineering *(paper tag `r5`)*
    - ✗ P1 *(high level):* ground the representation in metabolism; recommend targets
    - ✗ P2: metabolism integration (Yeast9 / gene–reaction)
    - ✗ P3: strain-design recommendations + iBioFoundry DBTL loop
    - ✗ P4: inference at scale
- ✗ **Discussion**
- ✗ **Methods** *(subsection status inline in `methods.tex`)*

Supplementary Information

- ■ **Note 1 – Functional equivalence of the perturbation operator**
  - ✗ Supplementary Fig. 1 – empirical equivalence (CGT- vs GNN-style, KO fitness)
  - ■ Setting · Concrete example · Origin of approximation · Precision · Relation to prior work · Empirical validation
- ■ **Note 2 – Graph-regularized attention contains graph attention**
  - ✗ Supplementary Fig. 2 – graph-reg relevance (λ sweep)
  - ■ Setting · Precision · Empirical validation

■ **Note 3 – Static-graph assumption and learning the perturbation**

■ Supplementary Table 1 – per-batch compute (two routes)
■ Static-graph assumption · Limitations · Single general data object · Compute & complexity · Empirical cost · Trade-offs

■ **Note 4 – Gene interactions as a learned function of fitness**

✗ Supplementary Fig. 3 – do interactions emerge from fitness?
■ Setting · Toward a formal statement · What would be surprising · Implication for metabolism · The atomic unit of data

✗ **Note 5 – Persistent entities and contingent observations** *(theory of the cut; Fig. 1c)*

✗ Supplementary Table 2 – persistent-entity corpora, measured from the public archives by `experiments/016-information-accounting`
✗ Setting · The measure (Prop.: compressed length bounds, not entropy) · The two corpora · Two asymmetries · Consequence · Objection · Precision
✗ *Open:* contingent-corpus gzip figure still computed on gilahyper – commit that script and regenerate Supplementary Table 15 with a compressed-size column

✗ **Note 6 – Classical-ML case study: representation & pooling** *(empirical counterpart of Note 5)*

✗ Supplementary Fig. 4 – classical-ML baselines (compose from `*_spearman_spearman_*_palette` bars + `node_embedding_performance_*_palette`) · Supplementary Table 3–5 (Spearman, MSE, Pearson)
■ Setup · Representation is identity · The barrier is pooling

✗ **Note 7 – Gene–gene graphs used as attention priors** *(the nine graphs of experiment 010; STRING releases)*

✗ Supplementary Fig. 6 – each graph on its own: sizes + union row (45 genes in no graph), degree, structure, top-3 hubs, other components (Yeast9, essential, trigenic panel, shared reaction/subsystem/GO process) · Supplementary Fig. 7 – how the graphs relate: Jaccard, containment, shared pairs, multiplicity, regulatory vs TFLink; STRING release drift moved to Supplementary Fig. 8a (`experiments/010-kuzmin-tmi/scripts/graph_statistics.py`) · Supplementary Table 6–7
✗ Sources · Size, overlap, and structure · Hubs and the two transcription-factor graphs · STRING releases

✗ **Note 8 – Constructing DANGO in TorchCell and reproducing its published result** *(Kuzmin 2018 screen alone; 19 runs at $r = 0.42$; the lambda fall-through)*

✗ Supplementary Fig. 8 – a STRING release drift (motivates the λ rule), b model in the shared notation (MathJax), c decreased zeros, d best r by release and schedule, e curves (`experiments/005-kuzmin2018-tmi/scripts/dango_construction_si.py`, `dango_string_version_sweep.py`) · Supplementary Table 8
✗ Model in the manuscript's notation · Per-channel zero weights · Dataset, result, and what is not matched ✗ *Open:* DANGO reference key `zhangDANGOPredictingHigherorder2020` not yet in `references.bib`

✗ **Note 9 – DANGO on the full trigenic dataset** *(the Fig. 2d value; canonical statement of the 006-vs-010 build mismatch)*

✗ Supplementary Fig. 9 – curves, best r by release, epochs to $r \geq 0.35$, data effect (91k Kuzmin 2018 build vs 332k 006 build: 0.42 vs 0.36); panels e–g are placeholders needing a GPU run (`experiments/010-kuzmin-tmi/scripts/dango_full_dataset_si.py`) · Supplementary Table 9
✗ Dataset builds · Training and result · Release and dataset effects ✗ *Open:* no DANGO or DCell run on the experiment-010 build (the like-for-like baseline)

✗ **Note 10 – DCell in TorchCell: the ontology-structured model** *(GO DAG as the model graph; the rebuilt 2,655-subsystem ontology)*

✗ Supplementary Fig. 10 – rendered filtered ontology (2,655 subsystems, 13 strata) with a real Kuzmin 2018 triple traced to the root, MathJax equations, subsystems per stratum, genes per subsystem, subsystems per gene (`experiments/006-kuzmin-tmi/scripts/dcell_model_go_stats.py`) · Supplementary Table 10
✗ The graph is the ontology · Building and measuring the ontology ✗ *Open:* DCell (`maUsingDeepLearning2018`) and Gene Ontology reference keys not yet in `references.bib`

✗ **Note 11 – DCell training cost, instability, and the checkpoint-based baseline** *(one 30.8-day run; the Fig. 2d error bar is checkpoint spread, not replicates)*

✗ Supplementary Fig. 11 – validation curve with the three checkpoints, losses, GPU-hours and throughput vs DANGO and CGT, speed-up stage table, CPU per-op profile of one step

(644,603 aten ops), data effect (005 vs 006 build, one run each); GPU profile still to do (`experiments/006-kuzmin-tmi/scripts/dcell_training_wandb.py`) · Supplementary Table 12–13
✗ Training run and checkpoint statistic · Cost and where the time goes

✗ **Note 12 – Yeast9 flux-balance baseline for trigenic interactions**

✗ Supplementary Fig. 12 – pipeline schematic, tau and fitness hexbins, growth bands, model coverage; panel f placeholder for the rerun with the screen's SD/MSG medium at 26 C (`experiments/007-kuzmin-tm/scripts/fba_baseline_si.py`) · Supplementary Table 14
✗ Model and medium · Deletion simulation · From growth to interaction · Result (frozen file gives $r = 0.0019$, not the 0.0006 in Fig. 2d) · Caveats (medium, 26 C, KG query filters YEPD 30 C)

■ **General SI figures** (follow the Notes)

■ Supplementary Fig. 13 – TorchCell overview schematic ■ Supplementary Fig. 14 – ontology data model (hourglass)
■ Supplementary Fig. 15 – KG normalization (convert/dedup/aggregate) ■ Supplementary Fig. 16 – DNA sequence selection
■ Supplementary Fig. 17 – DNA tokenization ✗ Supplementary Fig. 18 – expression & multitask diagnostics
✗ Supplementary Fig. 19 – inference at scale ■ Supplementary Fig. 20 – guilt-by-association annotation

■ **SI tables**: ■ Supplementary Table 1 compute ✗ Supplementary Table 2 entity corpora ✗ Supplementary Table 3–5 classical-ML ■ Supplementary Table 15 datasets ✗ Supplementary Table 6 graphs (now script-generated; counts changed) ✗ Supplementary Table 7 STRING releases ✗ Supplementary Table 8 DANGO by release ✗ Supplementary Table 9 DANGO full-dataset runs ✗ Supplementary Table 10 DCell GO filter ✗ Supplementary Table 12 DCell checkpoints ✗ Supplementary Table 13 training cost

✓ready ■author-reviewed, keep stable ✗not done, agents may edit freely

Contents

1 Introduction (target 570 words) [362/570] ■

Mapping genotype to phenotype is a central goal of biology and the rate-limiting step of metabolic engineering, where the targets of interest – titre, rate, and yield – are polygenic phenotypes that emerge from many genes acting across transcription, translation, and metabolism [1, 2]. Unlike monogenic disorders, such phenotypes cannot be read from a single gene: their causal structure is obscured by epistasis and by the cellular layers that separate genotype from measurement [3, 4]. Growth is the most tractable of them – assays are cheap and scalable, gene–gene interactions are directly quantifiable, and the same readout spans applications from suppressing proliferation in disease to maximizing growth-coupled production.

Deep learning has transformed protein structure and function prediction, yet systems biology and metabolic engineering lag [5]. Two obstacles recur. First, phenotype data are scattered across small, heterogeneous studies with no shared schema, and broad genome-wide screens rarely interoperate with the deep, pathway-focused campaigns of metabolic engineering [6, 7]. Second, although multimodal measurements improve prediction, combining more than a few modalities is difficult and rarely scales [8]. Recent foundation cell models pretrain on single-cell gene-expression matrices and fine-tune for phenotype [9, 10], but industrial microbes lack such data; what they do have is diverse perturbation data and curated interaction graphs, including metabolism.

Existing approaches leave this gap open: constraint-based flux analysis captures metabolism but misses epistasis [2, 11], and recent perturbation transformers do not consistently beat linear baselines [12]. Our own classical baselines make the point sharply – with enough data they predict knockout fitness from almost any gene representation, yet fail on gene interactions (Supplementary Fig. 4) – so fitness alone does not justify a deep model, but the harder, structured labels do.

Here we present TorchCell, a schema-grounded knowledge graph that curates heterogeneous *S. cerevisiae* phenotype data into queryable multimodal datasets, in the data-standardization spirit of TorchGeo [13], together with the Cell Graph Transformer (CGT): a model that encodes a cell once, applies perturbations as a learned operator in embedding space, and folds biological networks into the training objective as a soft prior rather than a hard graph. It predicts fitness, gene interactions, gene expression, and morphology jointly, and needs only a genome – not an interaction graph – to run. We outline the data model and architecture in Supplementary Fig. 13.

2 Results (target 1,900 words) [679/1900] ✗

A shared ontology and knowledge graph unify metabolic-engineering data (380 words) [457/380] ✗**.** TorchCell confronts the data-fragmentation barrier by unifying heterogeneous phenotype data under a single schema and exposing it as deep-learning-ready datasets. Each experimental record is typed by an experiment ontology grounded in the Biolink Model and validated at ingestion – for acyclicity, subtype substitutability, Biolink conformance, and referential integrity – so that records from unrelated studies share one machine-checkable structure (Fig. 1a; ontology data model in Supplementary Fig. 14). Validated records are emitted as typed subgraphs into a Neo4j knowledge graph hosted at NCSA, and datasets are recovered as queries over this shared structure: a data-specific query returns one instance with its typed neighborhood, while a cross-experiment query joins studies through shared entities such as a common environment or genotype. This turns modular literature and public databases into interoperable, queryable datasets *[placeholder: TorchCell currently spans ∼N studies and ∼M labeled instances across fitness, gene interaction, expression, and morphology; dataset scale in Supplementary Table 15, integrated graphs in Supplementary Table 6].*

Rather than store a genome per strain, TorchCell factorizes data as a shared reference cell acted on by a perturbation (Fig. 1b). Each experiment is a tuple of a cell graph, an environment, a perturbation, and a phenotype; the minimal reference is a genome, over which optional gene, regulatory, protein-interaction, and metabolic networks add structure. The yeast knockout library, for instance, is a single reference with $\sim 10^{4-6}$ deletion strains, each represented as a perturbation over that reference rather than as an independently rebuilt graph. A mechanistic join-and-aggregation step then merges compatible records – recasting lethality to fitness, or pooling equivalent genotypes measured under different assays – enlarging datasets while shrinking the label vocabulary the model must decode (Supplementary Fig. 15).

This organization exposes a favorable information budget (Fig. 1c). The cell's molecular parts – DNA, RNA, protein, and small molecules – are *persistent entities*: a sequence is the same object in a glucose assay and in an oxidative-stress assay, and the public archives already hold $\sim 10^{13}$ compressed bits of them (Supplementary Table 2). Measured phenotypes are *contingent observations*, values pinned to one genotype, environment, and assay; the corpus integrated here amounts to $\sim 10^{10}$ compressed bits under the same measure. The two therefore differ by three orders of magnitude in scale and, more consequentially, in reuse: an entity representation serves every instance in which that entity appears, whereas a phenotype record constrains the map at one context and no other. We therefore cut the model at the shared representation, taking entity encoders already trained on the abundant side and spending the scarce labels only on the perturbation operator and the decoders (Supplementary Note 5). Consistent with this budget, classical baselines

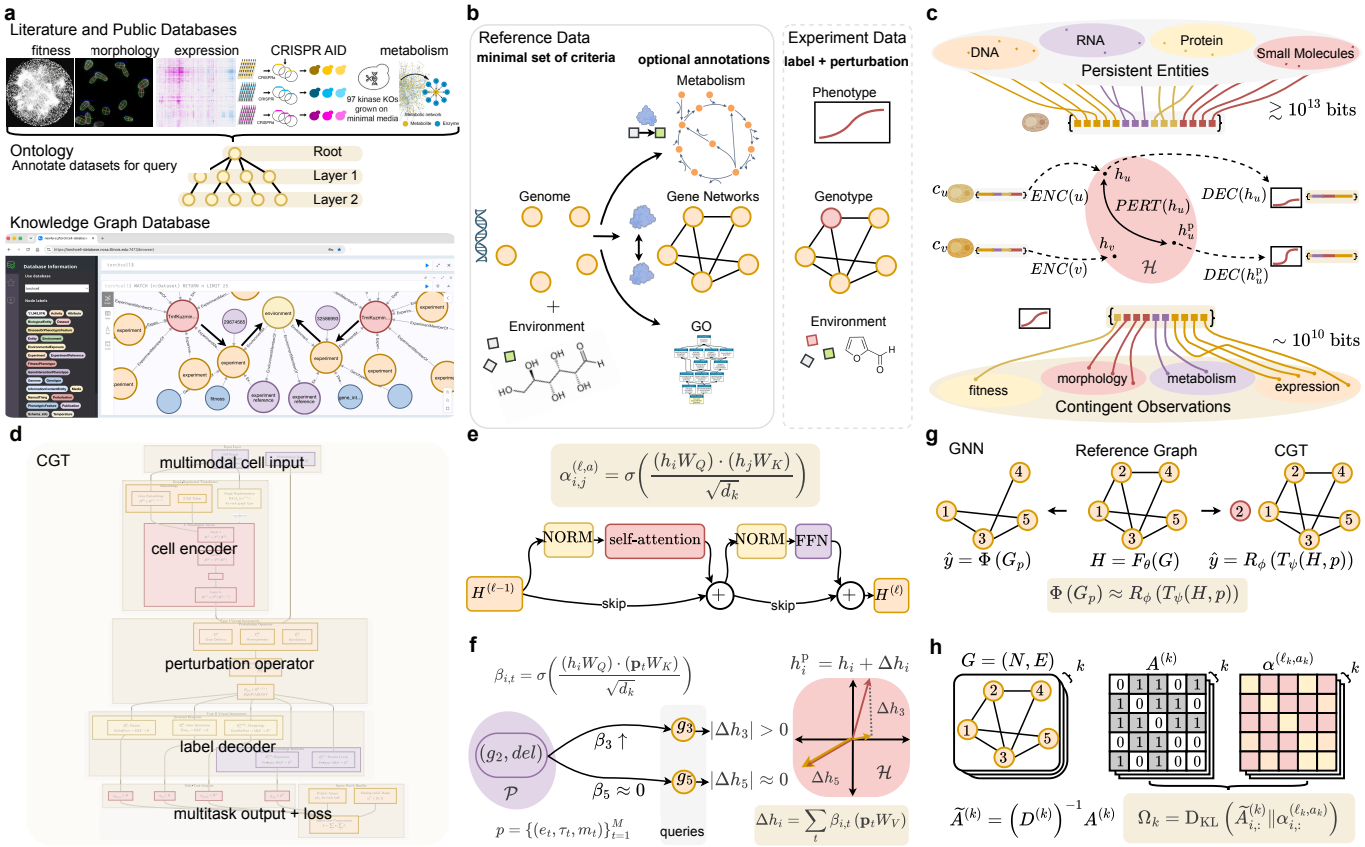


Fig. 1 TorchCell and the Cell Graph Transformer (CGT). **Top row – data. a**, Literature and public databases are annotated with TorchCell's Biolink-grounded experiment ontology and ingested into a Neo4j knowledge graph (hosted at NCSA), queryable across datasets. **b**, Each experiment is a shared reference cell (genome, gene networks) acted on by an experiment-specific perturbation (genotype, environment) with a measured phenotype; many strains reuse one reference. Gene, regulatory, and protein-interaction networks form a multigraph over genes; metabolism and Gene Ontology form bipartite graphs that can be mapped into the model or held aside as verification artifacts (for example, to test whether the model captures Gene Ontology it was not given; Methods). **c**, Information accounting: molecular parts (DNA, RNA, protein, small molecules) enter as *persistent entities* – identities that do not change between assays, and whose public archives carry $\gtrsim 10^{13}$ compressed bits – and are mapped by encode (F_θ), perturb ($\mathcal{T}_\psi$), and decode ($\mathcal{R}_\phi$) into *contingent observations*, phenotypes measured in one genotype, environment, and assay, of which we integrate $\sim 10^{10}$ compressed bits. Bit counts are compressed lengths of the distributed archives, an upper bound on description length, not an information content. The model is cut at the shared representation H. Supplementary Note 5 gives the full accounting for this panel and states precisely what the bit counts do and do not mean; Supplementary Table 2 reports the measured size of every archive summed here, with its source and release. **Bottom row – model. d**, CGT overview: multimodal cell input → cell encoder (ENC) → perturbation operator (PERT) → multitask decoder (DEC) → per-task outputs and loss. **e**, Cell-encoder self-attention block: scaled dot-product attention $\alpha^{(\ell,a)}$ with pre-norm, feed-forward, and residual connections. **f**, Perturbation operator: a soft cross-attention $\beta_{i,t}$ over perturbation tokens displaces each gene embedding, $h_i^{\mathrm{pert}} = h_i + \Delta h_i$; the same operator extends from gene deletions to drug and environment perturbations. **g**, Functional equivalence: applying the operator to the reference representation approximates re-encoding the perturbed graph with a graph network, $\Phi(G_p) \approx \mathcal{R}_\phi(\mathcal{T}_\psi(H, p))$, so a strain is obtained without rebuilding or re-encoding its graph. The reference is encoded once and reused, and only the small-parameter operator runs per perturbation, amortizing compute across strains (proof and compute accounting in Methods and Supplementary Note 1). **h**, Graph-regularized attention: each biological adjacency $A^{(k)}$ is row-normalized to $\tilde{A}^{(k)} = (D^{(k)})^{-1}A^{(k)}$ and matched to an attention head by a KL prior Ω_k that biases the head toward the biological graph: a soft constraint, not a hard mask that forces it, so supervision can overrule an edge the labels contradict (Methods and Supplementary Note 2).

predict knockout fitness from almost any representation yet fail on gene interactions (Supplementary Fig. 4) – so fitness alone does not justify a deep model, but the harder, structured labels do, motivating the architecture that follows.

The Cell Graph Transformer predicts trigenic gene-gene interactions at state of the art (380 words) [53/380] ✗. *[FILLER – R2.]* The Cell Graph Transformer (CGT; architecture in Fig. 1c and Methods) constrains attention with biological interaction graphs and maps the reference cell through an equivariant perturbation operator. On trigenic gene-gene interactions (GGI), where classical ML fails (Supplementary Fig. 4), CGT reaches Pearson $r = 0.454$ (Spearman 0.421), outperforming Yeast9 FBA, DCell, and DANGO (Fig. 2).

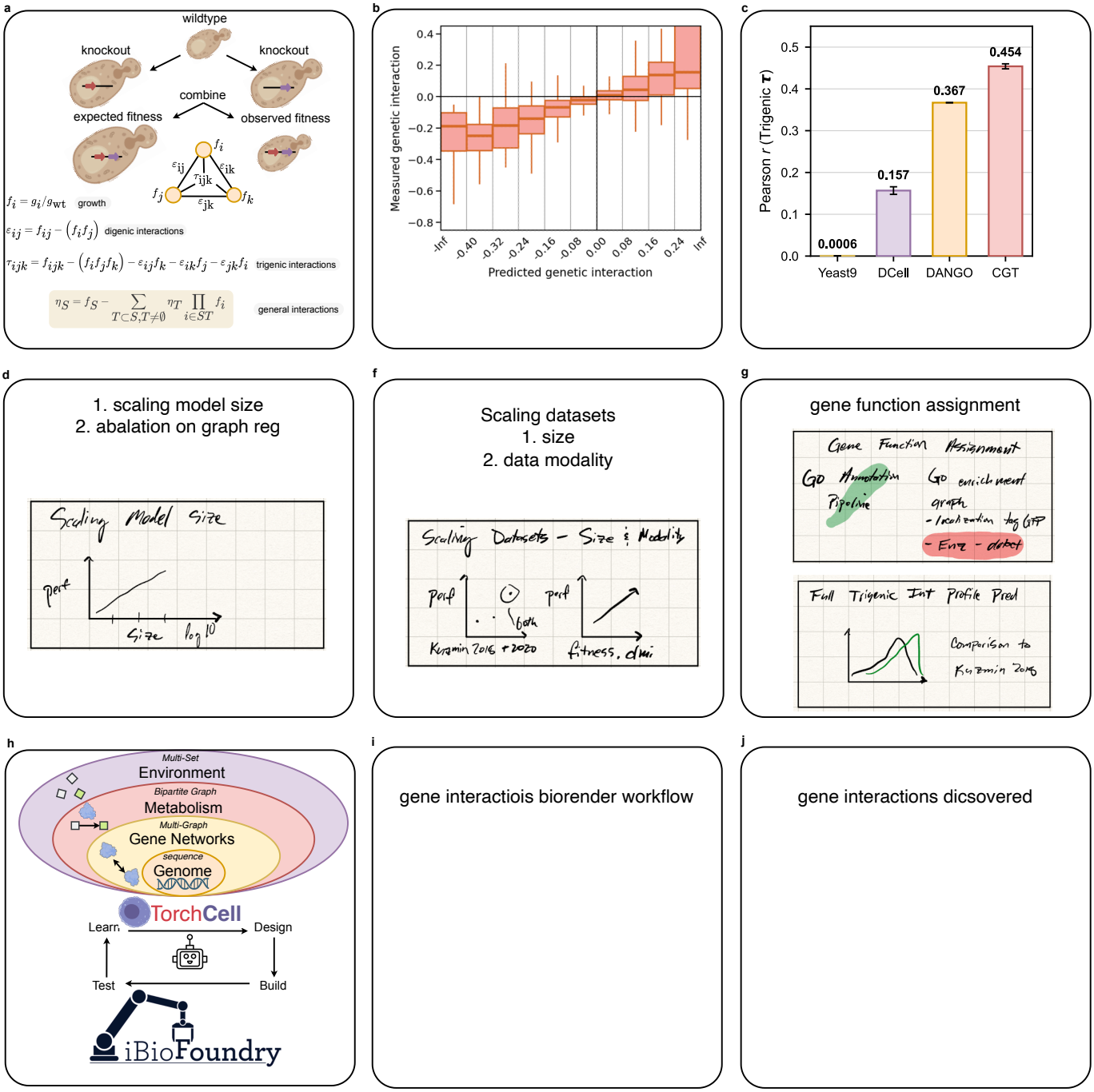


Fig. 2 TorchCell for gene–gene interaction prediction and strain design. **a**, Higher-order genetic interactions are defined by subtracting lower-order effects: to isolate a k-way interaction you must peel off all interactions from subcollections of the same genes. For a trigenic interaction, the observed triple-mutant fitness is compared against the fitness expected from the single- and double-mutant terms, $\tau_{ijk} = f_{ijk} - f_i f_j f_k - \varepsilon_{ij} f_k - \varepsilon_{ik} f_j - \varepsilon_{jk} f_i$, with digenic interactions $\varepsilon_{ij} = f_{ij} - f_i f_j$. **b**, The nested cell representation – genome, gene networks, metabolism, and environment – feeds TorchCell and a Learn–Design–Build–Test loop with the NSF iBioFoundry. **c**, Predicted versus measured trigenic gene–gene interaction. **d**, State of the art on trigenic interactions: Pearson r for Yeast9 FBA (0.0006), DCell (0.157), DANGO (0.367), and TorchCell (0.454).

One embedding, many phenotypes: expression, morphology, and fitness (380 words) [20/380] ✗. *[FILLER – R3.]* The same embedding predicts knockout expression ($r = 0.543$), single-knockout morphology ($r = 0.619$), and fitness (Fig. 3; expression diagnostics in Supplementary Fig. 18).

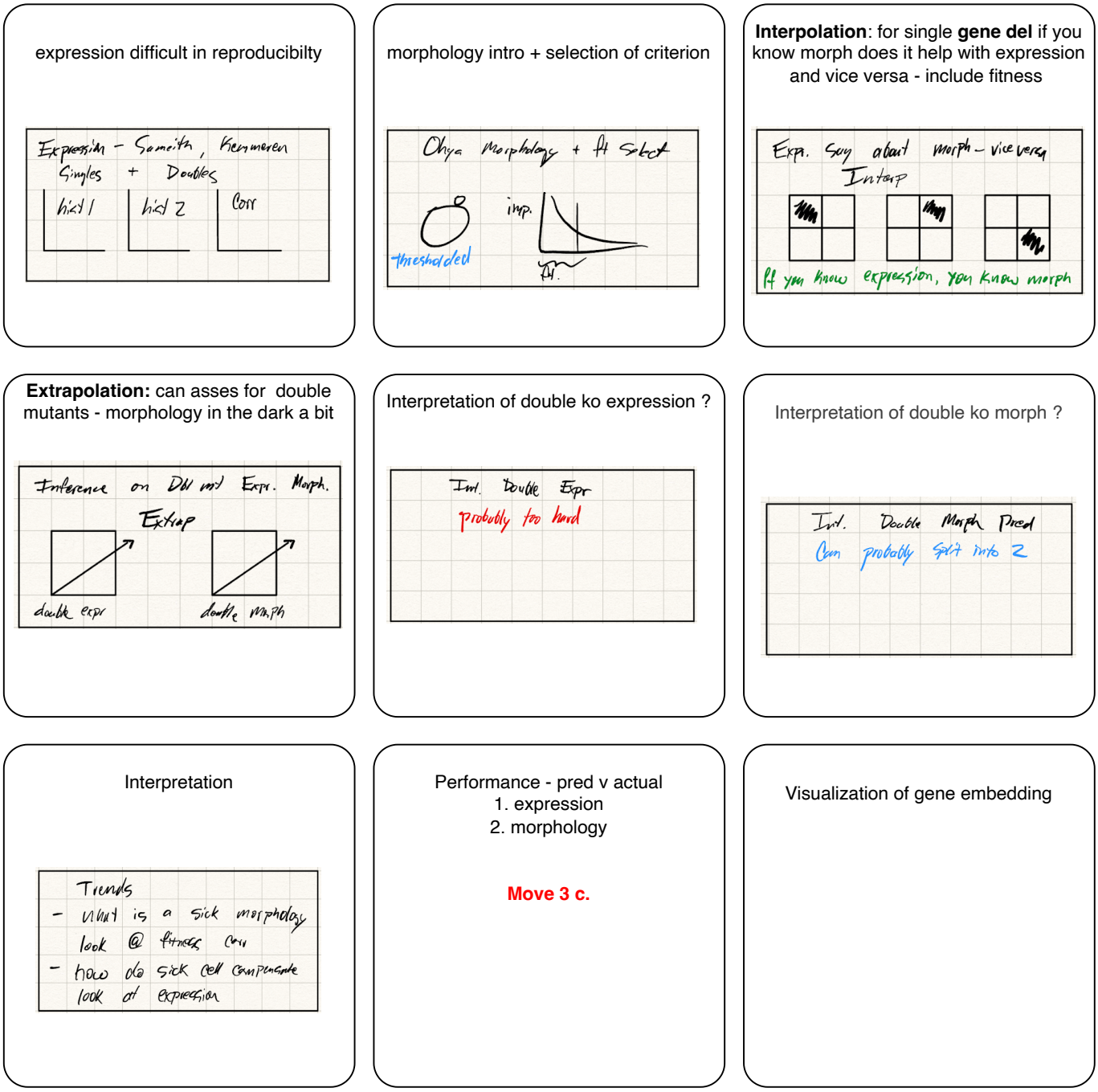


Fig. 3 One latent cell embedding generalizes across systems-biology phenotypes – expression, morphology, and fitness. **a**–**e**.

Natural genetic variation versus model-designed perturbations (380 words) [21/380] ✗. *[FILLER – R4.]* We compare phenotypes from natural strain genetic variation against deep-learning-designed gene perturbations across genome and environment (Fig. 4).

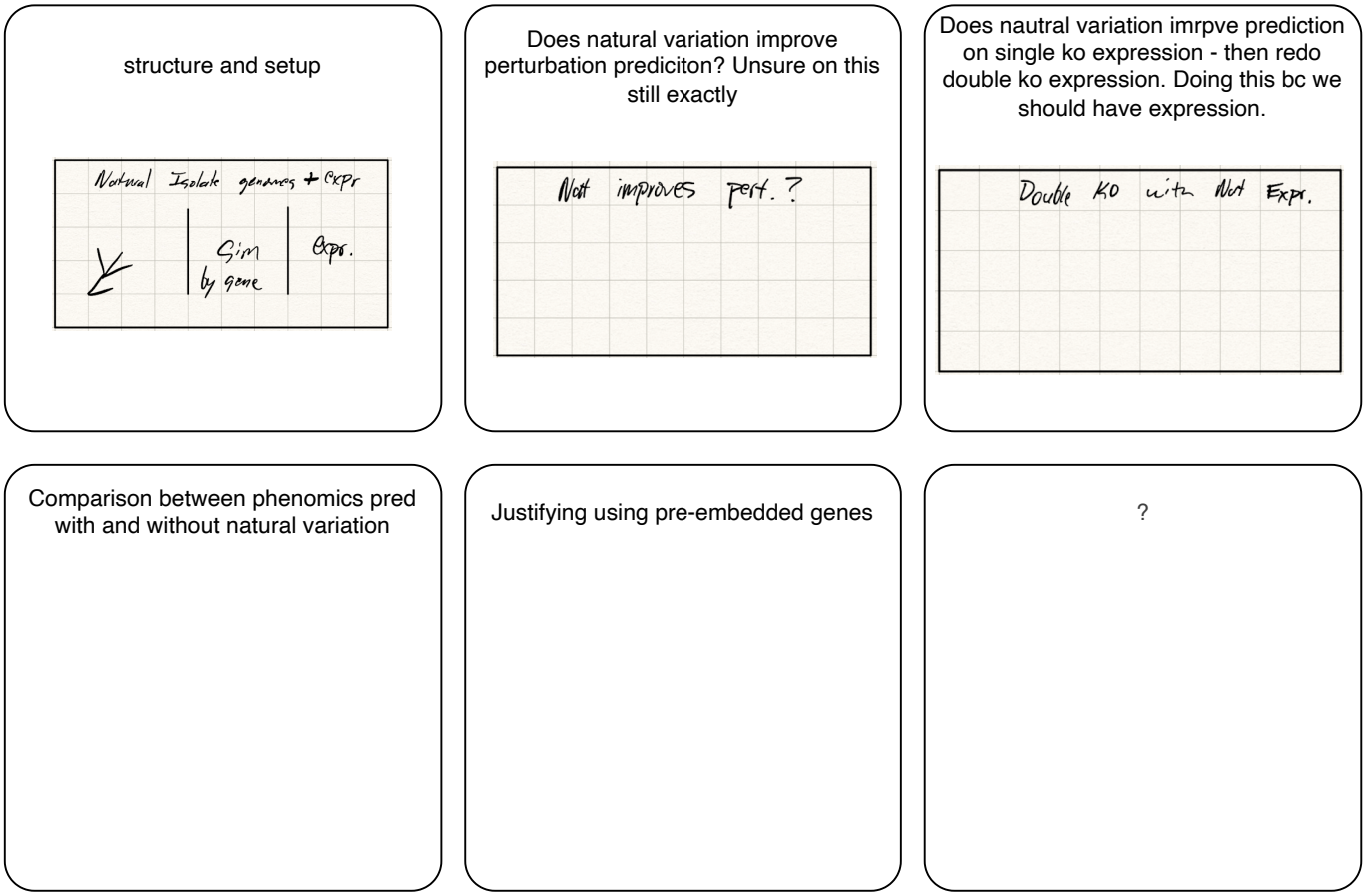


Fig. 4 Natural genetic variation versus model-designed perturbations. **a**–**c**.

Drug exposure and robustness of the learned cell representation (380 words) ✗**.** *[FILLER – R-robust.]* The perturbation operator extends from gene deletions to drug and environment exposure, and the learned cell representation is robust: predictions stay stable across dataset splits, tolerate embedding and interaction-graph ablations, and degrade gracefully as supervision is reduced (Fig. 5).

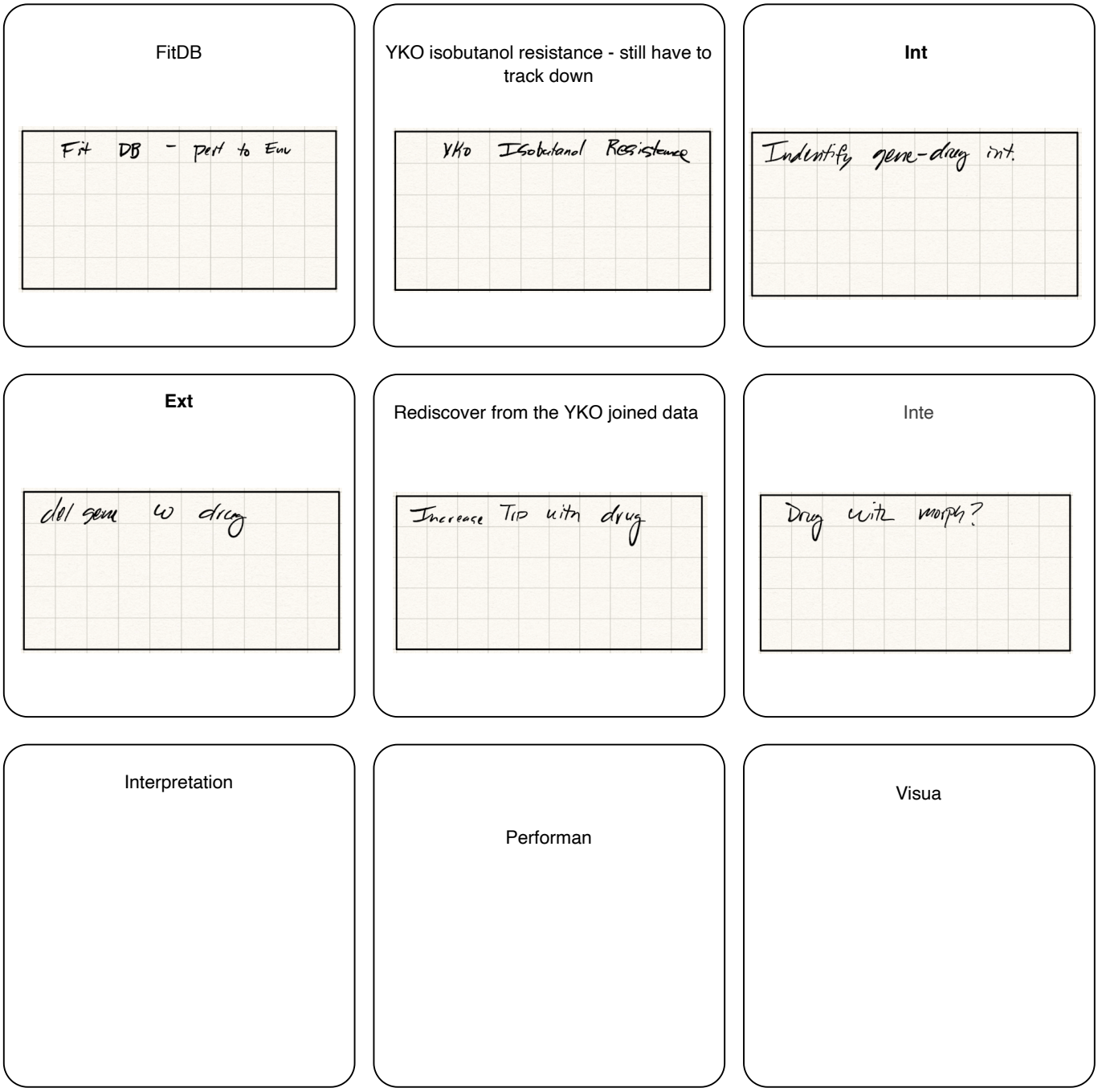


Fig. 5 Drug exposure and robustness of the learned cell representation. **a**–**f**.

Extending to metabolism and strain design (380 words) [42/380] ✗. *[FILLER – R5.]* Grounding the cell representation in metabolism, CGT recommends gene targets for production phenotypes and pairs with the UIUC iBioFoundry for iterative design (Fig. 6; inference at scale in Supplementary Fig. 19).

Fig. 6 Extending TorchCell to metabolism and metabolic-engineering strain design. **a**–**f**.

3 Discussion (target 570 words) [170/570] ✗

[FILLER – Discussion.] A strain-aware, biologically grounded representation is the differentiator; limitations include single-organism scope, morphology-data maturity, and error-bar provenance.

A central modeling assumption deserves explicit statement. TorchCell treats the catalogued biological networks (Fig. 1c) as annotations of a fixed reference and assumes the wiring is perturbation-invariant, so a strain is a thin operator applied to the reference rather than a rebuilt graph. This is a hypothesis, not a fact: an edge can effectively vanish when, for example, perturbing an upstream regulator drives a partner's expression so low that the interaction no longer occurs in the cell, and such condition-specific rewiring is not captured by a static adjacency. We adopt the assumption deliberately – it makes the perturbation a learnable displacement in representation space, keeps the biological graphs as a soft prior the data can overrule rather than a hard substrate, and yields a single general data object that different modeling paradigms can consume – and we treat its biological limits, computational advantages, and pipeline generality at length in Supplementary Note 3.

Table 1 Notation. The ENC/PERT/DEC stages (Fig. 1c) are implemented for CGT as $F_\theta/\mathcal{T}_\psi/\mathcal{R}_\phi$.

Symbol	Meaning
Sets and indices	
$\mathcal{U},\ \mathcal{I}$	encoded entities; labeled instances, $\mathcal{I} = \operatorname{supp} D$
$\mathcal{Y}$	phenotype space; y observed, $\hat{y}$ predicted
$\mathcal{P}$	perturbation space, $p \in \mathcal{P}$
N	number of nodes (genes); $N = 6607$, indexed $g_1, \dots, g_N$
$k,\ \ell,\ a,\ B$	relation type, layer, head, batch size
Graphs	
$G = (N, E)$	cell graph on N nodes with edge set E; relation-k adjacency $A^{(k)} \in \{0,1\}^{N\times N}$
$\tilde{A}^{(k)}$	row-normalized adjacency
Transformer	
H	cell representation $(h_{\mathrm{CLS}}, h_1, \dots, h_N)$, $h_i \in \mathbb{R}^d$
W_Q, W_K, W_V	query/key/value maps; head width $d_k = d/h$
$\alpha^{(\ell,a)}$	encoder self-attention map (rows sum to 1)
$\beta_{i,t}$	operator cross-attention weights
Constraint-based metabolism	
$S,\ v$	stoichiometric matrix $\in \mathbb{R}^{m\times r}$, flux; $Sv = 0$
ρ	gene–reaction association (GPR), gene $\mapsto 2^{[r]}$
Model	
Embed	entity encoder $\mathcal{U} \to \mathbb{R}^d$, or lookup $\mathbf{E}$
F_θ (ENC)	cell encoder, $H = F_\theta(G)$
p	perturbation set $\{(e_t, \tau_t, m_t)\}_{t=1}^M$: entity, type, magnitude; token $\mathbf{p}_t$, matrix $\mathbf{P} = (\mathbf{p}_1, \dots, \mathbf{p}_M)$ (keys/values)
ε	environment (media, temperature)
$\mathcal{T}_\psi$ (PERT)	perturbation operator, $H_{\mathrm{pert}} = \mathcal{T}_\psi(H, p)$
$\mathcal{R}_\phi$ (DEC)	decoder, $\hat{y} = \mathcal{R}_\phi(H_{\mathrm{pert}}, \varepsilon)$
$\Theta,\ \mathcal{L},\ D$	parameters (θ, ψ, ϕ), loss, data law over (G, ε, p, y)

4 Methods (no limit) [2479] ✗

Problem setup

We cast multimodal phenotype prediction as supervised learning of a single map from a cell to its phenotype. A model $\hat{f}_\theta$ reads a cell graph G, an environment ε, and a perturbation p and predicts a phenotype $\hat{y} = \hat{f}_\theta(G, \varepsilon, p)$, which we fit by minimizing the expected loss over the data distribution D,

$$\hat{\Theta} = \arg\min_{\Theta}\ \mathbb{E}_{(G,\varepsilon,p,y)\sim D}\big[\mathcal{L}\big(\hat{f}_\theta(G, \varepsilon, p),\ y\big)\big]. \tag{1}$$

The contribution is the structure of $\hat{f}_\theta$, which factors into encode, operate, and decode,

$$\hat{f}_\theta(G, \varepsilon, p) = \mathcal{R}_\phi\big(\mathcal{T}_\psi(F_\theta(G), p),\ \varepsilon\big), \tag{2}$$

so that abundant entity data trains the encoder F_θ while the scarce phenotype labels need only fit the perturbation operator $\mathcal{T}_\psi$ and the decoders $\mathcal{R}_\phi$. The notation, the data, and each factor are specified below.

Notation

We fix one notation, chosen to stay compatible with three literatures – machine learning on graphs, transformers, and constraint-based metabolic modeling – and to avoid the symbol collisions between them. Calligraphic letters denote spaces and learned operators. We cast the model as a three-stage framework – encode (ENC), perturb (PERT), decode (DEC), the stage names in Fig. 1c – instantiated for CGT by the encoder F_θ, perturbation operator $\mathcal{T}_\psi$, and decoder $\mathcal{R}_\phi$ (Table 1): ENC/PERT/DEC name the concept, $F_\theta/\mathcal{T}_\psi/\mathcal{R}_\phi$ the CGT implementation. Table 1 is the full glossary.

Ontology-grounded schema and knowledge graph

Every experimental record is typed by a schema Σ grounded in the Biolink Model: each class maps to a Biolink category and each relation to a Biolink predicate. At ingestion, records are validated programmatically against properties that keep the schema machine-checkable:

- *acyclicity* – Σ is a directed acyclic graph, so type and relation dependencies have no cycles and ingestion terminates;
- *Liskov substitutability* – each subtype is usable wherever its supertype is, so the class hierarchy is sound and a query on a supertype returns its subtypes;
- *Biolink conformance* – each class and relation resolves to a Biolink category/predicate, with field- and cardinality-level constraints;
- *referential integrity* – each typed edge connects declared node types.

Σ acts as an hourglass whose narrow waist forces heterogeneous sources into one typed structure. A raw record x is validated and emitted as a typed subgraph $\iota(x)$ into a Neo4j knowledge graph $\mathcal{K}$. Datasets are then queries over a shared structure: a *data-specific* query returns one instance with its nearest typed edges, while a *cross-experiment* query returns all instances sharing a structural entity (e.g. a common environment), joining studies through shared schema entities (Fig. 1a). This homogenizes modular literature and public data into queryable datasets, addressing the data-fragmentation barrier to deep learning in metabolic engineering (Supplementary Figs. 14, 15).

Datasets: a reference acted on by a perturbation

Each experiment is a tuple (G, ε, p, y) (Fig. 1b). The minimal *reference* is a genome – a sequence (FASTA) and an annotation (GFF) giving gene positions and sequence features; this alone is a valid record. Optional annotations add structure over that genome – gene, regulatory, and protein-interaction networks $\{A^{(k)}\}$, metabolism (the bipartite graph from (S, ρ)), and Gene Ontology – and are optional precisely because they presuppose the genome. Data factorize as *few references, many perturbations*: rather than store a genome per strain (the yeast knockout library is one reference and $\sim 10^{4-6}$ deletion strains), we store a shared reference G and a perturbation p over it, while natural isolates or divergent genomes become new references. An experiment additionally specifies a perturbation to the genome (p, the genotype) and possibly the environment ($\Delta\varepsilon$, as in a drug or tolerance study), together with a phenotype y. The phenotype may be a scalar such as fitness, a vector such as a gene-level expression or protein-abundance profile, or a tensor such as a set of morphology traits or metabolite concentrations, and the formulation is agnostic to its tensor rank. A mechanistic, explainable join/aggregation step merges instances across compatible conditions (e.g. across environments, or recasting lethality to fitness), enlarging datasets and shrinking the decoder's output domain $\mathcal{Y}$ to fewer symbol types (Supplementary Fig. 15).

Encoded entities

The cell's molecular parts – DNA, RNA, protein, and small molecules such as metabolites and drugs – form the universe $\mathcal{U}$ of *encoded entities*: each entity arrives with a representation $\mathrm{Embed} : \mathcal{U} \to \mathbb{R}^d$, supplied by a self-supervised model (a DNA, RNA, or protein language model, or a small-molecule encoder), a learnable lookup $\mathbf{E} \in \mathbb{R}^{N \times d}$, or a fixed featurization. Membership asks only that an entity *can be encoded*, not that it be pretrained. These encoders represent any candidate molecule rather than only those native to the organism, and they place similar entities near one another in $\mathbb{R}^d$, so the geometry is a usable prior: entities with similar representations are expected to act similarly on the cell.

The framing is deliberately *reductionist* – a cell is described by its parts – while an instance is *holistic*, a specific cell being a collection of entities assembled under a perturbation and an environment whose phenotype is a property of the whole. We hypothesize that the phenotype of a combination of entities can be estimated from supervised data, and that it takes far less phenotype data to do so because the entities arrive with knowledge already baked in: the self-supervised models supply statistical regularities of sequence and structure, and the genome annotation that defines each entity supplies curated structural knowledge. The encoder therefore begins from informative representations rather than from scratch, moving work off the scarce phenotype labels and onto data that is abundant. This factorization is a response to the limited-data regime; were phenotype data abundant, the simpler and more general choice would be to learn the cell-to-phenotype map end to end. Abundant entity data and scarce labels meet at the representation H, where we cut the model so that the labels fit only the operator and the decoder.

A measured information accounting motivates this cut (Fig. 1c; Supplementary Note 5). Both sides are measured with one instrument, the compressed length $L_C(D) = 8|C(s(D))|$ of a corpus D under a fixed serialization s and lossless compressor C. On the entity side we take the public archives in the representation in which each is actually distributed – nucleotide (NCBI `nt`), protein (UniProtKB/TrEMBL), small molecule (PubChem), and structure (wwPDB) – whose sizes are read from HTTP `Content-Length`, BLAST metadata, and the wwPDB index without downloading any

payload (Supplementary Table 2); they total $\sim 10^{13}$ bits. A counting bound corroborates the compressor: `nt` holds 4.06×10^{12} nucleotides, which a four-letter alphabet cannot write in under 8.1×10^{12} bits, and the archive is distributed in 7.1×10^{12}. On the label side, the phenotype corpus we integrate (Supplementary Table 15) occupies $\sim 1.4 \times 10^{9}$ compressed bytes, or $\sim 10^{10}$ bits – an asymmetry of $\sim 10^{3}$.

Scale is only half the argument, and the weaker half. An entity representation is a function of the entity alone, so one sequence informs the encoder for every instance in which that entity appears; a phenotype record is pinned to a single genotype, environment, and assay, and constrains the map nowhere else. Nor are these compressed sizes usable supervision: both are upper bounds on description length (Supplementary Note 5, Proposition 5), and the labels in particular are noisy and strongly redundant – interaction matrices are approximately low rank – which places the effective supervision nearer 10^{6}–10^{7} bits than 10^{10}. That discount only sharpens the conclusion. A sequence encoder carries 10^{8}–10^{10} parameters and the phenotype labels cannot pay for it, whereas the operator and decoder together carry $\sim 5 \times 10^{4}$ and they can. This is exactly the encode/operate/decode cut.

Multimodal cell representation

A cell is represented across nested biological scales (Fig. 1d). The genome is a sequence, the gene networks form a multi-graph with K relation types and adjacencies $\{A^{(k)}\}$, metabolism is the bipartite gene–reaction graph induced by (S, ρ), and the environment is a multi-set of conditions. Each gene g_i receives a node feature $x_i = \mathrm{Embed}(g_i) \in \mathbb{R}^d$ from its sequence and modality embeddings, and a learnable whole-cell token x_{CLS} is prepended to form the input states

$$H^{(0)} = (x_{\mathrm{CLS}}, x_1, \ldots, x_N) \in \mathbb{R}^{(N+1)\times d}. \tag{3}$$

The relation adjacencies $\{A^{(k)}\}$ are not consumed as message-passing edges. They enter the model only as priors on attention (Eq. 16), so the encoder reads the cell as a set of gene tokens whose interactions are learned rather than fixed in advance.

Cell Graph Transformer: forward pass

The encoder F_θ maps the input states $H^{(0)}$ to the cell representation $H = H^{(L)}$ through L transformer layers (Fig. 1e). Layer ℓ updates every token with multi-head self-attention followed by a position-wise feed-forward network. For head $a \in [h]$ of width $d_k = d/h$, the layer forms queries, keys, and values

$$Q^{(\ell,a)} = H^{(\ell-1)} W_Q^{(\ell,a)}, \qquad K^{(\ell,a)} = H^{(\ell-1)} W_K^{(\ell,a)}, \qquad V^{(\ell,a)} = H^{(\ell-1)} W_V^{(\ell,a)}, \tag{4}$$

and computes an attention map and head output

$$\alpha^{(\ell,a)} = \mathrm{softmax}\Big(\frac{Q^{(\ell,a)} \, (K^{(\ell,a)})^\top}{\sqrt{d_k}}\Big), \qquad O^{(\ell,a)} = \alpha^{(\ell,a)} V^{(\ell,a)}. \tag{5}$$

The heads are concatenated and mixed, then combined with the feed-forward network through residual connections,

$$\hat{H}^{(\ell)} = H^{(\ell-1)} + \big[\, O^{(\ell,1)} \,\|\, \cdots \,\|\, O^{(\ell,h)} \,\big] W_O^{(\ell)}, \qquad H^{(\ell)} = \hat{H}^{(\ell)} + \mathrm{FFN}^{(\ell)}\big(\mathrm{LN}(\hat{H}^{(\ell)})\big), \tag{6}$$

with layer normalization applied before each sublayer. During training, selected heads are aligned to biological graphs (Eq. 16), and this is the only place the networks $\{A^{(k)}\}$ act on the model. Because the wildtype cell graph G is shared across the many strains built on it, the encoder runs once per reference,

$$H = F_\theta(G) = (h_{\mathrm{CLS}}, h_1, \ldots, h_N), \tag{7}$$

and every perturbation reuses this H. The reparametrization replaces a per-strain re-encoding of perturbed graphs, of cost $\mathcal{O}(B\,L\,|E|)$, with one wildtype encode of cost $\mathcal{O}(B\,L)$ followed by the cheap operator below.

Perturbation operator

A strain is the reference acted on by a perturbation, and we realize this as a transformation within embedding space rather than a re-encoding. The operator $\mathcal{T}_\psi$ applies the perturbation set $p = \{(e_t, \tau_t, m_t)\}_{t=1}^{M}$ by cross-attention, in

which every cell token is a query and the perturbation tokens are the keys and values. Each perturbation token is embedded as

$$\mathbf{p}_t = r(e_t) + \mathbf{t}_{\tau_t} + m_t\,\mathbf{w}, \qquad r(e_t) = \begin{cases} h_j & \text{if } e_t = g_j \text{ is an in-cell gene,} \\ \mathrm{Embed}(e_t) & \text{otherwise,} \end{cases} \tag{8}$$

where $\mathbf{t}_\tau$ is a learned type embedding that carries the sign of the perturbation and $\mathbf{w}$ scales it by the magnitude. With $\mathbf{P} = (\mathbf{p}_1, \ldots, \mathbf{p}_M)$, every gene attends over the perturbation context and is updated by a weighted blend of the values,

$$\beta_{i,t} = \underset{t\in[M]}{\mathrm{softmax}} \frac{(h_i W_Q)\cdot(\mathbf{p}_t W_K)}{\sqrt{d_k}}, \qquad \Delta h_i = \sum_{t=1}^{M} \beta_{i,t}\,(\mathbf{p}_t W_V). \tag{9}$$

A residual block then produces the perturbed states,

$$\hat{h}_i = \mathrm{LN}(h_i + \Delta h_i), \qquad h_i^{\mathrm{pert}} = \mathrm{LN}\big(\hat{h}_i + \mathrm{FFN}(\hat{h}_i)\big), \qquad H_{\mathrm{pert}} = \mathcal{T}_\psi(H, p). \tag{10}$$

The update is a soft blend over the whole set, since $\sum_t \beta_{i,t} = 1$, and is not a dictionary lookup. Because p is a set, the operator is permutation-invariant in the perturbation tokens and accepts any number of them, and it is equivariant in the genes, so permuting genes permutes H_{pert} in step. The same operator therefore extends from gene deletions to drug, inhibitor, and environment perturbations by widening p, with no change to the architecture (Fig. 1f).

Two advantages follow from realizing a strain as an operator rather than a graph. First, no graph is constructed per strain: the wildtype is encoded once (Eq. 7) and each of the $\sim 10^{4-6}$ strains is obtained by applying $\mathcal{T}_\psi$ to the shared H, so the data pipeline never builds or re-encodes a perturbed graph, and the operator is a learned, functionally equivalent substitute for re-encoding – applying it to the reference representation approximates re-encoding the perturbed graph with a graph network, $\Phi(G_p) \approx \mathcal{R}_\phi(\mathcal{T}_\psi(H, p))$ (Fig. 1g; proved in Supplementary Note 1) – at $\mathcal{O}(BL)$ rather than $\mathcal{O}(BL|E|)$ cost. Second, the model does not require a graph at all: genes enter as a token set and interact through learned attention, with the networks $\{A^{(k)}\}$ acting only as an optional soft prior (Eq. 16), never as a message-passing substrate. Its minimal input is therefore a genome – a set of encodable entities – so interaction networks sharpen the model where available but are not a prerequisite, which lowers the minimal data criterion for a new organism or dataset.

Multitask readouts

Each phenotype is read out of the same perturbed representation by a task head $\mathcal{R}_\phi^t$, so one latent state serves every task and the heads differ only in how they pool over genes. Invariant heads predict a whole-cell scalar. The fitness head pools all genes,

$$\hat{y}_{\mathrm{fit}} = \mathrm{MLP}_{\mathrm{fit}}\Big(\tfrac{1}{N}\textstyle\sum_{i=1}^{N} h_i^{\mathrm{pert}}\Big), \tag{11}$$

and the genetic-interaction head pools the perturbed genes and compares them against the whole-cell token,

$$\hat{y}_{\mathrm{int}} = \mathrm{MLP}_{\mathrm{int}}\Big(\big[\, h_{\mathrm{CLS}}^{\mathrm{pert}} \,\big\|\, \tfrac{1}{|p|}\textstyle\sum_{g_j\in p} h_j^{\mathrm{pert}} \,\big]\Big). \tag{12}$$

Equivariant heads predict one value per gene, as for transcript or protein abundance,

$$\hat{y}_{\mathrm{expr},i} = \mathrm{MLP}_{\mathrm{expr}}\big(h_i^{\mathrm{pert}}\big), \tag{13}$$

and gene-set heads pool over a fixed set $\mathcal{G}_s$ of genes, as for the morphology traits of a cellular structure,

$$\hat{y}_{\mathrm{morph}} = \mathrm{MLP}_{\mathrm{morph}}\Big(\tfrac{1}{|\mathcal{G}_s|}\textstyle\sum_{i\in\mathcal{G}_s} h_i^{\mathrm{pert}}\Big). \tag{14}$$

The environment ε conditions every head, and together the encoder, operator, and heads realize the predictor $\hat{y}$ of Eq. (1). Adding a phenotype is adding a head, not changing the backbone.

Data splits and preprocessing

[FILLER.] Train/validation/test partitioning (and any gene- or interaction-disjoint splits); label normalization; handling of missing labels.

Graph-regularized attention and the training objective

A biological graph and an attention map are objects of the same shape (Fig. 1h). The adjacency $A^{(k)} \in \{0,1\}^{N\times N}$ over genes matches the gene–gene block of an attention map $\alpha^{(\ell,a)} \in [0,1]^{N\times N}$, so the two can be compared row by row. We row-normalize each adjacency into a distribution over neighbors,

$$\tilde{A}^{(k)} = (D^{(k)})^{-1} A^{(k)}, \qquad D^{(k)} = \operatorname{diag}\big(A^{(k)}\mathbf{1}\big), \tag{15}$$

and align a chosen head to graph k by penalizing the divergence of each row,

$$\Omega_k = \sum_{i\in\mathcal{I}_k} D_{\mathrm{KL}}\big(\tilde{A}^{(k)}_{i,:} \,\big\|\, \alpha^{(\ell_k,a_k)}_{i,:}\big), \tag{16}$$

where (ℓ_k, a_k) is the layer and head assigned to graph k and $\mathcal{I}_k$ is a sampled set of rows. The supervised term is a multitask loss over a batch of B instances. The tasks t index the phenotype heads and b indexes the instances in the batch. Most instances carry a label for only some tasks, so a mask $m_t^{(b)} \in \{0,1\}$, equal to 1 when the label $y_t^{(b)}$ is measured and 0 otherwise, restricts each term to the labels that exist,

$$\mathcal{L}_y = \sum_t w_t \sum_{b=1}^{B} m_t^{(b)}\, \ell_t\big(\hat{y}_t^{(b)}, y_t^{(b)}\big), \tag{17}$$

with task weights w_t. The full objective adds the graph priors to this supervised loss,

$$\mathcal{L} = \mathcal{L}_y + \sum_{k=1}^{K} \lambda_k\, \Omega_k. \tag{18}$$

Using the graphs as a soft prior rather than a hard graph-convolutional layer is deliberate. An edge with $A^{(k)}_{ij} = 1$ carries experimental evidence and is trusted, but $A^{(k)}_{ij} = 0$ usually means the pair has not been tested rather than that no interaction exists. A hard operator would commit to those uncertain zeros, whereas the KL prior only pulls attention toward the known edges and lets the data overrule it where the labels demand. Formally, this soft prior is the relaxation of masked graph-attention: a graph-attention layer confines each gene to its neighbors through an additive $-\infty$ mask on non-edges, and $\lambda\to\infty$ drives the regularized self-attention onto that same neighborhood, so the encoder is functionally equivalent to a graph-attention network while its forward pass stays mask-free. rather than masking is the right choice here because the graph is invariant across perturbations: a per-example mask re-imposes the same static adjacency on every forward pass and forces a rebuilt graph for each perturbed strain, whereas a constraint constant across examples is more naturally imposed once, in the objective, with the perturbation carried by the operator. Two further benefits follow. The same graphs can later be promoted from priors to prediction targets, and a head that keeps strong weight on a non-edge, where $A^{(k)}_{ij} = 0$ yet α_{ij} stays large, flags a candidate interaction that the networks have not yet recorded. Only gene–gene graphs are regularized. In the trigenic experiment these are the $K = 9$ physical, regulatory, and TFLink networks together with six STRING channels, one graph per head (sizes, overlap, and sources in Supplementary Note 7). Metabolism is bipartite and has no $N \times N$ adjacency, so it enters as a representation annotation and is never used as an attention prior.

Model training

[FILLER.] Optimizer, learning-rate schedule, batch size, early stopping; hardware (GPUs, distributed/DDP), wall-clock; software versions; random seeds.

Baselines

[FILLER.] DANGO, DCell, and Yeast9 flux balance analysis; classical-ML baselines (Supplementary Fig. 4); matched training data and evaluation protocol.

Evaluation metrics

[FILLER.] Pearson correlation (r) and any secondary metrics; aggregation across folds; definition of error bars (see Statistics).

Gene–gene interaction prediction

[FILLER.] Trigenic interaction prediction setup; performance vs. interaction magnitude; ablations (Fig. 2).

Expression and morphology prediction

[FILLER.] Knockout transcriptome and single-knockout morphology tasks; shared latent embedding; cross-phenotype transfer (Fig. 3).

CGT-guided strain design

[FILLER.] Objective and candidate enumeration for beta-carotene and betaxanthin production; ranking and selection of recommended gene deletions (Fig. 6).

Strain construction and cultivation

[FILLER.] Strains, plasmids, and genome edits; the TorchCell + UIUC iBioFoundry design–build–test–learn loop; cultivation and fermentation conditions.

Metabolite quantification

[FILLER.] Sampling, extraction, and analytical method (HPLC/LC–MS); calibration and titer quantification.

Inference at scale

[FILLER.] Large-scale inference pipeline and resources (Supplementary Fig. 19).

Statistics

[FILLER.] Statistical tests used and whether one- or two-sided; sample sizes (n); definition of center and dispersion; box-plot conventions (median, IQR hinges, whiskers $\leq 1.5\times$IQR); multiple-comparison handling; significance thresholds.

Reporting summary

Further information on research design is available in the Nature Portfolio Reporting Summary linked to this article.

Data availability

[FILLER.] Datasets are available on Hugging Face (https://huggingface.co/...); the Neo4j knowledge graph is hosted at NCSA (...); trained model weights are available at ..... Source data are provided with this paper.

Code availability

[FILLER.] TorchCell and the Cell Graph Transformer are open source at https://github.com/Mjvolk3/torchcell (license: ...), with a release archived at Zenodo (https://doi.org/...).

Acknowledgments.

Declarations

Supplementary Information ✗

Contents

Supplementary Note 1: functional equivalence of the perturbation operator ■

Encoding the wildtype cell once and applying a perturbation-conditioned operator is at least as expressive as rebuilding and re-encoding a separate graph for every strain. The operator can represent any predictor the rebuild-and-re-encode route can, and strictly more whenever the rebuilt graph would discard information the perturbation carries. Figure 1g and the Methods assert the approximation $\Phi(G_p) \approx \mathcal{R}_\phi(\mathcal{T}_\psi(H, p))$; this note separates it into two claims that are easy to conflate. The first is an exact reparameterization, a statement about which functions are expressible that does not depend on training. The second is a learned approximation: the trained operator matches the rebuild route only to the extent that the encoder loses no information and the operator has the capacity to fit the target.

Setting. There are two ways to predict a phenotype under a perturbation p. Perturbing the *data* materializes a rebuilt graph $G_p = \kappa(G, p)$, deleting or rewiring vertices and edges, and encodes it,

$$\hat{y} = \Phi(G_p).$$

The deterministic rebuild map $\kappa : (G, p) \mapsto G_p$ runs once per strain (written κ, not ρ, because ρ already denotes the gene–reaction association). Perturbing the *representation* encodes the reference once and operates on it,

$$\hat{y} = \mathcal{R}_\phi\big(\mathcal{T}_\psi(F_\theta(G), p), \varepsilon\big),$$

so the graph enters only through F_θ and the representation $H = F_\theta(G)$ is reused across all p. Collect the predictors each route can express into two function classes: $\mathcal{H}_{\text{data}} = \{\Phi \circ \kappa\}$, the maps built by perturbing the data, and $\mathcal{H}_{\text{rep}} = \{\Psi : \mathcal{G} \times \mathcal{P} \to \mathcal{Y}\}$, the maps expressible as a direct function of the pair (G, p), which is the amortized family CGT targets. Here Φ, Ψ are arbitrary measurable target maps and D is the data distribution over (G, p); remaining notation follows Table 1.

Proposition 1 (Reparameterization; amortization containment)**.**
With the function classes $\mathcal{H}_{\text{data}}$ and $\mathcal{H}_{\text{rep}}$ defined above, $\mathcal{H}_{\text{data}} \subseteq \mathcal{H}_{\text{rep}}$, with equality if and only if the rebuild map κ is injective on $\operatorname{supp} D$.

In words: the two routes target the same set of predictors, and the amortized route is strictly larger exactly when the rebuild destroys information. This is the amortization-containment property: the encode-once-and-operate family contains the data-rebuilding family, and strictly extends it whenever a feature or parametric perturbation such as dose or type survives in (G, p) but is erased by the rebuilt topology G_p.

Proof. We establish the two inclusions in turn.

Containment. The map κ is deterministic, so (G, p) fixes G_p. For any target $\Phi^\star$ we have $\Phi^\star(G_p) = (\Phi^\star \circ \kappa)(G, p) =: \Psi^\star(G, p)$, an equality of functions, which gives $\mathcal{H}_{\text{data}} \subseteq \mathcal{H}_{\text{rep}}$.

Strictness. The reverse inclusion would require the pair (G, p) to be recoverable from the rebuilt graph alone, that is $\sigma(G_p) = \sigma(G, p)$ as generated σ-algebras. But the rebuild discards information: p is generally not recoverable from G_p, so $\sigma(G_p) \subsetneq \sigma(G, p)$ whenever κ is non-injective, and a map $\Psi \in \mathcal{H}_{\text{rep}} \setminus \mathcal{H}_{\text{data}}$ then exists. □

Concrete example. A three-gene example makes both the equality and its failure concrete. Take base graph G the path $A - B - C$. In the injective case, deleting B with $p = \{(g_B, \text{del})\}$ drops B and its edges, so $G_p = \{A, C\}$ with both nodes isolated, and the target $\Phi(G_p) = \#\text{edges} = 0$ is reproduced with no rebuild by $\Psi(G, p) = \#\text{edges}(G) - \#\{\text{edges incident to } B\} = 2 - 2 = 0$. In the non-injective case, overexpressing B at two doses, $p_1 = \{(g_B, \text{OE}, 1)\}$ against $p_2 = \{(g_B, \text{OE}, 3)\}$, changes no topology, so $\kappa(G, p_1) = \kappa(G, p_2) = G$ and any $\Phi(G_p)$ returns one value for both, yet the dose-response phenotypes differ. The pair (G, p) still carries the magnitude $m \in \{1, 3\}$ and separates the two cases; continuous dose, and likewise the perturbation type τ, lives in p and not in a bare rebuilt topology. This realizes $\mathcal{H}_{\text{rep}} \setminus \mathcal{H}_{\text{data}}$.

Origin of the approximation. Proposition 1 is about expressibility; it does not by itself say the trained CGT attains the match, and that is where the approximation sign originates. The ideal amortizer is $\Psi^\star = \Phi^\star \circ \kappa$. CGT realizes it not by that literal composition but by the learned $\mathcal{R}_\phi \circ \mathcal{T}_\psi$ acting on $H = F_\theta(G)$. Suppose F_θ is injective on $\operatorname{supp} D$, so that (H, p) carries the same information as (G, p), which is free for a fixed gene set because H keeps one identifiable row per gene; and suppose cross-attention over the perturbation set followed by an MLP readout is a universal approximator of set-conditioned permutation-equivariant maps. Then for every $\epsilon > 0$ there exist ψ, ϕ with

$$\sup_{(G,p)} \big\| \mathcal{R}_\phi\big(\mathcal{T}_\psi(F_\theta(G), p), \varepsilon\big) - \Psi^\star(G, p) \big\| < \epsilon, \tag{19}$$

[Suppl. Fig. – empirical equivalence: CGT-style vs. GNN-style on single-gene KO fitness]

Idealized test of Supplementary Note 1 on single-gene knockout fitness. Two capacity-matched small models trained on identical data, features, and splits: a data-perturbing GNN that rebuilds G_p and re-encodes ($\hat{y} = \Phi(G_p)$) vs. a representation-perturbing CGT that encodes the wildtype once and applies the operator ($\hat{y} = \mathcal{R}_\phi(\mathcal{T}_\psi(H, p))$). **a**, predicted vs. actual fitness for both models; **b**, held-out accuracy (Pearson r / MSE), expected near-equal; **c**, agreement of the two models' predictions ($\hat{y}_{\text{GNN}}$ vs. $\hat{y}_{\text{CGT}}$, near the identity line); **d**, compute cost: per-strain rebuild+encode (GNN) vs. encode-once+operator (CGT), with the growth of Supplementary Note 3, Supplementary Table 1. *[placeholder: run the experiment, then compose.]*

Supplementary Figure 1 Empirical functional equivalence of the perturbation operator on single-gene knockout fitness (support for Supplementary Note 1).

and, with Proposition 1, $\mathcal{R}_\phi(\mathcal{T}_\psi(F_\theta(G), p), \varepsilon) \approx \Phi(\kappa(G, p)) = \Phi(G_p)$.

Precision. The two claims are distinct. The reparameterization of Proposition 1 is exact, a set-theoretic identity, because κ is deterministic. The trained $\Phi(G_p) \approx \mathcal{R}_\phi(\mathcal{T}_\psi(H, p))$ is a learned correspondence, not a mathematical identity: it holds to the extent that F_θ is lossless and $\mathcal{T}_\psi, \mathcal{R}_\phi$ can fit $\Phi^\star \circ \kappa$. The realizability conditions are standard: permutation-invariant set functions are universally approximable by the Deep Sets construction, self-attention is a universal approximator of permutation-equivariant sequence maps, and conditioning on a perturbation rather than refitting on modified data is amortization in the sense of hypernetworks. **[refs to add: Zaheer et al., *Deep Sets*, NeurIPS 2017; Yun et al., *Are Transformers Universal Approximators of Seq-to-Seq Functions?*, ICLR 2020; Ha, Dai & Le, *HyperNetworks*, ICLR 2017.]**

Relation to prior work. The closest precedent is the expressivity result that a structural graph transformation applied at preprocessing, followed by standard message passing, is at least as expressive as the corresponding improved message-passing algorithm; that result is topological and isomorphism-flavored, whereas Proposition 1 is a function-class containment for feature and parametric perturbations p (dose m, type τ) that a bare rebuilt topology cannot carry. Running a network on perturbed copies of a graph is known to exceed 1-WL expressiveness, by random node dropout or by aggregating a policy-drawn bag of subgraphs, with 1-WL the reference power for message passing. GEARS instantiates the data-graph side biologically, materializing perturbations as edges on a gene–gene graph [14]; CGT is its amortized counterpart, keeping the graph fixed and learning the perturbation in representation space. **[refs to add / search: Jogl, Thiessen & Gärtner, *Weisfeiler and Leman Return with Graph Transformations*, MLG 2022 (preprocessing transform $\geq$ improved MP); Papp et al., *DropGNN*, NeurIPS 2021 (node-dropout $>$ 1-WL); Bevilacqua et al., *Equivariant Subgraph Aggregation Networks* (ESAN), ICLR 2022 (bag-of-subgraphs $>$ 1-WL); Xu et al., *How Powerful Are GNNs?* (GIN), ICLR 2019 (1-WL bound).]**

Empirical validation. A minimal, idealized experiment on single-gene knockout fitness tests the equivalence directly (Supplementary Fig. 1). Two capacity-matched small models are trained on the same knockouts, gene features, and splits, differing only along the perturb-data versus perturb-representation axis: a data-perturbing GNN that rebuilds the perturbed graph $G_p = \kappa(G, p)$, deleting the knocked-out gene and its edges, and re-encodes, $\hat{y} = \Phi(G_p)$; and a representation-perturbing CGT that encodes the wildtype once, $H = F_\theta(G)$, and applies the operator, $\hat{y} = \mathcal{R}_\phi(\mathcal{T}_\psi(H, p))$. A single deletion gives an injective κ, so Proposition 1 predicts equal attainable accuracy: the amortized model should match the data-perturbing one on held-out fitness while encoding once rather than rebuilding and re-encoding per strain. The compute gap this avoids is quantified in Supplementary Note 3.

Supplementary Note 2: graph-regularized attention contains graph attention ■

A graph-attention network hard-wires each gene to attend only to its catalogued neighbors. CGT instead lets every gene attend to all genes and adds a soft penalty that pulls each attention row toward the graph. This note shows the two coincide in a precise sense: as the penalty weight λ grows, the soft head reproduces the hard-masked one exactly, and at any finite λ the soft head can do strictly more, either keeping attention on a pair the graph omits, which surfaces a candidate interaction, or pulling attention off a listed edge the data contradict, neither of which a hard mask permits.

Figure 1h and the Methods, in the graph-regularized attention subsection, state that penalizing the divergence of an attention head from a row-normalized adjacency is the soft relaxation of masked graph attention, and that $\lambda \to \infty$ makes the two coincide (Eqs. 15–18). This note makes that precise: hard-masked graph attention is the infinite-penalty limit of the KL-regularized head, and at finite λ the regularized head is strictly more expressive. Notation follows Table 1.

Setting. A graph-attention head confines gene i to its neighbors $\mathcal{N}(i) = \{j : A_{ij}^{(k)} = 1\}$ through an additive mask,

$$\alpha_{i,:}^{\mathrm{GAT}} = \mathrm{softmax}_j\big(s_{ij} + M_{ij}\big), \qquad M_{ij} = 0 \text{ on edges}, \quad M_{ij} = -\infty \text{ on non-edges},$$

so $\operatorname{supp} \alpha_{i,:}^{\mathrm{GAT}} = \mathcal{N}(i)$. The CGT head is unmasked,

$$\alpha_{i,:}^{\mathrm{CGT}} = \mathrm{softmax}_j\big(q_i \cdot k_j / \sqrt{d_k}\big),$$

and the graph enters only through the prior Ω_k (Eq. 16).

Proposition 2 (Graph attention as the $\lambda \to \infty$ limit)**.**
Fix a graph k and, for the head (ℓ_k, a_k) aligned to it, write $\lambda = \lambda_k$, $\tilde{A} = \tilde{A}^{(k)}$, and $\alpha_{i,:}$ for its attention row. Let $\mathcal{H}_{\mathrm{mask}}$ be the family of hard-masked graph-attention rows with support fixed to $\mathcal{N}(i)$, and $\mathcal{H}_{\mathrm{soft}}(\lambda)$ the family of KL-regularized rows under objective (18). Then $\mathcal{H}_{\mathrm{mask}} \subseteq \mathcal{H}_{\mathrm{soft}}(\lambda)$ for every λ, hard-masked graph attention is the $\lambda \to \infty$ limit of the regularized head, and at every finite λ the inclusion is strict.

Proof. The two parts match the two clauses of the proposition.

The limit. The supervised loss $\mathcal{L}_y$ is bounded in $\alpha_{i,:}$, being an average of bounded per-task losses, whereas the prior term $\lambda\, D_{\mathrm{KL}}(\tilde{A}_{i,:} \,\|\, \alpha_{i,:})$ scales with λ. As $\lambda \to \infty$ any minimizer of the full objective therefore drives $\alpha_{i,:}$ to the minimizer of $D_{\mathrm{KL}}(\tilde{A}_{i,:} \,\|\, \cdot)$. The forward KL is non-negative and, by Gibbs' inequality, equals 0 if and only if $\alpha_{i,:} = \tilde{A}_{i,:}$, and it is strictly convex in $\alpha_{i,:}$, so this minimizer is unique. Hence $\alpha_{i,:} \to \tilde{A}_{i,:}$, whose support is the graph neighborhood, $\operatorname{supp} \tilde{A}_{i,:} = \{j : A_{ij}^{(k)} = 1\} = \mathcal{N}(i) = \operatorname{supp} \alpha_{i,:}^{\mathrm{GAT}}$. The regularized head collapses onto the same neighbor-restricted attention that a hard $-\infty$ mask produces, so $\mathcal{H}_{\mathrm{mask}}$ is recovered in the limit and is contained in $\mathcal{H}_{\mathrm{soft}}(\lambda)$ for every λ.

Strictness at finite λ. Split the per-row penalty over the two kinds of column,

$$D_{\mathrm{KL}}(\tilde{A}_{i,:} \,\|\, \alpha_{i,:}) = \underbrace{\sum_{j:\, A_{ij}^{(k)}=1} \tilde{A}_{ij} \log \frac{\tilde{A}_{ij}}{\alpha_{ij}}}_{\text{edges}} + \underbrace{\sum_{j:\, A_{ij}^{(k)}=0} \tilde{A}_{ij} \log \frac{\tilde{A}_{ij}}{\alpha_{ij}}}_{\text{non-edges}}. \tag{20}$$

On every non-edge $\tilde{A}_{ij} = 0$ and $0 \log(0/\alpha_{ij}) = 0$, so the prior places no penalty on attention mass assigned off the graph, and this asymmetry is exactly what a hard mask lacks. A head may hold α_{ij} large where $A_{ij}^{(k)} = 0$ at zero cost to the prior, the only force on that mass being the supervised loss; reading the trained attention map and finding persistent off-graph mass, which is the signal that the model keeps populating an entry absent from the base graph, therefore flags a gene pair the model finds predictive that the catalogued networks have not recorded, a candidate missing interaction. On a true edge $\tilde{A}_{ij} > 0$, by contrast, driving $\alpha_{ij} \to 0$ sends its edge term to $+\infty$, so the prior resists dropping a catalogued edge, yet at finite λ a strong supervised gradient can still pull attention off an edge the labels contradict, trading a finite prior penalty for a better fit. A hard mask permits neither move: it sets the off-graph logit to $-\infty$ so that $\alpha_{ij} \equiv 0$ and the candidate edge is unrepresentable, and it makes the neighborhood structural and unoverridable. Any attention row with nonzero off-graph mass therefore lies in $\mathcal{H}_{\mathrm{soft}}(\lambda) \setminus \mathcal{H}_{\mathrm{mask}}$, giving $\mathcal{H}_{\mathrm{mask}} \subsetneq \mathcal{H}_{\mathrm{soft}}(\lambda)$ at every finite λ. □

Precision. Two caveats keep the caption honest. First, the equivalence is at the level of attention support, which genes attend to which, not of the within-neighborhood weights: the prior's target $\tilde{A}_{i,:}$ is uniform over $\mathcal{N}(i)$, whereas a graph-attention layer learns non-uniform neighbor weights, and at finite λ CGT's own $\alpha_{i,:}$ is data-shaped inside the softly enforced neighborhood. The defensible shared claim is same neighbors, learned weights, not identical weights.

[Suppl. Fig. – empirical relevance of graph-regularized attention: the λ sweep]

Idealized test of Supplementary Note 2. One cell-encoder head aligned to a single gene–gene graph, trained on held-out fitness with the graph-prior weight λ swept from 0 (free attention) to the hard-mask limit. **a**, attention-to-adjacency divergence $D_{\mathrm{KL}}(\tilde{A}_{i,:} \,\|\, \alpha_{i,:})$ and support overlap with $\mathcal{N}(i)$ vs. λ, decreasing to 0 as $\lambda \to \infty$ (the masked graph-attention limit of Prop. 2); **b**, held-out accuracy (Pearson r) vs. λ, peaked at intermediate λ, above both the free head ($\lambda = 0$) and the hard mask; **c**, discovery: precision–recall of recovering held-out interactions from the top off-graph attention entries, against a hard-masked control that cannot place off-graph mass; **d**, overruling: catalogued edges the head down-weights, enriched for label-contradicted pairs relative to a null. *[placeholder: run the λ sweep, then compose.]*

Supplementary Figure 2 Empirical relevance of graph-regularized attention: the finite-λ soft prior recovers graph attention in the limit, wins at intermediate λ, and enables edge discovery and overruling (support for Supplementary Note 2).

Second, as in Supplementary Note 1 the correspondence is learned, not a set-theoretic identity: it holds to the extent that the aligned head is trained toward $\tilde{A}$ and the supervised loss allows. The discovery-and-overrule asymmetry is a direct consequence of the KL direction $D_{\mathrm{KL}}(\tilde{A} \,\|\, \alpha)$, with the prior as reference and the attention as model; reversing the arguments would penalize off-graph mass and destroy discovery.

Proposition 2 is the companion of Proposition 1. Note 1 is the equivalence along the perturbation axis, operating on H rather than rebuilding G_p; this note is the equivalence along the graph-structure axis, regularizing attention rather than masking it. Together they justify the two architectural choices the bottom row of Fig. 1 depicts.

Empirical validation. A small, idealized experiment tests the proposition directly (Supplementary Fig. 2). A single cell-encoder head is aligned to one gene–gene graph and trained on held-out fitness while the prior weight λ is swept from 0, which is free attention, to large, which is the hard-mask limit. The proposition predicts three signatures. First, as λ grows the head's attention row $\alpha_{i,:}$ should converge to the row-normalized adjacency $\tilde{A}_{i,:}$, its support contracting onto the neighborhood $\mathcal{N}(i)$ and reproducing masked graph attention. Second, held-out accuracy should peak at an intermediate λ, above both the unconstrained head at $\lambda = 0$ and the hard-mask limit, locating the value in the soft regime. Third, at that intermediate λ the off-graph attention mass, meaning entries with $A^{(k)}_{ij} = 0$ yet α_{ij} large, should be enriched for gene pairs held out of the training graph but present in an independent interaction set, exhibiting the discovery channel, while catalogued edges the head down-weights should be enriched for pairs the labels contradict, exhibiting overruling. A hard-masked control produces neither signal by construction, which is the empirical face of the strict inclusion.

Supplementary Note 3: the static-graph assumption and the case for learning the perturbation ■

This note sets out the reasons for representing a strain as a perturbation of a fixed reference rather than as a rebuilt graph. They are of three kinds: a biological assumption together with its limits, a practical goal of one reusable data object, and a computational argument that the rebuild alternative does not scale.

The static-graph assumption. TorchCell treats the biological networks as catalogued objects that annotate a fixed reference (Fig. 1c, top): a genome is the minimal reference, and gene, regulatory, protein-interaction, and metabolic graphs are optional structure laid over it. A strain is then a perturbation applied to that shared reference rather than an independently rebuilt graph. This carries a biological hypothesis, that the catalogued wiring is perturbation-invariant: deleting or dosing a gene does not rewire the reference network, so the identified structures are essentially static across the strain library. The hypothesis is defensible for much of the graph, because a physical protein–protein interaction reflects a binding capability that a distant genetic perturbation usually leaves intact.

Limitations of the assumption. The assumption is a modeling choice, not a fact, and it has a clear failure mode. An edge that exists in the catalogue can effectively cease to exist in a particular strain: if perturbing an upstream regulator drives a partner's expression so low that the protein is essentially absent, the physical interaction cannot occur in that cell even though the reference graph still lists it. More generally, condition- and genotype-specific rewiring is not represented by a single static adjacency. Two features soften, though do not eliminate, this cost. First, the graph is a soft prior, not a hard substrate (Supplementary Note 2): the supervised loss can pull attention off an edge the labels contradict, and persistent off-graph attention can flag a context-specific interaction the catalogue omits. Second, the reference is the wildtype, so the assumption is only ever used to displace from a well-characterized baseline rather than to assert absolute structure. A genuinely dynamic, expression-gated edge is still only approximately handled, and we flag this as a limitation and a target for condition-conditioned graph priors.

A single general data object. Beyond expressivity (Supplementary Note 1), the design was chosen for a practical reason: keeping the reference fixed lets a data loader emit one general, read-ready object on request, which can then be consumed by different modeling paradigms without re-deriving the data. The same reference-plus-perturbation object is intended to serve, from one representation, machine learning on graphs, attention or transformer models, and constraint-based metabolic modeling; a stoichiometric or flux-balance formulation reads the metabolic subgraph, and a coupled-ODE kinetic model reads the same entities as state variables. Getting the data-transformation layer right once decouples the choice of modeling method from the data pipeline and makes multimodal method exploration a matter of consuming the same object differently. This was the original motivation, programmatic generality, and it is orthogonal to but reinforced by the expressivity and cost arguments below.

Compute and complexity. The alternative to learning the perturbation is to materialize and re-encode a perturbed graph per strain, which is expensive in a way that is easy to underestimate. Let N be the number of genes, $|E|$ the edges, d the hidden width, L the encoder depth, $|p|$ the perturbed genes per strain (with $|p| \ll N$), B the strains in a batch, and L_{op} the depth of the perturbation operator. Supplementary Table 1 tracks the growth of each route.

	Perturb data (rebuild)	Perturb representation (operator)
Encode reference	folded into per-strain	$\mathcal{O}(L\|E\|d)$, once
Per strain	$\mathcal{O}(L\|E\|d)$	$\mathcal{O}(\|p\|Nd)$
Batch of B strains	$\mathcal{O}(B\,L\|E\|d)$	$\mathcal{O}(L\|E\|d + B\|p\|Nd)$
Deeper operator, L_{op} layers over N genes	–	$\mathcal{O}(L\|E\|d + B\,L_{\text{op}}N^2d)$

Supplementary Table 1 Growth of per-batch compute for the two routes. The operator route wins by amortizing the encode across the batch and keeping the per-strain operator thin; a deep operator over all genes gives that advantage back.

The advantage has two sources, and both are necessary. The first is amortization: the encode $\mathcal{O}(L|E|d)$ is paid once and shared across the batch, so its per-strain share is $\mathcal{O}(L|E|d/B)$ and vanishes as B grows, whereas the rebuild route repeats the full encode for every strain and grows as $\mathcal{O}(B\,L|E|d)$. The second is thinness: the per-strain operator is a single cross-attention with $|p| \ll N$, costing $\mathcal{O}(|p|Nd)$, far below the rebuild's $\mathcal{O}(L|E|d)$. If the operator is deepened to L_{op} self-attention layers over all N genes, the per-strain cost climbs to $\mathcal{O}(L_{\text{op}}N^2d)$ and the batch term returns to $\mathcal{O}(B\,L_{\text{op}}N^2d)$, which is again processing a batch of B dense N-token graphs, the same fan-out the operator was meant to avoid. The design therefore keeps the operator shallow and adds capacity to fit the perturbation in the thin operator and the readout, not by re-encoding.

Empirical cost analysis. We tested this directly, and even in the simplest case, a single-gene deletion, optimizing the data step cannot close the gap. Building a strain by subgraphing the reference measured about 44 ms per sample

against about 0.4 ms for a thin perturbation-index representation, a gap that ranged from about 86 to 123× across configurations. An engineered zero-copy loader, which stores the full `edge_index` once, precomputes an $N \times K$ node-to-incident-edge index, and applies a per-strain boolean mask that zeros the edges touching perturbed genes in $\mathcal{O}(|p|\, d)$ rather than rebuilding in $\mathcal{O}(|E|)$, cut graph-processing time by 3.65× and edge-tensor memory by 93.7%, yet total per-sample time fell only 1.21× (from about 54 to 45 ms). Even precomputing a mask for every possible single deletion, the most favorable case available, reached only about 1.6× and never closed the residual 70-to-875× gap to the thin route. The reason is that the cost is not in loading, which profiled at essentially zero, but in the model: each strain in a batch carries a different mask, so message passing must be run separately on all B masked copies of the graph, inflating a 6,607-node forward pass to about 192,000 nodes and 2.4 million edge operations to about 67 million per batch. Masking rather than rebuilding only simulates the subgraph inside the same $\mathcal{O}(B\, L|E|)$ fan-out; it does not remove it *[numbers from the SubgraphRepresentation versus Perturbation and lazy-mask benchmarks; notes/experiments.006 subgraph-optimization and weekly 2025.44–45 reports]*. This is the empirical face of Supplementary Table 1: the encode cannot be avoided per strain on the rebuild route, no matter how fast the loader, which is exactly what the operator route amortizes.

Trade-offs between the two routes. The rebuild route is not strictly dominated. Because it encodes each strain directly, it never needs the wildtype embedding as an explicit object, whereas the operator route embeds the reference once and derives every strain from it, so every perturbation is referenced to the wildtype. In practice the operator route's single shared encode is the advantage, since the perturbations do originate from the wildtype and reuse dominates, but the trade-off is real, and these arguments, biological, computational, and pipeline-level, are early and warrant further work.

Supplementary Note 4: gene interactions as a learned function of fitness ■

We train one model to predict fitness and gene interactions together, and ask whether the interactions can be recovered from the fitness predictions alone, without the algebraic definition of an interaction ever entering the objective. If they can, it is evidence that a correctly trained model learns a complex nonlinear relationship from data rather than from a hand-written loss term. How far that generalizes is unclear, but the same logic would suggest that, with the right data, a model could come to respect the constraints of metabolism without an explicit physics- or flux-balance-informed regularizer. This note states the hypothesis, asks how much of it can be made a theorem, and marks the experiment as a placeholder.

Setting. For genes i, j, k let f_S denote the fitness of the mutant carrying the perturbations in $S \subseteq \{i, j, k\}$, normalized so the wildtype has fitness 1; a triple has seven such values, the singles f_i, f_j, f_k, the doubles f_{ij}, f_{ik}, f_{jk}, and the triple f_{ijk}. The trigenic interaction is a fixed multilinear map $g : \mathbb{R}^7 \to \mathbb{R}$ of these fitnesses,

$$\tau_{ijk} = g(f) = f_{ijk} - f_i f_j f_k - \varepsilon_{ij} f_k - \varepsilon_{ik} f_j - \varepsilon_{jk} f_i, \qquad \varepsilon_{ab} = f_{ab} - f_a f_b,$$

that is, what remains of the triple-mutant fitness after removing the multiplicative expectation and the pairwise interactions (Fig. 2a). Two ways to train follow. The *direct* route supervises τ_{ijk} as a label. The *factored* route supervises the fitnesses f_S and reads interactions off them, either by applying g to the predicted fitnesses or by a separate interaction head trained jointly; the test is whether the two agree even though g was never in the loss.

Toward a formal statement. Universal approximation is the wrong tool here: it guarantees that a large enough network can represent g composed with a fitness predictor, but it says nothing about whether training on fitness recovers τ, nor how accurately, so it is far too broad to be the claim. Two narrower statements are provable and sharpen the intuition.

Proposition 3 (Sufficiency: interactions need no extra capacity)**.**
If a representation H is sufficient for the sub-genotype fitnesses, meaning $f_S = \phi_S(H)$ for each S, then $\tau_{ijk} = g(\phi(H))$ is also a function of H. Any representation sufficient for fitness is therefore sufficient for interactions.

Proof. The claim is immediate. The composition of the measurable maps ϕ_S with the polynomial g is measurable, so τ_{ijk} is a deterministic function of H; predicting it requires no capacity, target, or objective term beyond those already needed to predict fitness. □

Proposition 3 is what makes the idea reasonable: the interaction is a fixed readout of quantities the model already predicts, so the definition g never has to appear in the loss. It does not, however, say the readout is easy to recover, because τ is a small residual of large terms.

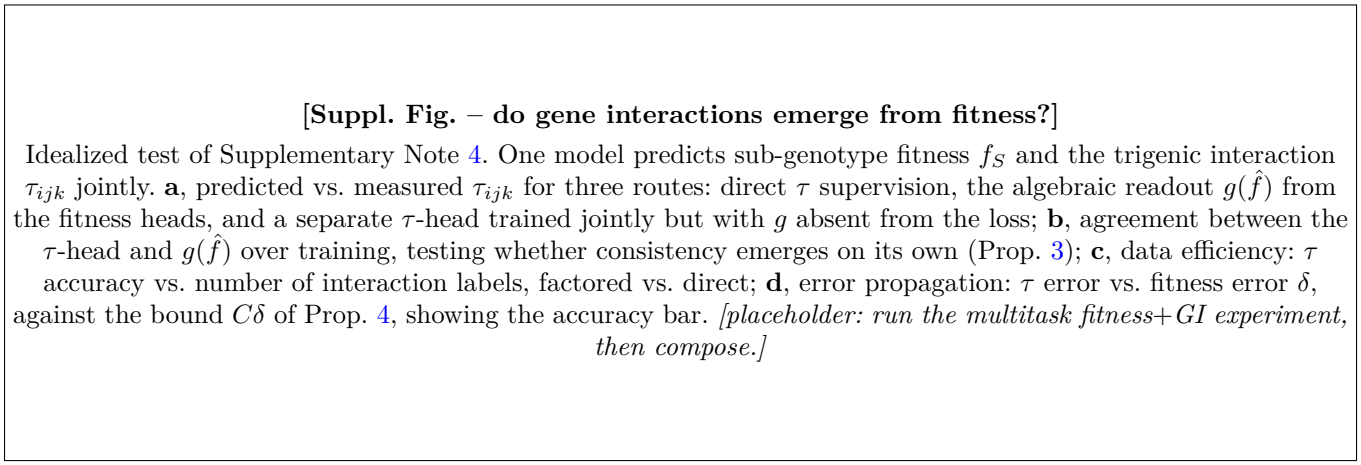
Proposition 4 (Conditioning: the accuracy bar)**.**
The map g is locally Lipschitz, and near the typical regime $f \approx \mathbf{1}$ its gradient is bounded by a constant $C = \mathcal{O}(1)$. A fitness estimate with $\|\hat{f} - f\|_\infty \le \delta$ then gives $|g(\hat{f}) - \tau_{ijk}| \le C\delta$. Since τ_{ijk} is a higher-order residual with $|\tau_{ijk}| \ll 1$, the relative error is $\approx C\delta/|\tau_{ijk}|$, which exceeds one unless $\delta \lesssim |\tau_{ijk}|/C$.

Proof. Each partial derivative of g is a sum of products of at most two fitnesses, bounded near $f \approx \mathbf{1}$; a first-order expansion gives the absolute bound, and dividing by $|\tau_{ijk}|$ gives the relative one. □

Together the two say the factored route is expressible for free but conditioning-limited: recovering interactions from fitness demands fitness accuracy finer than the interaction magnitude to resolve its sign and scale.

What would be surprising, and is only empirical. The claim we actually want to test is neither proposition. It is that joint training on the denser fitness signal makes an interaction head align with $g(\hat{f})$ *without* g in the loss, and generalizes better than direct τ supervision. This is a statement about inductive bias and data efficiency, not a theorem: whether optimization discovers the readout depends on the architecture, the data, and the training set-up. The favorable case is clear from Proposition 3, since a representation pinned down by abundant fitness labels is already sufficient for τ, but that it happens is for the experiment to show (Supplementary Fig. 3).

Implication for metabolism. If a measured phenotype is a fixed function of a latent cellular state the model predicts, the same reasoning suggests the model could come to satisfy the governing constraints without a physics-informed or genome-scale-metabolic regularizer in the loss: the constraint would be a readout, not an objective term. The caveat is identifiability. The trigenic case is clean because τ is a function of *observed* fitnesses, whereas metabolic constraints $Sv = 0$ involve fluxes v that are largely unobserved, so the constraint is not a function of measured phenotypes alone. Gene interactions are the fully observed instance where the idea can be tested before it is extrapolated to metabolism.



Supplementary Figure 3 Emergent recovery of gene interactions from fitness: whether a jointly trained model reproduces the interaction algebra without it appearing in the objective (support for Supplementary Note 4).

The atomic unit of data. The factored route trades per-instance flexibility, treating each strain as its own cell graph, for information density, since fitness labels are more abundant and lower-variance than interaction residuals, and it requires non-standard queries that assemble all sub-genotypes of a triple rather than one instance at a time. We pursue it because it may be the route that scales: rather than encode the interaction algebra, or the constraints of metabolism, in the objective, let the model learn them from data keyed to the atomic unit of the problem, the identity of the cell, its genome and strain. This is the bitter-lesson stance applied to phenotype prediction, and the reason the direction is worth the loss of flexibility.

Supplementary Note 5: persistent entities and contingent observations ✗

The molecular parts a cell is assembled from and the measurements made on cells are not two halves of one dataset. They differ in scale, by about three orders of magnitude, and they differ in kind. A protein sequence is the same object in a glucose assay and in an oxidative-stress assay; the measured phenotype is not. This note makes both statements precise, says exactly what the bit counts in Fig. 1c do and do not mean, and draws the consequence that determines the architecture: the model is cut at the shared representation H, entity encoders are taken already trained, and the phenotype corpus is spent only on the perturbation operator and the decoder.

Setting. Write $\mathcal{U}$ for the *persistent entities* – DNA, RNA and protein sequences, small molecules, and experimentally determined structures – and $\mathcal{I}$ for the *contingent observations*, the phenotype records of Supplementary Table 15. An entity $x \in \mathcal{U}$ is an identity, the same object in every experiment in which it occurs. An observation is a value measured on a cell in a context,

$$y \;=\; f(\{x_i\}_{i\in p},\; p,\; \varepsilon,\; a) \;+\; \eta, \tag{21}$$

with p the perturbation, ε the environment, a the assay, strain background and protocol, and η measurement noise. The entities in (21) recur in every observation that contains them. The observation itself is pinned to a single point of the context space and, being noisy, is not exactly reproducible even there. This asymmetry, rather than the raw size of the two corpora, is what forces the factorization.

The measure. Fig. 1c reports each corpus in bits. The quantity is the *compressed length*: for a corpus D, a fixed serialization s, and a fixed lossless compressor C,

$$L_C(D) \;=\; 8\,\big|C(s(D))\big| \quad \text{bits.} \tag{22}$$

This is a deliberately modest quantity. Because *information content* is used loosely and means two different things, Proposition 5 states which of them L_C is, and which it is not.

Proposition 5 (Compressed length is a bound, not an entropy)**.**
Let C be a lossless compressor with decompressor C^{-1} and self-delimiting output. For every corpus D,

$$K(D) \;\le\; L_C(D) \;+\; K\big(C^{-1}\big) \;+\; O(1), \tag{23}$$

where K denotes Kolmogorov complexity; and there is a sub-probability distribution q_C for which $L_C(D) = -\log_2 q_C(s(D))$. Thus L_C is at once a computable upper bound on the Kolmogorov complexity of D and an exact Shannon codelength under the model q_C that C implicitly assumes. It is not an estimate of the entropy of the source that generated D, and it depends on the choice of s and C.

Proof. Both halves follow from the definitions.

Kolmogorov bound. The compressed file together with the decompressor is a program that outputs D: run C^{-1} on $C(s(D))$ and deserialize. Its length is $L_C(D) + K(C^{-1}) + O(1)$, and $K(D)$ is by definition the length of the shortest program with that output, which gives (23). The bound is one-sided, and can be loose by any amount, because C removes only the redundancy it was designed to see.

Shannon codelength. C is injective with self-delimiting output, so its codewords satisfy the Kraft inequality, $\sum_D 2^{-L_C(D)} \le 1$, and $q_C(\cdot) := 2^{-L_C(\cdot)}$ is a sub-probability. Hence $L_C(D) = -\log_2 q_C(s(D))$ is the self-information of D under q_C. Nothing constrains q_C to resemble the true source, and the excess of L_C over the source entropy is exactly the Kullback–Leibler divergence between them. □

Two consequences govern how Fig. 1c should be read. Because L_C is an upper bound on each corpus and not an entropy, no single number in the figure is an information content. Because both corpora are measured with the *same* s and C, however, the comparison between them is like for like, and it is the ratio – to order of magnitude – that the figure claims.

The persistent-entity corpora. Supplementary Table 2 evaluates (22) for the public archives in the representation in which each is actually distributed. No payload is downloaded: the sizes come from HTTP `Content-Length`, from NCBI BLAST metadata, and from the wwPDB directory index, so every entry is re-derivable by a reader with a network connection. Nucleotide sequence dominates, and the total is

$$L_C(\mathcal{U}) \;\approx\; 9.8\times10^{12} \;\approx\; 10^{13} \text{ bits}, \tag{24}$$

falling only to 9.2×10^{12} if RefSeq transcripts are dropped as already contained in `nt`. An independent count corroborates the compressor. NCBI `nt` holds 4.15×10^{12} nucleotides, which a four-letter alphabet cannot write in fewer than 8.3×10^{12}

Supplementary Table 2 Persistent-entity corpora. Compressed size of each public archive in the representation it is actually distributed in, obtained without downloading any payload (HTTP `Content-Length`, NCBI BLAST metadata, or the wwPDB directory index). Compressed size $L_C(D) = 8|C(s(D))|$ is a computable upper bound on Kolmogorov complexity and a codelength under `gzip`'s implicit model, not an intrinsic information content; it is applied identically to both worlds of Fig. 1c, so the *ratio* is what is claimed. RefSeq transcripts also occur in `nt`, so the conservative total omits them. Sizes are decimal GB (10^9 bytes); PubChem and the PDB are rolling archives, so their *Release* is the fetch date. Measured 2026-07-13 (04:02:43 UTC) by the script named above and archived as a dated snapshot; no number here is hand-entered. These archives grow monotonically, so re-running the script returns a *larger* entity total and a larger ratio – the separation reported here is conservative.

Modality	Corpus	Release	Contents	GB	Bits
Protein	UniProtKB/TrEMBL	2026_02	149,234,636 sequences	40.5	3.2×10^{11}
Protein	UniProtKB/Swiss-Prot	2026_02	575,503 sequences	0.1	7.5×10^{8}
DNA / nucleotide	NCBI nt	2026-07-08	128,624,482 sequences (4.1×10^{12} nt)	884.1	7.1×10^{12}
RNA (transcripts)	NCBI RefSeq RNA	2026-06-26	81,647,541 sequences (2.4×10^{11} nt)	67.4	5.4×10^{11}
Small molecule	PubChem Compound	2026-07-13	357 SDF blocks, CID 1–178,500,000	115.8	9.3×10^{11}
Structure	wwPDB mmCIF	2026-07-13	255,710 structures	89.2	7.1×10^{11}
Total (all rows)				1,197.1	9.6×10^{12}
Total (non-overlapping: `nt` + TrEMBL + PubChem + PDB)				1,129.6	9.0×10^{12}

bits, and the archive is distributed in 7.2×10^{12}: the compressed size recovers 87% of a counting bound derived without reference to any compressor. The two routes agree, so (24) measures sequence content rather than an artifact of file formatting.

The contingent-observation corpus. The phenotype corpus integrated here (Supplementary Table 15) holds $\sim 10^7$–10^8 labeled instances across the studies it unifies. Serialized as the per-study label tables and compressed with the same instrument, it occupies $\sim 1.4 \times 10^9$ bytes, so

$$L_C(\mathcal{I}) \;\approx\; 1.1 \times 10^{10} \;\approx\; 10^{10} \text{ bits}, \tag{25}$$

about three orders of magnitude below (24). The corpus aggregates the large-scale yeast genotype–phenotype studies available to us under a single ontology, which is the point: the separation in Fig. 1c is not an artifact of having assembled a small phenotype corpus, and it would survive the addition of any comparable study.

Two asymmetries. The first is scale, and it is the weaker of the two: $L_C(\mathcal{U})/L_C(\mathcal{I}) \approx 10^3$. The second is persistence, and it is the reason the first matters. An entity representation is a function of the entity alone, so one sequence informs the encoder for *every* instance in which that entity appears, across assays, environments and laboratories. A phenotype record, by (21), constrains f at one point of the context space and nowhere else, and must be re-measured when the context changes. Scale alone would be an argument about sample size; persistence is an argument about what a bit buys.

Consequence for the model. An entity encoder is therefore fit where its data is, and the contingent corpus is spent only on what nothing else can pay for. This is the cut at H of Fig. 1c: F_θ maps persistent entities into the shared representation, and the phenotype labels fit only $\mathcal{T}_\psi$ and $\mathcal{R}_\phi$. The capacity arithmetic is one-sided. A sequence encoder carries 10^8–10^{10} parameters, whereas the operator and decoder together carry $\sim 5 \times 10^4$; and the *usable* supervision in the phenotype corpus is far below its compressed size, because the measurements are noisy and strongly redundant – interaction matrices are approximately low rank – which places the effective supervision nearer 10^6–10^7 bits than 10^{10}. Fitting an entity encoder from phenotype labels is thus underdetermined by orders of magnitude, while fitting the operator and decoder is not. Supplementary Note 6 is the empirical counterpart: across ~ 17 gene encodings, no pretrained biological embedding beats a one-hot code on fitness, which is what one expects when the labels cannot pay for a representation.

An objection, and what persistence answers. It may be objected that only the *yeast* entities are needed, and that the yeast proteome – some 3×10^6 residues, on the order of 10^7 bits – is smaller than the phenotype corpus, so the accounting would favour the labels. The objection is correct about the arithmetic and inverts the argument. An entity encoder is not fit on the yeast slice; it is fit on the whole archive and *applied* to the yeast slice, because a protein sequence means the same thing in every organism. Fungal DNA language models are built in exactly this way, pretrained across many fungal genomes and then read out on one [15]. That transfer is precisely what persistence buys, and it is why the relevant comparison is the full archive against the phenotype corpus rather than one organism's parts list against it.

Precision. Four things are not claimed. First, no number here is an information content: by Proposition 5, L_C bounds Kolmogorov complexity from above and is a codelength under `gzip`'s model, so only the ratio, and only to order of magnitude, is asserted. Second, compressed storage is not usable supervision; both sides are upper bounds on their effective content, the phenotype side discounted by noise and low rank, the entity side by homology redundancy. The latter is worth stating plainly, because it is the mechanism the architecture exploits: a protein language model compresses TrEMBL far better than `gzip` does, and the gap between the two is exactly the evolutionary structure a pretrained encoder supplies and `gzip` cannot see. Third, the archives overlap – RefSeq transcripts also occur in

`nt` – so Supplementary Table 2 reports a conservative total that omits the duplication. Fourth, corpora of *predicted* structures are excluded, being model outputs rather than observations; including them would widen the separation while weakening its interpretation.

Supplementary Note 6: a classical-machine-learning case study on gene representation and pooling ✗

Before introducing the Cell Graph Transformer we ask what simple models already achieve, and where they stall – because the answer dictates two architectural choices. Trained on 10^3–10^5 instances over ∼17 gene encodings, classical models (Supplementary Table 3, Supplementary Fig. 4) show a sharp split: *knockout fitness is essentially saturated by gene identity alone* – one-hot encoding reaches Pearson ≈ 0.87–0.90 at 10^5 and no pretrained biological embedding beats it – whereas *gene interactions are not learnable from any encoding*, plateauing at Pearson ≈ 0.4 (Spearman ≈ 0.47) regardless of representation or dataset size. The first result says a deep model is unnecessary for fitness; the second says the barrier to interactions lies not in the features but in how genes are combined.

Setup. A pooled bag of per-gene embeddings is about the simplest representation of a genome one can write down, and it is the object tested here. Each strain is featurized as $\phi = Mx$, where $x \in \{0,1\}^N$ ($N = 6607$) selects genes and the columns of M are per-gene vectors: one-hot ($M = I$), random (i.i.d., dimension 1–1000), or pretrained biological embeddings – the protein language models ESM2 and ProtT5, the codon language model CaLM, nucleotide-transformer windows, and a species-aware DNA language model [16]. The bag is taken over either the perturbed (deleted) genes or the intact genome and pooled by sum or mean, giving one fixed-length vector per strain (Supplementary Fig. 4a). Fitness data are single/double/triple deletion strains ($smf/dmf/tmf$, quality filter $\sigma_f < 0.15$); interaction data are an equal (≈50/50) mixture of digenic and trigenic Kuzmin 2018 measurements. Elastic net, random forest and support-vector regression are tuned per encoding by validation Spearman; reported values are held-out test scores, and the tables give the two nonlinear regressors, whose sweep is complete across all encodings and sizes.

Representation is about identity, not biology. On fitness a purely random code matches or exceeds every biological embedding once its dimension is large enough, and one-hot is best of all. The feature $\phi = Mx$ with ≤ 3 perturbed genes is a k-sparse recovery problem, so a random projection of dimension $d \gtrsim k \log N$ – a few hundred – already preserves gene identity (Johnson–Lindenstrauss; compressed sensing [cite: Johnson–Lindenstrauss 1984; Candès–Tao 2005]). Extra dimensions add no identity information, so accuracy rises with d and saturates at the exactly-orthogonal one-hot limit; a wider random vector cannot surpass it, and trails only for axis-aligned tree models, because a random projection is a rotation that removes the gene-aligned coordinates trees split on [cite: Grinsztajn et al. 2022]. Design consequence: preserve gene identity with a learnable token rather than hard-coding a biological prior at the reference cell.

The barrier is pooling, not features. Interactions are epistasis – defined by subtracting lower-order effects – so they live in the *combination* of perturbed genes. Additive sum/mean pooling of Mx is permutation-invariant and keeps only the bag of perturbed genes; the combination structure is discarded before the model sees it, and no choice of M can restore it. Hence the flat ≈ 0.47 ceiling across encodings and sizes. Within additive pooling, sum outperforms mean, and not by accident: sum is injective on the multiset of perturbed genes whereas mean discards its cardinality, making sum the strictly more expressive aggregator [cite: Xu et al. 2019, GIN]; concretely the number of deleted genes – and hence single/double/triple order – is linearly recoverable from a summed embedding but not a mean.

How far below the achievable does this plateau sit? The relevant bound is the *limit of experimental reproducibility* – the correlation between independent replicate measurements, which no predictor of a single measurement can exceed [cite: Ma et al. 2018, DCell]. Double-mutant fitness reproduces at r=0.89 across 211 duplicate genome-wide SGA screens [cite: Baryshnikova et al. 2010], so the one-hot fitness result (≈ 0.87–0.90) already sits *at* this ceiling – fitness is saturated by identity. Interaction scores are noisier but still highly reproducible: r=0.82–0.87 for digenic [cite: Baryshnikova et al. 2010][**?**] and r=0.74–0.81 for adjusted trigenic scores [**?**], a variance-weighted ceiling of r≈ 0.81–0.86 for the ≈50/50 mixture (dashed line, Supplementary Fig. 4). The classical plateau near 0.45 therefore lies far below the noise ceiling – the barrier is model capacity, not measurement error, and substantial reproducible interaction signal remains for a readout that preserves higher-order structure. Together these results motivate the two design choices of the Cell Graph Transformer – identity-preserving gene tokens (this note) and attention over the gene set in place of additive pooling (Supplementary Note 4) – so that higher-order structure survives to the readout.

Scope. These encodings are deliberately minimal. A pooled bag of pretrained gene vectors is among the simplest genome representations available, and the biological embeddings differ mainly in the domain they were pretrained on – protein, codon, or regulatory-DNA context – so what generality they carry is inherited from that pretraining rather than learned from phenotype here. Richer, learned cell representations exist and are advancing quickly [17]; the point is not that additive pooling of gene embeddings is the right way to represent a cell, but that even this minimal encoding already saturates fitness and stalls on interactions, which is what isolates the readout – not the features – as the object the Cell Graph Transformer must improve. This is an early-stage baseline, and it is meant to be one.

Supplementary Table 3 Classical-ML gene-representation benchmark – test Spearman ρ at $n = 10^3, 10^4, 10^5$ for random forest (RF) and support-vector regression (SVR), validation-selected best configuration per encoding. **Bold** = best encoding per column within each dataset; ± is the 5-fold CV s.d. ($n = 10^3, 10^4$; $n = 10^5$ single split). Regenerated by `traditional_ml-summary_table.py`.

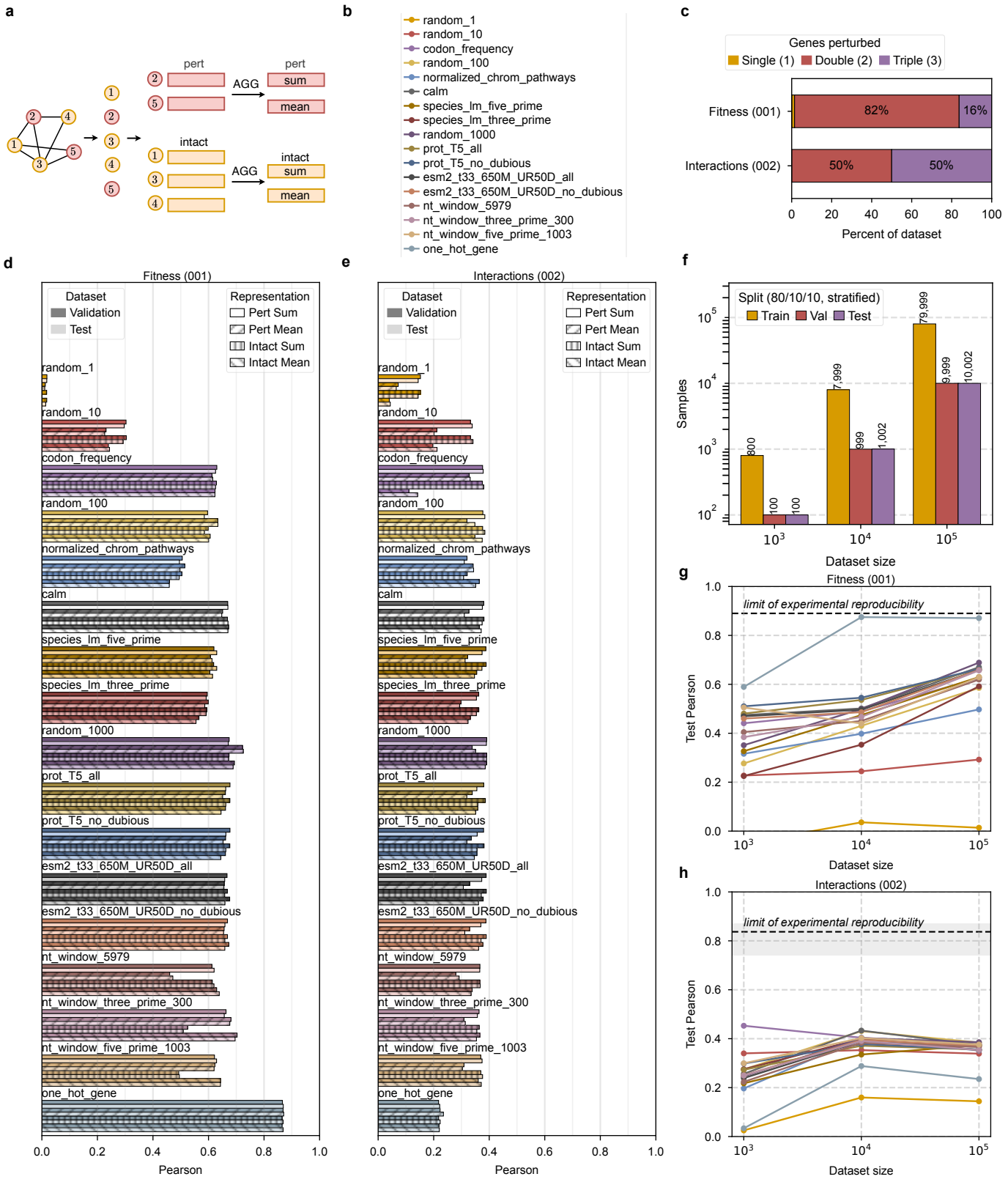
	RF			SVR		
Encoding	10^3	10^4	10^5	10^3	10^4	10^5
Fitness						
Random projection						
random (d=1)	0.013 ± 0.044	0.036 ± 0.015	0.033	-0.168 ± 0.036	0.044 ± 0.018	–
random (d=10)	0.138 ± 0.058	0.229 ± 0.023	0.263	0.075 ± 0.030	0.210 ± 0.016	–
random (d=100)	0.248 ± 0.057	0.422 ± 0.029	0.646	**0.532 ± 0.066**	0.642 ± 0.011	–
random (d=1000)	0.367 ± 0.053	0.565 ± 0.021	0.756	0.461 ± 0.046	0.763 ± 0.010	0.870 ± 0.002
Hand-crafted						
codon freq.	0.384 ± 0.055	0.452 ± 0.014	0.599	0.282 ± 0.043	0.286 ± 0.014	0.328
chrom. pathways	0.208 ± 0.053	0.353 ± 0.020	0.436	0.336 ± 0.060	0.307 ± 0.020	0.353
Biological						
CaLM	0.435 ± 0.067	0.510 ± 0.016	0.662	0.400 ± 0.054	0.719 ± 0.016	0.825
species LM up	0.296 ± 0.114	0.453 ± 0.027	0.635	0.356 ± 0.036	0.687 ± 0.012	0.818
species LM down	0.232 ± 0.110	0.391 ± 0.008	0.605	0.427 ± 0.030	0.651 ± 0.013	0.804
ProtT5	0.414 ± 0.050	0.522 ± 0.032	0.675	0.453 ± 0.080	0.737 ± 0.010	0.840
ProtT5 (nd)	0.421 ± 0.052	0.520 ± 0.030	0.677	0.453 ± 0.080	0.738 ± 0.010	0.841
ESM2	0.427 ± 0.049	0.510 ± 0.017	0.669	0.479 ± 0.043	0.774 ± 0.018	0.795
ESM2 (nd)	0.399 ± 0.061	0.506 ± 0.014	0.669	0.384 ± 0.031	0.777 ± 0.018	0.796
NT 5979	0.330 ± 0.038	0.427 ± 0.031	0.623	0.300 ± 0.043	0.802 ± 0.015	0.749
NT 3′	0.367 ± 0.066	0.477 ± 0.024	0.716	0.400 ± 0.061	0.797 ± 0.007	0.792
NT 5′	0.348 ± 0.061	0.432 ± 0.033	0.650	0.358 ± 0.051	0.788 ± 0.016	0.753
Identity						
one-hot	**0.561 ± 0.064**	**0.872 ± 0.002**	**0.886**	0.445 ± 0.060	**0.803 ± 0.009**	**0.900**
Interaction						
Random projection						
random (d=1)	0.183 ± 0.045	0.218 ± 0.022	0.209	0.240 ± 0.090	0.226 ± 0.017	0.209
random (d=10)	0.366 ± 0.120	0.461 ± 0.020	0.422	0.378 ± 0.100	0.470 ± 0.019	0.422
random (d=100)	0.411 ± 0.067	0.492 ± 0.013	0.451	0.415 ± 0.063	0.472 ± 0.022	0.445
random (d=1000)	0.322 ± 0.075	0.498 ± 0.015	0.466	**0.451 ± 0.088**	0.458 ± 0.383	**0.467**
Hand-crafted						
codon freq.	0.432 ± 0.089	0.487 ± 0.015	0.459	0.381 ± 0.078	0.486 ± 0.016	0.443
chrom. pathways	0.301 ± 0.092	0.487 ± 0.019	0.450	0.267 ± 0.080	0.446 ± 0.025	0.431
Biological						
CaLM	0.327 ± 0.081	**0.514 ± 0.021**	0.461	0.359 ± 0.096	0.484 ± 0.019	0.461
species LM up	0.404 ± 0.057	0.485 ± 0.009	**0.468**	0.448 ± 0.081	0.492 ± 0.016	0.461
species LM down	0.388 ± 0.064	0.485 ± 0.014	0.460	0.401 ± 0.067	0.477 ± 0.017	0.452
ProtT5	0.376 ± 0.065	0.485 ± 0.021	0.466	0.332 ± 0.103	0.493 ± 0.012	0.455
ProtT5 (nd)	0.353 ± 0.064	0.478 ± 0.021	0.468	0.332 ± 0.103	0.493 ± 0.012	0.455
ESM2	0.387 ± 0.062	0.501 ± 0.019	0.461	0.395 ± 0.083	**0.506 ± 0.017**	0.466
ESM2 (nd)	0.369 ± 0.061	0.490 ± 0.016	0.464	0.395 ± 0.083	**0.506 ± 0.017**	0.466
NT 5979	**0.435 ± 0.090**	0.453 ± 0.014	0.462	0.377 ± 0.081	0.491 ± 0.014	0.461
NT 3′	0.408 ± 0.124	0.500 ± 0.017	0.461	0.359 ± 0.078	0.485 ± 0.007	0.454
NT 5′	0.329 ± 0.070	0.502 ± 0.013	0.467	0.375 ± 0.085	0.461 ± 0.012	0.462
Identity						
one-hot	0.193 ± 0.051	0.401 ± 0.022	0.446	0.294 ± 0.078	0.478 ± 0.018	0.464

Supplementary Table 4 Classical-ML gene-representation benchmark – test MSE ($\times 10^{-3}$) at $n = 10^3, 10^4, 10^5$ for random forest (RF) and support-vector regression (SVR), validation-selected best configuration per encoding. **Bold** = best encoding per column within each dataset; ± is the 5-fold CV s.d. ($n = 10^3, 10^4$; $n = 10^5$ single split). †SVR fit diverged: configurations are selected by validation Spearman, which is scale-invariant, so a well-ranked fit can still be badly scale-calibrated and carry an enormous squared error. Regenerated by `traditional_ml-summary_table.py`.

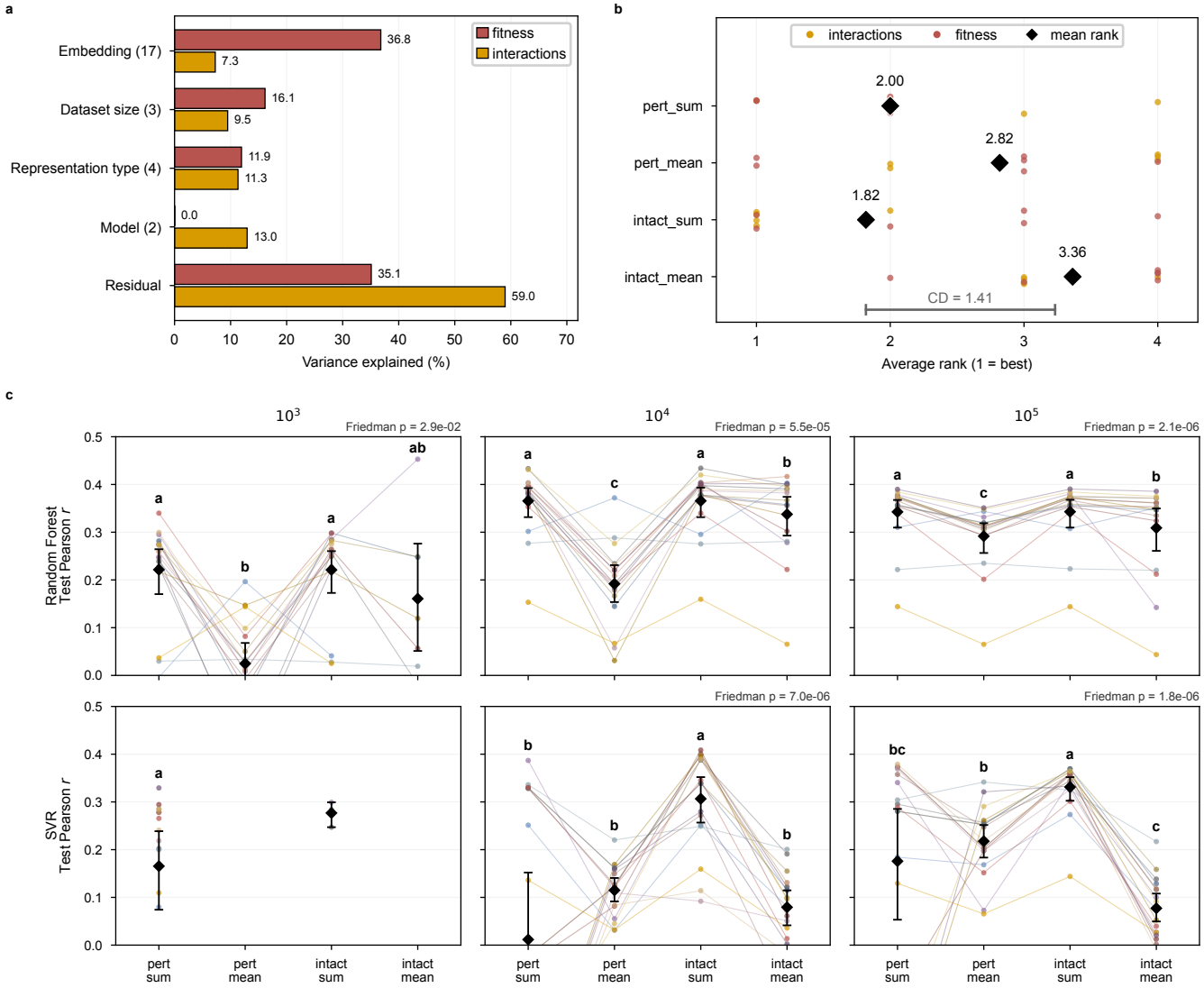
	RF			SVR		
Encoding	10^3	10^4	10^5	10^3	10^4	10^5
Fitness						
Random projection						
random (d=1)	25.6 ± 3.0	19.9 ± 0.7	21.3	23.5 ± 3.0	19.9 ± 0.6	–
random (d=10)	22.3 ± 2.7	18.9 ± 0.8	19.4	23.1 ± 2.8	18.9 ± 0.8	–
random (d=100)	22.1 ± 2.6	16.6 ± 0.6	15.3	20.8 ± 2.7	11.8 ± 0.4	–
random (d=1000)	20.2 ± 2.3	15.8 ± 0.7	13.6	22.8 ± 2.6	8.2 ± 0.2	5.1 ± 0.2
Hand-crafted						
codon freq.	18.6 ± 2.5	15.1 ± 0.4	13.5	20.5 ± 2.8	18.1 ± 0.8	18.9
chrom. pathways	22.0 ± 2.8	16.9 ± 0.6	16.0	20.2 ± 2.6	17.8 ± 0.8	17.8
Biological						
CaLM	18.6 ± 2.8	14.9 ± 0.4	12.7	21.1 ± 2.9	9.4 ± 0.2	6.5
species LM up	21.0 ± 1.2	15.8 ± 0.8	13.7	20.1 ± 2.6	11.0 ± 0.7	6.8
species LM down	22.2 ± 1.8	17.4 ± 0.7	14.9	21.5 ± 3.5	10.9 ± 0.5	7.3
ProtT5	19.0 ± 2.4	14.5 ± 0.5	12.8	17.8 ± 2.4	8.0 ± 0.5	6.5
ProtT5 (nd)	18.8 ± 2.3	14.6 ± 0.4	12.8	17.8 ± 2.4	8.0 ± 0.5	6.3
ESM2	18.7 ± 2.4	15.4 ± 0.5	13.1	$\mathbf{17.7 \pm 2.6}$	7.3 ± 0.8	7.3
ESM2 (nd)	19.2 ± 2.6	15.4 ± 0.6	12.9	17.7 ± 2.8	7.2 ± 0.8	7.3
NT 5979	20.0 ± 2.3	16.3 ± 0.6	14.3	28.7 ± 2.1	5.9 ± 0.4	8.7
NT 3′	21.1 ± 2.7	15.9 ± 0.6	13.5	19.7 ± 2.8	6.1 ± 0.4	9.9
NT 5′	20.4 ± 2.3	16.3 ± 0.8	14.0	20.8 ± 2.8	6.6 ± 0.4	10.0
Identity						
one-hot	$\mathbf{17.2 \pm 4.3}$	$\mathbf{4.8 \pm 0.3}$	**5.4**	20.2 ± 2.9	$\mathbf{5.9 \pm 0.4}$	**4.5**
Interaction						
Random projection						
random (d=1)	4.4 ± 0.9	3.6 ± 0.4	3.8	4.3 ± 0.9	3.9 ± 0.3	4.6×10^{16} †
random (d=10)	$\mathbf{3.9 \pm 1.1}$	3.2 ± 0.4	3.4	4.3 ± 0.9	$\mathbf{3.7 \pm 0.3}$	4.6
random (d=100)	4.1 ± 1.1	$\mathbf{2.9 \pm 0.4}$	3.3	$\mathbf{4.1 \pm 0.9}$	4.9 ± 0.3	4.4
random (d=1000)	4.2 ± 1.2	3.1 ± 0.4	**3.3**	4.4 ± 0.9	5.7×10^{13} †	4.3
Hand-crafted						
codon freq.	3.9 ± 1.1	3.0 ± 0.4	3.3	4.1 ± 1.0	4.2 ± 0.4	4.8
chrom. pathways	4.3 ± 1.0	3.1 ± 0.4	3.4	4.4 ± 0.9	4.1 ± 0.4	4.8
Biological						
CaLM	4.4 ± 1.1	2.9 ± 0.4	3.3	4.2 ± 1.1	4.2 ± 0.3	4.6
species LM up	4.2 ± 1.1	3.1 ± 0.4	3.4	5.3 ± 1.2	3.9 ± 0.4	4.6
species LM down	4.2 ± 1.0	3.1 ± 0.4	3.4	4.9 ± 1.2	4.4 ± 0.4	4.2
ProtT5	4.2 ± 1.1	3.1 ± 0.4	3.4	4.3 ± 1.0	4.0 ± 0.4	4.5
ProtT5 (nd)	4.3 ± 1.1	3.1 ± 0.4	3.4	4.3 ± 1.0	4.0 ± 0.4	4.5
ESM2	4.1 ± 1.1	3.1 ± 0.4	3.4	4.6 ± 1.2	4.2 ± 0.4	4.2
ESM2 (nd)	4.3 ± 1.1	3.1 ± 0.4	3.4	4.6 ± 1.2	4.2 ± 0.4	4.2
NT 5979	4.3 ± 1.0	3.2 ± 0.4	3.4	4.7 ± 1.2	4.3 ± 0.4	**4.2**
NT 3′	4.1 ± 1.2	3.1 ± 0.4	3.4	5.9 ± 0.9	4.7 ± 0.3	4.6
NT 5′	4.3 ± 0.9	3.0 ± 0.4	3.4	4.2 ± 1.1	4.5 ± 0.4	4.6
Identity						
one-hot	4.7 ± 1.1	3.5 ± 0.4	3.8	4.5 ± 0.9	3.8 ± 0.4	4.2

Supplementary Table 5 Classical-ML gene-representation benchmark – test Pearson r at $n = 10^3, 10^4, 10^5$ for random forest (RF) and support-vector regression (SVR), validation-selected best configuration per encoding. **Bold** = best encoding per column within each dataset; ± is the 5-fold CV s.d. ($n = 10^3, 10^4$; $n = 10^5$ single split). Regenerated by `traditional_ml-summary_table.py`.

	RF			SVR		
Encoding	10^3	10^4	10^5	10^3	10^4	10^5
Fitness						
Random projection						
random (d=1)	-0.060 ± 0.056	0.069 ± 0.023	0.014	-0.027 ± 0.019	0.066 ± 0.025	–
random (d=10)	0.227 ± 0.079	0.244 ± 0.031	0.296	0.136 ± 0.046	0.235 ± 0.030	–
random (d=100)	0.236 ± 0.034	0.430 ± 0.024	0.601	0.506 ± 0.049	0.655 ± 0.009	–
random (d=1000)	0.446 ± 0.100	0.502 ± 0.022	0.689	0.448 ± 0.039	0.791 ± 0.007	0.881 ± 0.004
Hand-crafted						
codon freq.	0.490 ± 0.038	0.512 ± 0.009	0.625	0.371 ± 0.061	0.311 ± 0.022	0.341
chrom. pathways	0.250 ± 0.066	0.394 ± 0.017	0.497	0.392 ± 0.068	0.336 ± 0.025	0.404
Biological						
CaLM	0.493 ± 0.051	0.529 ± 0.007	0.670	0.393 ± 0.087	0.738 ± 0.015	0.837
species LM up	0.317 ± 0.103	0.469 ± 0.034	0.630	0.389 ± 0.036	0.709 ± 0.022	0.829
species LM down	0.225 ± 0.091	0.368 ± 0.021	0.592	0.410 ± 0.038	0.696 ± 0.026	0.815
ProtT5	0.480 ± 0.068	0.549 ± 0.022	0.662	0.502 ± 0.045	0.777 ± 0.020	0.836
ProtT5 (nd)	0.510 ± 0.074	0.545 ± 0.021	0.663	0.502 ± 0.045	0.776 ± 0.019	0.842
ESM2	0.480 ± 0.062	0.496 ± 0.010	0.659	**0.516 ± 0.055**	0.803 ± 0.024	0.815
ESM2 (nd)	0.458 ± 0.069	0.496 ± 0.010	0.659	0.501 ± 0.071	0.805 ± 0.025	0.815
NT 5979	0.406 ± 0.073	0.444 ± 0.027	0.639	0.230 ± 0.076	0.842 ± 0.015	0.781
NT 3′	0.328 ± 0.069	0.482 ± 0.017	0.696	0.416 ± 0.084	0.837 ± 0.014	0.801
NT 5′	0.389 ± 0.064	0.441 ± 0.030	0.643	0.340 ± 0.061	0.826 ± 0.017	0.773
Identity						
one-hot	**0.589 ± 0.090**	**0.873 ± 0.010**	**0.871**	0.491 ± 0.055	**0.844 ± 0.013**	**0.902**
Interaction						
Random projection						
random (d=1)	0.073 ± 0.068	0.158 ± 0.016	0.145	0.110 ± 0.085	0.158 ± 0.014	0.144
random (d=10)	**0.337 ± 0.091**	0.353 ± 0.036	0.339	0.279 ± 0.087	0.339 ± 0.033	0.329
random (d=100)	0.280 ± 0.059	**0.448 ± 0.039**	0.373	0.284 ± 0.095	0.383 ± 0.037	0.361
random (d=1000)	0.213 ± 0.106	0.401 ± 0.033	**0.390**	0.155 ± 0.037	0.388 ± 0.298	**0.384**
Hand-crafted						
codon freq.	0.326 ± 0.100	0.414 ± 0.035	0.377	0.292 ± 0.100	0.398 ± 0.038	0.346
chrom. pathways	0.139 ± 0.063	0.395 ± 0.038	0.350	0.097 ± 0.065	0.363 ± 0.044	0.345
Biological						
CaLM	0.190 ± 0.069	0.441 ± 0.031	0.379	0.247 ± 0.084	0.381 ± 0.033	0.364
species LM up	0.239 ± 0.073	0.381 ± 0.038	0.372	**0.302 ± 0.099**	0.400 ± 0.029	0.362
species LM down	0.260 ± 0.051	0.400 ± 0.039	0.342	0.273 ± 0.093	0.398 ± 0.036	0.373
ProtT5	0.276 ± 0.068	0.381 ± 0.027	0.352	0.223 ± 0.094	**0.410 ± 0.034**	0.372
ProtT5 (nd)	0.252 ± 0.073	0.384 ± 0.028	0.354	0.223 ± 0.094	**0.410 ± 0.034**	0.372
ESM2	0.280 ± 0.083	0.389 ± 0.037	0.368	0.270 ± 0.085	0.405 ± 0.037	0.382
ESM2 (nd)	0.243 ± 0.085	0.388 ± 0.038	0.372	0.270 ± 0.085	0.405 ± 0.037	0.382
NT 5979	0.224 ± 0.108	0.376 ± 0.023	0.360	0.300 ± 0.097	0.402 ± 0.039	0.380
NT 3′	0.268 ± 0.109	0.395 ± 0.035	0.358	0.186 ± 0.067	0.404 ± 0.028	0.360
NT 5′	0.247 ± 0.046	0.430 ± 0.039	0.368	0.241 ± 0.088	0.377 ± 0.029	0.369
Identity						
one-hot	0.131 ± 0.073	0.288 ± 0.040	0.343	0.118 ± 0.052	0.399 ± 0.029	0.373



Supplementary Figure 4 Classical machine-learning baselines: fitness is saturable, interactions are not. a, Featurization. Each strain is reduced to a single fixed-length vector by mapping its genes through a per-gene encoder and pooling: the bag is taken over either the perturbed (deleted) genes or the intact genome and aggregated by sum or mean. **b**, The seventeen gene representations, grouped as identity (one-hot), random projections (d=1–1000), hand-crafted (codon frequency, chromosome/pathway features), and pretrained biological encoders (CaLM, a species-aware DNA language model [16], nucleotide-transformer windows, ProtT5, ESM2). **c**, Perturbation composition of the two datasets: knockout fitness (001; single/double/triple deletions) and the digenic/trigenic interaction mixture (002). (d,e) Validation and test Pearson r at n=10^5 for random forest across all representations and the four pooling variants (perturbed/intact × sum/mean), hyperparameters selected on validation MSE; Spearman and MSE views, and the support-vector and elastic-net baselines, are tabulated in Supplementary Table 3–5. **f**, Stratified 80/10/10 train/validation/test splits at each dataset size. (g,h) Best test Pearson per representation as dataset size grows n=10^3–10^5; the dashed line and shaded band mark the limit of experimental reproducibility (fitness r≈0.89; interactions r≈0.81–0.86). Fitness is saturated by gene identity – one-hot reaches Pearson ≈ 0.87 at n=10^5, approaching the reproducibility limit, and no pretrained embedding surpasses it – whereas interactions plateau near Pearson ≈ 0.4 regardless of representation or scale, the empirical backing for the information-accounting cut of Supplementary Note 5.



Supplementary Figure 5 Representation type – perturbed vs intact gene bag, sum vs mean pooling – is a consistent, second-order design choice. Every record is the best run by validation R^2 for one (task, model, dataset size, embedding, representation type) combination, scored as held-out test Pearson r (the same selection rule as Supplementary Fig. 4). **a**, Variance decomposition, computed separately for fitness (n=384 records) and gene interactions (n=350): type-II ANOVA over main effects, reporting $\eta^2 = \mathrm{SS}_{\mathrm{factor}}/\mathrm{SS}_{\mathrm{total}}$; parenthetical counts are each factor's number of levels, and the residual absorbs factor interactions and run-to-run noise. For interactions, representation type (11.3%) explains more variance than embedding identity (7.3%), whereas the large fitness embedding share (36.8%) restates the one-hot saturation of Supplementary Fig. 4. **b**, Average rank of the four representation types across the 11 complete task×model×scale configurations (dot = one configuration's rank, colored by task; diamond = mean rank; 1 = best). Friedman omnibus p=0.016; the ruler is the Nemenyi critical difference (CD=1.41, α=0.05), which formally separates only the extremes, intact_sum (mean rank 1.82) vs intact_mean (3.36). **c**, Gene-interaction detail: test Pearson by representation type for random forest and SVR at each dataset size; faint line = one embedding (17 total), diamond = mean with 95% bootstrap CI. Letters are a compact letter display – types sharing a letter are not significantly different (pairwise Wilcoxon, BH-FDR corrected), shown only when that panel's Friedman omnibus is significant (p above each panel). intact_sum sits in the top letter group in every complete panel and pert_mean is significantly worse beyond n=10^3; the missing points at SVR 10^3 reflect an incomplete sweep.

Supplementary Note 7: gene–gene graphs used as attention priors ✗

Nine gene–gene graphs regularize the cell encoder's attention in the trigenic experiment, one head per graph, and each enters the model only through the soft prior Ω_k of Eq. 16: none is used for message passing and none is imposed as a hard mask. Supplementary Table 6 and Supplementary Fig. 6 describe each graph on its own: size and coverage, degree distribution, neighborhood structure, hubs, and its relation to the other components of the cell representation. Supplementary Fig. 7 describes how the graphs relate to each other: overlap, containment, shared pairs, multiplicity, and the agreement of the two transcription-factor graphs. The drift of the STRING channels across releases is in Supplementary Table 7 and, because it motivates DANGO's release-based channel weights, in Supplementary Fig. 8a.

Sources. Two graphs are curated from the *Saccharomyces* Genome Database (SGD; Cherry et al. 1998): *physical*, the protein–protein interactions SGD records per gene, and *regulatory*, SGD's regulation annotations as directed regulator→target edges. *TFLink* (v1.0; Liska et al. 2022) is a directed transcription-factor–target compendium. The remaining six are the evidence channels STRING v12.0 (Szklarczyk et al. 2023) scores separately for *S. cerevisiae*: neighborhood, fusion, co-occurrence, co-expression, experimental, and database; an edge is any gene pair with a nonzero channel score, the cut DANGO trains on. Every graph is restricted to the 6,607-gene S288C reference; the SGD regulation records also name 35 protein-complex regulators and 405 non-coding features, and the 1,869 of 39,636 records that involve them are dropped, as they are when the graph is converted to gene indices for training.

Size, coverage, and structure. The graphs differ by a factor of three in the genes they reach (2,191 to 6,474, leaving 133 to 4,416 genes of the reference without an edge; together they reach 6,562 genes and leave 45) and by two orders of magnitude in edges (11,085 to 996,199; 1,514,071 distinct pairs in all) (Supplementary Fig. 6a; Supplementary Table 6). Since the prior pulls a gene's attention toward its neighbors, what a graph exposes to the model is its neighborhoods (Supplementary Fig. 6b, c): fusion and co-occurrence have no gene above degree 100 while TFLink carries the heaviest tail; eight graphs are essentially one connected component, co-occurrence splits into 486 near-clique components; and the mean number of genes within two edges of a gene spans three orders of magnitude, from 5,038 (TFLink) to 22 (co-occurrence). The highest-degree gene differs in every graph, and of the 74 genes in the nine top-ten lists only 14 recur (Supplementary Fig. 6d).

Other components. Supplementary Fig. 6e relates each graph to the metabolic network, the gene ontology, the essentiality labels, and the trigenic training genes of the same cell representation. The three genome-context STRING channels and the database channel are enriched for Yeast9 metabolic genes (26% to 39% of their covered genes, against 18% of the reference); the other five stay within five points of the reference, and the share of essential genes stays within 16% to 24% for every graph. The five graphs that reach over 5,000 genes give an edge to 96% to 100% of the 1,400 Kuzmin 2018 trigenic genes and to 82% to 99% of the 4,308 Kuzmin 2020 genes; database to 77% and 65%, and neighborhood, fusion, and co-occurrence to 34% to 53%. Against a degree-preserving random graph, physical, database, and co-occurrence edges share a Yeast9 reaction 24 to 150 times more often than chance (0.3%, 0.9%, and 4.2% of edges), and every STRING channel and the physical graph share a GO biological process 2.7 to 7.5 times more often (15% to 67% of edges). The two transcription-factor graphs are the exception on every pair-level measure: their regulator–target pairs share a reaction, a subsystem, or a process no more often than the random reference (7% of edges share a process, against 10% to 11% at random).

Overlap. The graphs are complementary rather than redundant: the Jaccard index of edge sets is at most 0.12 for every pair but one, co-expression against experimental (0.44) (Supplementary Fig. 7a), and 52% of the 1,514,071 distinct pairs appear in exactly one graph, none in more than seven (Supplementary Fig. 7d). Containment is asymmetric (Supplementary Fig. 7b): 76% of physical and 82% of neighborhood edges lie inside a dense STRING channel, and 45% of regulatory edges lie inside TFLink whereas 8% of TFLink edges are regulatory. Regulatory and TFLink share 200 of their 590 regulators and 4,963 of 6,252 targets but only 16,817 of 221,751 directed edges (7.6%), and the median Jaccard index between a shared regulator's two target sets is 0.03 (Supplementary Fig. 7e); they are kept as separate priors.

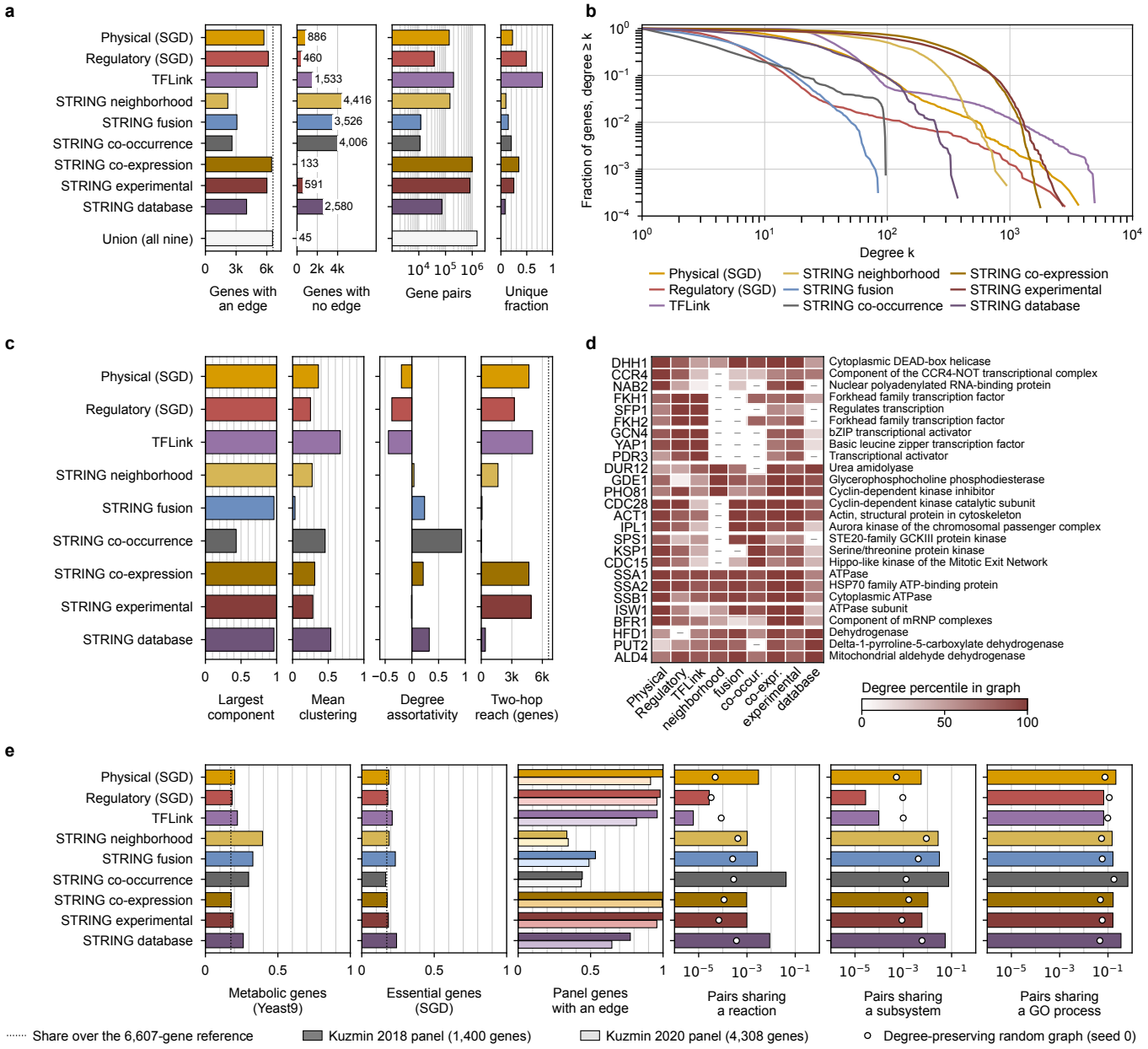
STRING releases. DANGO was built on STRING v9.1 and set its per-channel weights from the change to v11.0; the CGT experiment uses v12.0. Every channel grows at each release and loses part of what it had (Supplementary Fig. 8a; Supplementary Table 7): a channel retains 42% to 72% of its pairs from v9.1 to v11.0 and 51% to 75% from v11.0 to v12.0, and only 17% (fusion) to 45% (neighborhood) of a v12.0 channel's pairs were already in v11.0. Whether that difference moves DANGO is tested in Supplementary Note 8.

Supplementary Table 6 Gene–gene graphs that serve as attention priors in the trigenic experiment, over the shared S288C vocabulary of 6,607 genes. **Physical** and **Regulatory** are the curated SGD graphs; TFLink and the six STRING v12.0 evidence channels add the remaining rows. *Edges* is the native count (directed for Regulatory and TFLink), *Pairs* the distinct unordered gene pairs used for overlap, *Unique* the share of those pairs found in no other graph. The nine graphs together cover 1,514,071 distinct gene pairs and 6,562 of the 6,607 genes (45 genes have an edge in no graph); relative to the SGD-native baseline (Physical + Regulatory), the sum over graphs is 3.48× in nodes and 13.76× in edges. (STRING v9.1 and v11.0 are in Supplementary Table 7.)

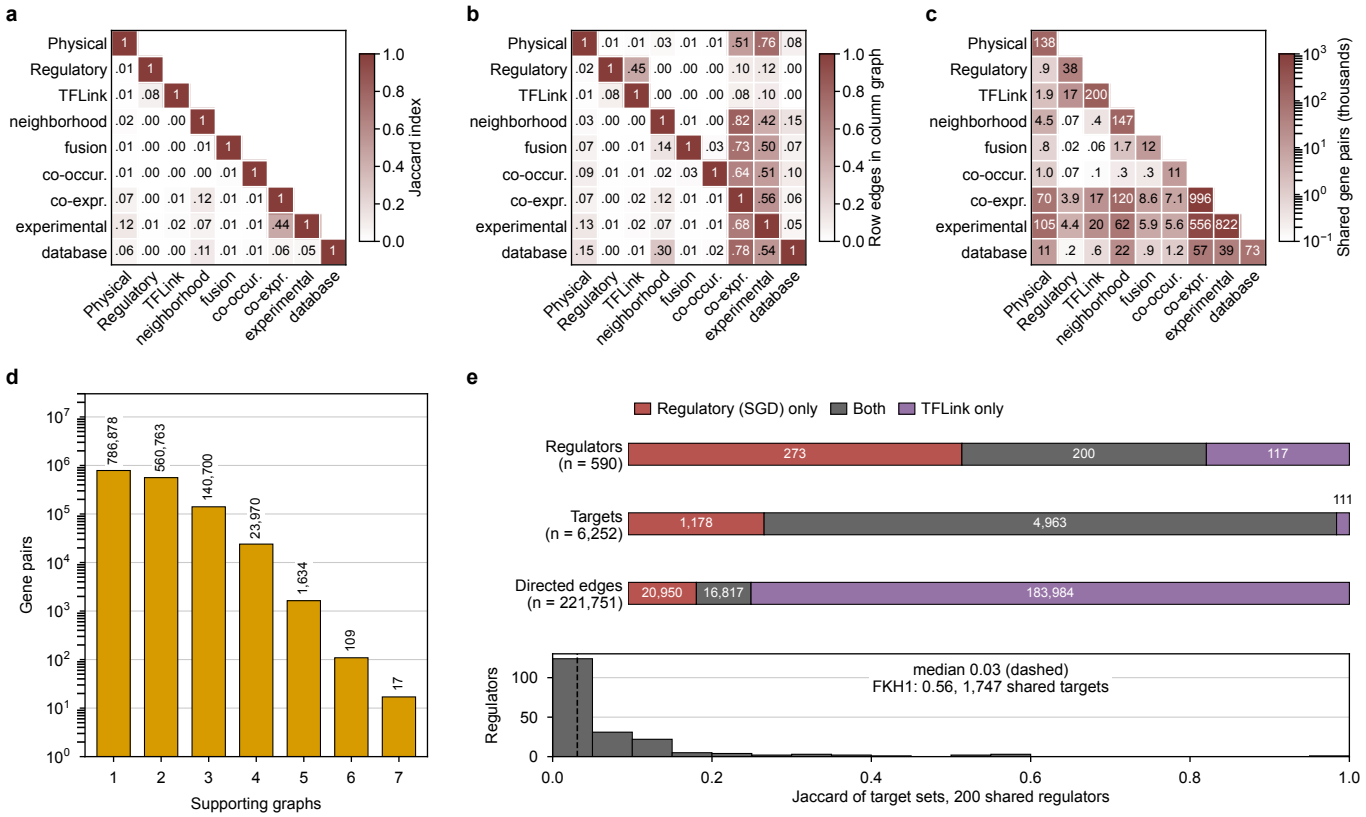
Graph	**Nodes**	**Edges**	**Pairs**	**Mean degree**	**Unique**
Physical (SGD)	5,721	139,463	137,604	48.1	23%
Regulatory (SGD)	6,147	37,767	37,677	12.3	49%
TFLink	5,074	200,801	199,551	78.7	80%
STRING neighborhood	2,191	146,713	146,713	133.9	10%
STRING fusion	3,081	11,787	11,787	7.7	14%
STRING co-occurrence	2,601	11,085	11,085	8.5	20%
STRING co-expression	6,474	996,199	996,199	307.8	35%
STRING experimental	6,016	822,094	822,094	273.3	25%
STRING database	4,027	72,617	72,617	36.1	8%
Sum (all graphs)	41,332	2,438,526	2,435,327		
Union (all nine)	6,562		1,514,071		
Sum (Physical, Regulatory)	11,868	177,230	175,281		

Supplementary Table 7 STRING evidence channels for *S. cerevisiae* across the three releases integrated into TorchCell, over the shared 6,607-gene vocabulary. Nodes are genes with at least one edge; edges are distinct unordered gene pairs with a nonzero channel score; *Retained* is the share of a release's pairs still present in the next release. DANGO was published on v9.1 with a v9.1→v11.0 comparison; the CGT trigenic experiment uses v12.0.

	Nodes			**Edges**			**Retained**	
Channel	v9.1	v11.0	v12.0	v9.1	v11.0	v12.0	9.1→11.0	11.0→12.0
neighborhood	2,170	2,115	2,191	45,601	120,965	146,713	72%	54%
fusion	1,187	2,338	3,081	1,358	3,914	11,787	42%	51%
co-occurrence	1,268	1,307	2,601	2,659	4,668	11,085	71%	57%
co-expression	5,796	5,654	6,474	313,688	554,833	996,199	59%	64%
experimental	6,079	5,964	6,016	219,450	392,772	822,094	56%	75%
database	2,712	3,081	4,027	33,486	46,816	72,617	59%	56%
Sum	19,212	20,459	24,390	616,242	1,123,968	2,060,495		



Supplementary Figure 6 Each gene–gene graph used as an attention prior in the trigenic experiment, on its own. Directed graphs (regulatory, TFLink) are compared as undirected gene pairs; every graph is restricted to the 6,607-gene reference; matrix axes in **d**, drop the STRING prefix. **a**, Per graph and, in the last row, for the union of all nine: genes with at least one edge (dotted line, the 6,607-gene reference); genes with no edge, counted at right (45 genes have an edge in no graph); distinct gene pairs; and the fraction of each graph's pairs found in no other graph (undefined for the union). **b**, Complementary cumulative degree distribution: fusion and co-occurrence have no gene above degree 100, the two dense STRING channels fall off past degree 10^3, and TFLink carries the heaviest tail. **c**, Per graph: fraction of covered genes in the largest connected component, mean clustering coefficient, degree assortativity, and mean number of genes within two edges of a gene (dotted line, the 6,607-gene reference); the four dense graphs place over 70% of the vocabulary within two edges of a typical gene, fusion and co-occurrence under 2%. **d**, The three highest-degree genes of every graph (26 genes; NAB2 is third in physical and first in experimental), grouped by the graph they top, in the order of (a), and by degree within a group, colored by degree percentile in each graph (dash, absent from the graph); at right, the leading phrase of the gene's SGD description (first clause, parentheticals removed, cut at a phrase boundary within 50 characters). Top-degree gene per graph: DHH1 in physical (3,606 partners), FKH1 in regulatory (2,787), GCN4 in TFLink (4,908), DUR12 in neighborhood (936), CDC28 in fusion (84), SPS1 and KSP1 tied in co-occurrence (97; ties are ordered by systematic name), SSA1 in co-expression (1,770), NAB2 in experimental (2,696), HFD1 in database (374); at the top 1% (66 genes per graph), 357 genes are hubs of one graph, 77 of two, 22 of three, 3 of four, and CDC28 of five. **e**, Relation to the other components of the cell representation: share of covered genes that are Yeast9 (yeast-GEM 9.0.2) metabolic genes, that is, genes of a gene–reaction rule (1,161), or essential, an SGD null-mutant *inviable* phenotype in S288C (1,140), the dotted line being the same share over the 6,607-gene reference; node coverage of the Kuzmin 2018 (graph color) and Kuzmin 2020 (its pale fill) trigenic gene panels, that is, the fraction of the panel's genes with at least one edge in the graph; and, on logarithmic axes, share of the graph's gene pairs whose genes share a Yeast9 reaction, a Yeast9 subsystem (107 subsystems), or a GO biological-process term (direct SGD annotations, root excluded, at most 500 genes per term; 1,944 terms), the open circle being the same share in a degree-preserving random graph (configuration model, seed 0). Source: `experiments/010-kuzmin-tmi/scripts/graph_statistics.py`.



Supplementary Figure 7 How the nine attention-prior graphs relate to each other. Directed graphs are compared as undirected gene pairs; matrix axes in **a**–**c** drop the STRING prefix. **a**, Jaccard index of edge sets. **b**, Containment: the fraction of the row graph's pairs present in the column graph. **c**, Gene pairs shared by each pair of graphs, in thousands (diagonal, the graph's own count; logarithmic color): co-expression and experimental share 556,106 pairs, neighborhood and co-expression 119,758, physical and experimental 104,771, the two transcription-factor graphs 16,885. **d**, Distinct gene pairs supported by exactly n graphs (52% in one, 37% in two, none in more than seven); the nine edge sets sum to 2,435,327 pairs, 1,514,071 distinct, 6.9% of the 21.8 million pairs the vocabulary allows. **e**, Top: regulators, targets, and directed regulator→target edges of the SGD regulatory graph (473 regulators, 37,767 edges) and TFLink (317 regulators, 200,801 edges), split into one-graph-only (20,950 regulatory, 183,984 TFLink edges) and both (16,817). Bottom: for the 200 shared regulators, the Jaccard index between the regulator's two target sets; the best case is FKH1 at 0.56 (1,747 shared targets). Source: `experiments/010-kuzmin-tmi/scripts/graph_statistics.py`.

Supplementary Note 8: constructing DANGO in TorchCell and reproducing its published result ✗

DANGO (Zhang et al. 2020) predicts a trigenic interaction from the six STRING evidence channels: one graph encoder per channel learns gene embeddings by reconstructing that channel, a learned average merges them, and a hypergraph self-attention head scores the three perturbed genes. It was rebuilt inside TorchCell rather than run from its released code. On the Kuzmin 2018 trigenic screen alone [4] the rebuilt model reaches a best validation Pearson r (maximum over epochs) of 0.42 in every one of 19 runs, whichever STRING release supplies the channels and whichever schedule hands over from pretraining to the interaction loss (Supplementary Fig. 8d, e; Supplementary Table 8). That plateau is close to the published value and cannot be equated with it, for the reasons given below; it is the baseline that Supplementary Note 9 retrains on the full trigenic dataset.

Model in the manuscript's notation. DANGO reads the six channels as adjacencies $A^{(k)} \in \{0,1\}^{N\times N}$ over the same $N = 6607$ vocabulary as the CGT (Supplementary Note 7), but as message-passing edges rather than attention priors (Supplementary Fig. 8b): a shared lookup $\mathbf{E} \in \mathbb{R}^{N\times d}$, $d = 64$, feeds one two-layer GraphSAGE encoder per channel, $H^{(k)} = F^{(k)}(\mathbf{E}, A^{(k)})$, whose linear head $\hat{A}^{(k)} = H^{(k)}W_k$ reconstructs the channel under a mean squared error with zero entries weighted by a per-channel λ_k,

$$\mathcal{L}_{\text{rec}} = \frac{1}{6}\sum_{k=1}^{6}\frac{1}{N^2}\sum_{i,j}\left(\hat{A}_{ij}^{(k)} - A_{ij}^{(k)}\right)^2\left[\mathbf{1}\left(A_{ij}^{(k)} \neq 0\right) + \lambda_k\,\mathbf{1}\left(A_{ij}^{(k)} = 0\right)\right]. \tag{26}$$

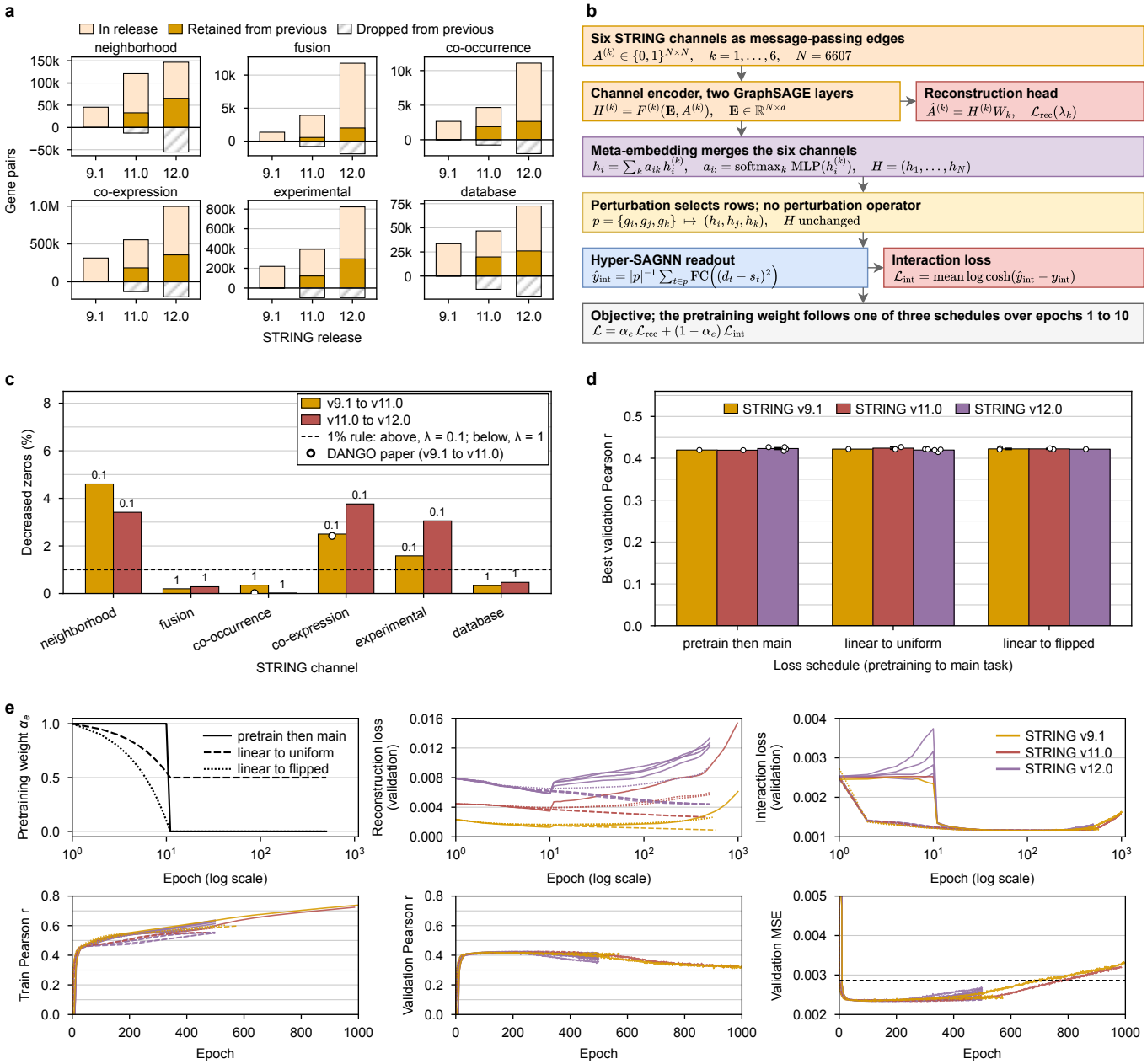
A meta-embedding $h_i = \sum_k a_{ik}h_i^{(k)}$, $a_{i:} = \text{softmax}_k\,\text{MLP}(h_i^{(k)})$, plays the role of Eq. 7; there is no perturbation operator, a strain $p = \{g_i, g_j, g_k\}$ selects rows of H, and the Hyper-SAGNN readout scores them under $\mathcal{L}_{\text{int}} = \text{mean}\log\cosh(\hat{y}_{\text{int}} - y_{\text{int}})$. TorchCell folds DANGO's pretraining and end-to-end stages into one run, $\mathcal{L} = \alpha_e\mathcal{L}_{\text{rec}} + (1-\alpha_e)\mathcal{L}_{\text{int}}$, with α_e following one of three schedules over the first ten epochs (Supplementary Fig. 8b, bottom, and e, top left).

Per-channel zero weights. DANGO sets $\lambda_k = 0.1$ where a channel's "percentage of decreased zeroes" from STRING v9.1 to v11.0 exceeds 1% and $\lambda_k = 1$ otherwise, reporting 0.02% (co-occurrence) to 2.42% (co-expression) without defining the computation. The rule responds to the release drift of Supplementary Fig. 8a, where every channel gains pairs at each release and loses part of what it had. TorchCell fixes one computation (Supplementary Fig. 8c): over the genes present in both releases, the pairs that were non-edges in the older release and edges in the newer, divided by the older release's non-edge pairs. Neighborhood, co-expression, and experimental exceed 1% (4.60%, 2.50%, 1.58%; $\lambda_k = 0.1$) and the other three do not (0.20%, 0.35%, 0.33%; $\lambda_k = 1$); co-expression lands near the paper's 2.42% and co-occurrence does not match its 0.02%. Read from the code, not measured in training: the training script looks the weights up under v9.1 channel names and the loss defaults a missing channel to $\lambda_k = 1$, so the v11.0 and v12.0 runs trained with $\lambda_k = 1$ for every channel. Whether that matters is not measured.

Result, and what is not matched. The screen was queried from the knowledge graph into 91,050 triple-mutant instances over 1,400 genes; no test-split metric was logged. Across the 19 runs the best validation r spans 0.415 to 0.427 (release-by-schedule means 0.420 to 0.424, SEM at most 0.003), peaks between epochs 106 and 439, and declines to 0.32 by epoch 1,000 while training r climbs to 0.55–0.74 (Supplementary Fig. 8e): a generalization limit, not an optimization failure, with no release or schedule separating from the others beyond the run-to-run spread. The schedules differ in the reconstruction loss after the hand-over, which rises under pretrain-then-main and falls under linear-to-uniform (Supplementary Fig. 8e, top), not in the interaction accuracy. DANGO reports about $r = 0.47$ on this screen under a random split (its Fig. 2b, read from the bar) and a replicate ceiling of about 0.59 for the labels. The architecture, the six channels at v9.1, the 1% rule, the log-cosh loss, and the screen match the paper. Five things do not: the data are the knowledge-graph build of the screen, not DANGO's released files or folds; the evaluation is one seeded 80/10/10 split scored on validation, not five-fold cross-validation with a replicate-consistency filter; the vocabulary is the 6,607-gene reference rather than the 1,395 screened genes; pretraining and fine-tuning are one scheduled run rather than two stages with DANGO's optimizer settings; and the statistic is a maximum over epochs. The 0.05 gap to the published value is one the split and the selection rule alone could account for; the record supports no stronger statement in either direction.

Supplementary Table 8 DANGO replication in TorchCell on the Kuzmin 2018 trigenic interactions: best validation Pearson r (maximum over epochs of the logged validation Pearson, the checkpoint-selection rule) by STRING release and loss schedule, with the training Pearson r at that epoch. Mean ± SEM over n runs; a single run reports its value with no whisker. Epochs is the range of epochs logged across those runs. Every run used AdamW (learning rate 10^{-5}, weight decay 10^{-6}, batch 32), hidden width $d = 64$, and four attention heads, on the seed-split 72,841 / 9,105 / 9,104 records (train / validation / test). No test-split metric was logged for these runs.

STRING	**Loss schedule**	n	**Val. Pearson** r	**Train** r **at best**	**Epochs**
v9.1	pretrain then main	1	0.420	0.535	1000
v9.1	linear to uniform	1	0.422	0.520	574
v9.1	linear to flipped	2	0.422 ± 0.001	0.520 ± 0.019	500–539
v11.0	pretrain then main	1	0.419	0.540	987
v11.0	linear to uniform	2	0.424 ± 0.003	0.531 ± 0.002	442–467
v11.0	linear to flipped	2	0.422 ± 0.001	0.546 ± 0.003	500
v12.0	pretrain then main	4	0.423 ± 0.002	0.541 ± 0.005	500
v12.0	linear to uniform	5	0.420 ± 0.001	0.515 ± 0.011	500
v12.0	linear to flipped	1	0.422	0.544	500



Supplementary Figure 8 DANGO rebuilt in TorchCell and trained on the Kuzmin 2018 trigenic screen. a, Gene pairs per STRING channel in releases v9.1, v11.0, and v12.0: the solid part of a bar was already in the previous release, the pale part is new, and the hatched bar below zero is what the previous release lost. **b**, The model in the manuscript's notation, one box per stage. The six channels $A^{(k)}$ are the message-passing edges of one two-layer GraphSAGE encoder each over a shared lookup $\mathbf{E}$ ($N = 6607$, $d = 64$); the reconstruction head $\hat{A}^{(k)} = H^{(k)} W_k$ gives the pretraining loss $\mathcal{L}_{\text{rec}}$ of Eq. 26, in which λ_k weights the zero entries; the meta-embedding weights a_{ik} merge the channels into H; a strain selects rows of H with no perturbation operator T_ψ; the Hyper-SAGNN readout scores the triple from its static (s_t) and dynamic (d_t, two self-attention layers over p) embeddings under a log-cosh loss. The objective mixes the two losses with a weight α_e that follows one of three schedules with transition epoch 10: pretrain then main ($\alpha_e = 1$ for $e < 10$, then 0), linear to uniform ($1 \to 0.5$ by epoch 10, then 0.5), and linear to flipped ($1 \to 0$ by epoch 10, then 0). **c**, Percentage of decreased zeros per channel for v9.1 to v11.0 and v11.0 to v12.0 with the resulting λ_k printed over each bar; dashed line, the 1% rule; open circles, the two values the DANGO paper prints; the same rule on v11.0 to v12.0 gives the same assignment. **d**, Best validation Pearson r (maximum over epochs) by loss schedule and STRING release; bar, mean over runs; whisker, SEM where $n > 1$; circle, one run (19 runs). **e**, Training curves of the same runs; color, STRING release; line style, schedule. Top row, on a logarithmic epoch axis: the pretraining weight α_e of the three schedules, then the reconstruction and interaction losses on the validation split. Pretrain-then-main leaves the interaction loss at its initial value for ten epochs, and once α_e drops to 0 the reconstruction loss rises in every such run (more slowly under linear-to-flipped); linear-to-uniform keeps it falling. The reconstruction loss is lowest with v9.1 throughout, the release with the fewest edges and the only one trained with $\lambda_k = 0.1$ on three channels; which of the two sets the level is not measured. Bottom row, linear epoch axis: training and validation Pearson r and validation MSE; dashed line, the variance of the interaction label over all 91,050 records (SD = 0.054), the MSE of predicting the mean. Validation r reaches 0.4 within 50 epochs; training r at the selected epoch spans 0.49 to 0.55; validation MSE crosses the label variance at epochs 669 and 771 in the two runs that reach 1,000 epochs. Sources: `experiments/010-kuzmin-tmi/scripts/graph_statistics.py` (a), `experiments/005-kuzmin2018-tmi/scripts/dango_construction_si.py` **c**, and `dango_string_version_sweep.py` (d, e).

Supplementary Table 9 Every DANGO run on the full trigenic dataset, and the three CGT replicates of Fig. 2d. Best is the maximum over epochs of validation Pearson r (an upward-biased order statistic; the epoch it occurs at follows in parentheses). At min MSE is the validation Pearson r at the epoch of minimum validation MSE, the checkpoint the trainer keeps; it is within 0.0023 of Best for every DANGO run. $r \geq 0.35$ is the first epoch at which the run reaches that validation Pearson. Epochs is the number of validation epochs logged; a 4-GPU job on a Delta A40 node stops at the 48 h queue limit, and the 2-GPU jobs on the IGB cluster ran to 1,000 epochs. Every DANGO run used hidden width 64, four heads, the linear-to-uniform loss schedule (transition at epoch 10), AdamW at learning rate 10^{-5}, and a per-GPU batch of 64, on the experiment-006 build; the CGT replicates trained on the experiment-010 build. Runs marked • are the three DANGO values averaged in Fig. 2d.

Model	STRING	Run	GPUs	Epochs	Wall (h)	Best r (epoch)	At min MSE	$r \geq 0.35$	
DANGO	v9.1	`6q08iign`	4	448	47.9	0.3664 (121)	0.366 (110)	23	•
DANGO	v9.1	`x3savllr`	4	446	47.9	0.3671 (109)	0.367 (110)	22	•
DANGO	v9.1	`014mprap`	2	1000	59.5	0.3676 (98)	0.367 (81)	16	•
DANGO	v9.1	`cbi441o8`	4	650	47.9	0.3658 (109)	0.366 (112)	21	
DANGO	v9.1	`p6urax0a`	4	655	47.9	0.3652 (140)	0.364 (129)	20	
DANGO	v11.0	`ok8k5e6z`	4	262	47.8	0.3536 (78)	0.354 (67)	29	
DANGO	v11.0	`mj7paqg8`	4	259	47.9	0.3544 (76)	0.354 (67)	27	
DANGO	v11.0	`jo24cfpf`	4	311	47.9	0.3555 (82)	0.355 (90)	30	
DANGO	v11.0	`3f9sh91s`	4	305	47.8	0.3592 (304)	0.359 (304)	27	
DANGO	v12.0	`vqnrz21e`	4	398	47.9	0.3582 (336)	0.357 (326)	25	
DANGO	v12.0	`5akd0qwt`	4	446	47.9	0.3602 (432)	0.359 (398)	25	
DANGO	v12.0	`9jpfy547`	2	1000	65.2	0.3545 (60)	0.354 (53)	28	
DANGO	v12.0	`xxtxdu3y`	4	634	47.9	0.3597 (450)	0.359 (433)	30	
DANGO	v12.0	`jww4tblb`	4	648	47.9	0.3577 (608)	0.355 (90)	28	
CGT	v12.0	`lzs9pcj3`	4	64	62.3	0.4520 (24)	0.449 (21)	2	
CGT	v12.0	`yv4r30bi`	4	64	62.6	0.4472 (25)	0.444 (20)	2	
CGT	v12.0	`c7671wgj`	4	49	47.8	0.4619 (24)	0.462 (24)	2	

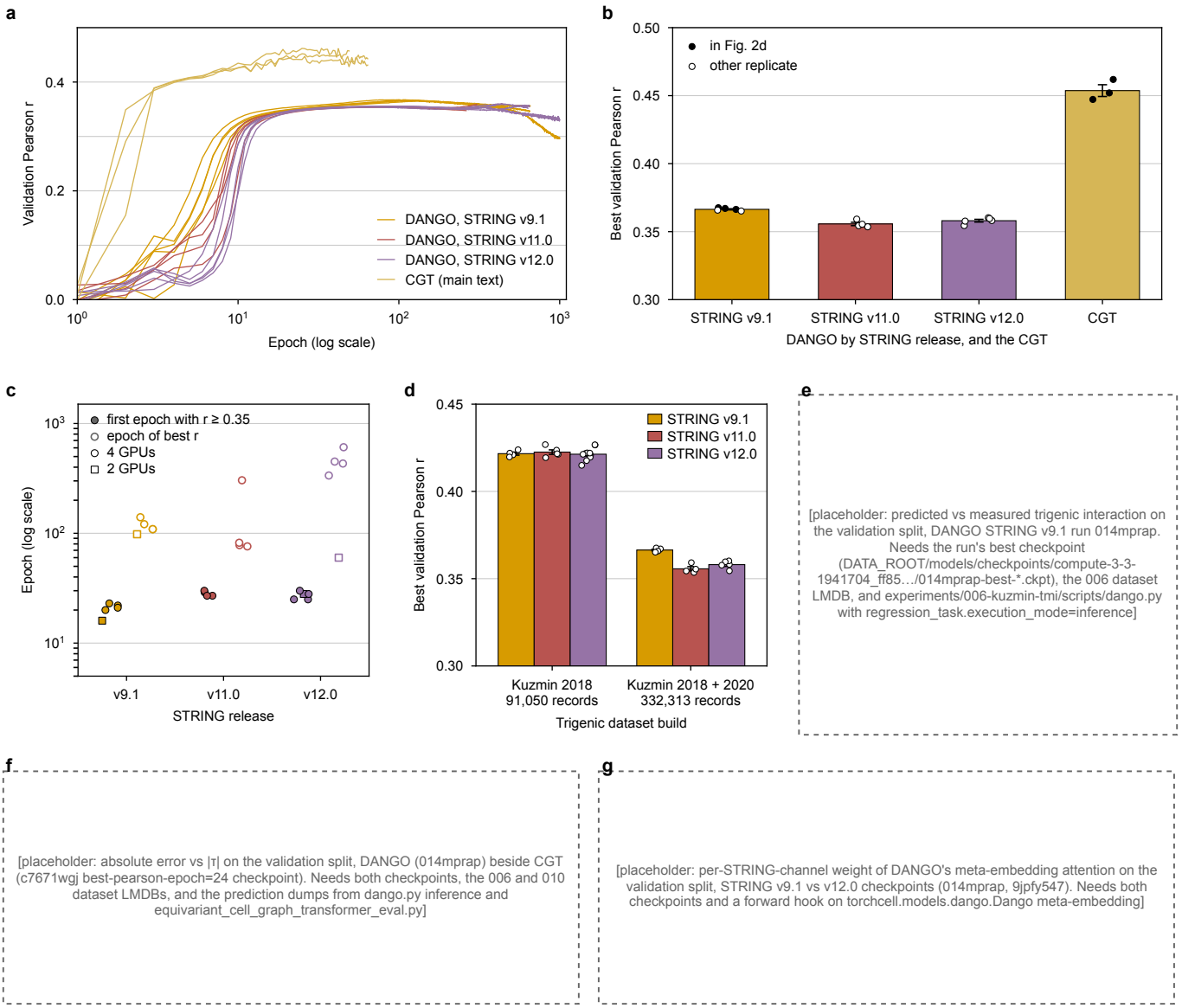
Supplementary Note 9: DANGO on the full trigenic dataset of the CGT experiment ✗

The DANGO value in Fig. 2d, Pearson $r = 0.367$, is the mean of the three highest of five DANGO runs trained with STRING v9.1 graphs on the trigenic dataset of the Cell Graph Transformer (CGT) experiment; Supplementary Table 9 lists every run, including nine with STRING v11.0 and v12.0 graphs, and Supplementary Fig. 9 their training curves. Two facts qualify the comparison: the DANGO and CGT runs were trained on two different builds of the trigenic dataset, and every DANGO run on this dataset plateaus at $r = 0.354$ to 0.368, well below the 0.42 the same implementation reaches on the Kuzmin 2018 screen alone (Supplementary Note 8).

Dataset builds. The DANGO and DCell baselines of Fig. 2d were trained on the experiment-006 build: the Kuzmin 2018 trigenic interactions [4] with any perturbation type plus the Kuzmin 2020 trigenic interactions restricted to deletions, 332,313 records. The CGT replicates were trained on the experiment-010 build, every perturbation type from both screens, 376,732 records. Both are split into training, validation, and test (nominally 80/10/10) by the same seeded splitter, stratified on phenotype label and perturbation count, so the two validation splits differ in membership. No DANGO or DCell run on the experiment-010 build exists; the like-for-like baseline requires that run.

Training and result. DANGO is the implementation of Supplementary Note 8, trained as Supplementary Table 9 records; the split seed is fixed and the initialization is not, so runs of one configuration are replicates over initialization and DDP nondeterminism. Read from the code, not measured in training: the v9.1-keyed weight lookup of Supplementary Note 8 is the same script here, so the v11.0 and v12.0 runs trained with $\lambda_k = 1$ for every channel. Scored as the best validation Pearson r (maximum over epochs), the five v9.1 runs reach 0.3664 ± 0.0004 (mean ± SEM), the four v11.0 runs 0.3557 ± 0.0012, and the five v12.0 runs 0.3581 ± 0.0010 (Supplementary Fig. 9b). The three runs behind Fig. 2d are the v9.1 runs `014mprap`, `x3savllr`, and `6q08iign`; the two launched later reach 0.3658 and 0.3652, so the mean over all five sits 0.0006 below the reported 0.3670. The three CGT replicates reach 0.4537 ± 0.0043 at epochs 24 to 25.

Release and dataset effects. Every DANGO run reaches $r \geq 0.35$ within 16 to 23 epochs with v9.1 and 25 to 30 with v11.0 or v12.0 (Supplementary Fig. 9a, c); what the release changes is the plateau, a later and lower maximum rather than a slower initial rise. Against the Kuzmin 2018 screen alone (Supplementary Fig. 9d; Supplementary Table 8), the only measured data-size comparison for DANGO, the two builds are split separately, so no like-for-like split can be claimed; dataset size, split membership, and the addition of the Kuzmin 2020 screen change at once, and the drop from 0.42 to 0.36 is not attributed to any one of them here. Hypothesis (untested): the Kuzmin 2020 triples, screened around duplicated gene pairs, are the harder part of the union, given that the CGT scores 0.396 on the deletion-only experiment-009 build and 0.454 on the experiment-010 build. Whether DANGO trained on the experiment-010 build would close any of the gap to the CGT is likewise not measured.



Supplementary Figure 9 DANGO on the full trigenic dataset. a, Validation Pearson r against epoch for every DANGO run on the experiment-006 build, colored by STRING release, and the three CGT replicates of Fig. 2d on the experiment-010 build; the epoch axis is logarithmic. With v9.1 the maximum comes at epoch 98 to 140 and the curve then declines (the 1,000-epoch run ends at 0.297); with v12.0 the four-GPU runs creep from 0.354 to 0.355 at epoch 100 to their maxima at epochs 336 to 608, a gain of 0.002 to 0.006, while the two-GPU v12.0 run peaks at epoch 60 and declines. **b**, Best validation Pearson r (maximum over epochs) per run; bar and whisker are the mean and SEM over runs; filled markers are the runs averaged in Fig. 2d; the axis is zoomed to 0.30 to 0.50. **c**, For each DANGO run, the first epoch at which validation Pearson reaches 0.35 (filled) and the epoch of its maximum (open); circles are four-GPU jobs, squares two-GPU jobs. The release moves the epoch of the maximum (98 to 140 with v9.1, up to 608 with v12.0), not the epoch of the rise (16 to 30 in every run). **d**, Best validation Pearson r (maximum over epochs) of the same implementation on the Kuzmin 2018-only build of Supplementary Note 8 (Kuzmin 2018 only, 91,050 records, 19 runs pooled over the three loss schedules) and on the experiment-006 build of this Note (Kuzmin 2018 plus the Kuzmin 2020 deletions, 332,313 records, 14 runs), by STRING release; bar, mean over runs; whisker, SEM; open circles, runs; the axis is zoomed to 0.30 to 0.45. The two builds are split separately, so the validation sets differ in membership as well as in size, and the 005 runs used batch 32 on one GPU where the 006 runs used batch 64 per GPU on two to four; this is the only measured data-size comparison for DANGO, not a controlled ablation. **e**–**g**Reserved: predicted against measured interaction, absolute error against $|\tau|$, and per-channel meta-embedding weights need a trained checkpoint and inference on the validation split; each box states the checkpoint and script that fill it. Source: `experiments/010-kuzmin-tmi/scripts/dango_full_dataset_si.py` (d also reads `experiments/005-kuzmin2018-tmi/results/dango_string_version_sweep.csv`).

Supplementary Note 10: DCell in TorchCell: the ontology-structured model in the shared notation ✗

DCell (Ma et al. 2018) is the visible-neural-network baseline of Fig. 2d: hidden units grouped into subsystems, one per Gene Ontology (GO; Ashburner et al. 2000) term, wired to each other as the terms are wired in the ontology. It was reimplemented in TorchCell from the published description rather than run from the released Torch7 code, so that it reads the same experiment records and 6,607-gene S288C reference as every other model in the comparison; extending the published single- and double-deletion model to trigenic interaction changes only which genes are zeroed. Supplementary Fig. 10 draws the adaptation and Supplementary Table 10 the ontology it is built on; its training cost is the subject of Supplementary Note 11.

The graph is the ontology. Where the cell encoder F_θ takes the gene–gene relations $\{A^{(k)}\}$ only as soft priors on attention (Eq. 16), DCell's graph is the GO directed acyclic graph $G_{\text{GO}} = (T, E_{\text{GO}})$, edges from each term to its parents under one synthetic super-root `GO:ROOT`, with the annotation relation genes$(t) \subseteq [N]$ of genes annotated directly to t; each hierarchy edge is hard wiring, a block of a weight matrix, not a regularizer. The perturbation enters as data: a strain with deleted set p is the gene-state vector $s \in \{0,1\}^N$, $s_i = 0$ if $g_i \in p$, and the loader copies the reference's table of 59,986 (term, gene) rows per strain and zeros the rows of deleted genes. DCell is therefore the rebuild route of Supplementary Note 1, with no shared encoding H and no operator $\mathcal{T}_\psi$ (the $\mathcal{O}(B\,L|E|d)$ column of Supplementary Table 1); a trigenic strain zeros three rows, and that is the entire trigenic extension. Every term t owns a subsystem that reads the outputs of its child subsystems ch(t) and the states of its own genes,

$$I_t = \big[\, \|_{c\in\text{ch}(t)}\, O_c \;\|\; s_{\text{genes}(t)} \,\big], \qquad O_t = \tanh\big(\text{BN}(W_t I_t + b_t)\big) \in [-1,1]^{L_t}, \qquad L_t = \max\big(20, \lceil 0.3\,|\text{genes}(t)|\rceil\big), \tag{27}$$

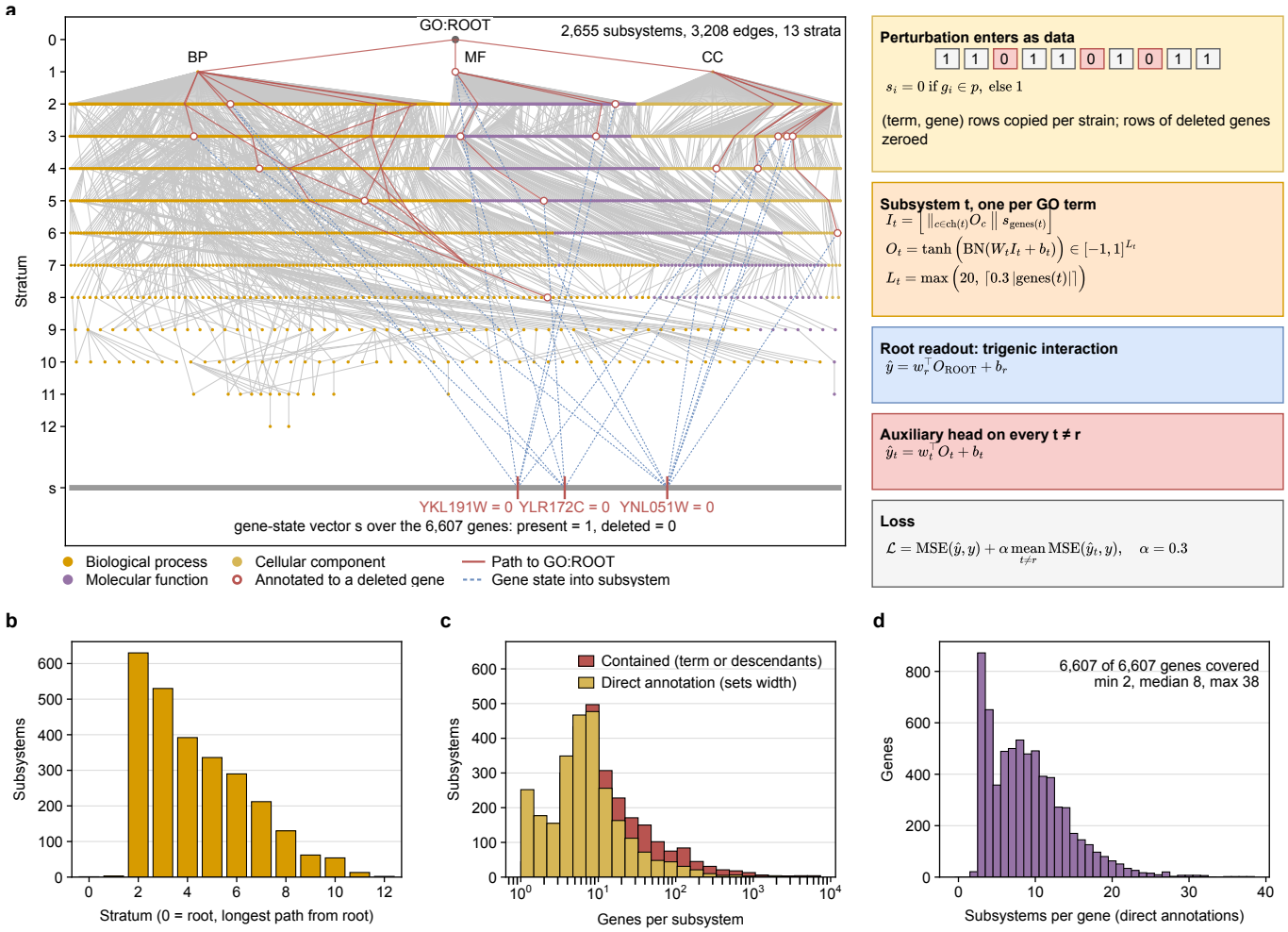
with BN batch normalization. Two details differ from the published BatchNorm(Tanh(Linear(I_t))): normalization precedes the tanh, and |genes(t)| counts direct annotations rather than the genes contained by t and its descendants. The root readout $\hat{y} = w_r^\top O_{\text{ROOT}} + b_r$ predicts the trigenic interaction y_{int} directly, with no fitness intermediate (contrast Supplementary Note 4); every other subsystem carries a head $\hat{y}_t = w_t^\top O_t + b_t$, and the loss

$$\mathcal{L} = \text{MSE}(\hat{y}, y) + \alpha \operatorname*{mean}_{t\neq r} \text{MSE}(\hat{y}_t, y), \qquad \alpha = 0.3, \tag{28}$$

averages the auxiliary terms where the original summed them; AdamW weight decay replaces the published $\lambda\|W\|_2$ penalty.

Building and measuring the ontology. The published subsystems are the GO terms left after three filters: terms with evidence code IGI (inferred from genetic interaction, a circularity), terms containing fewer than six of the training genes, and terms redundant with their children. TorchCell rebuilds that recipe on GO release 2025-03-16 with SGD annotations (Supplementary Table 10): IGI-evidence annotations are removed (1,921) and a term goes only when that empties it; a term whose gene set equals a parent's is removed; a term containing fewer than four of the 6,607 reference genes is removed, the threshold chosen so that the retained 2,655 subsystems approximate the published 2,526; and no date cutoff is applied, so the ontology carries annotations that postdate the screens it is trained on (a 2017-07-19 cutoff is implemented and would leave 1,408 subsystems, but was not used). The result has 2,655 subsystems on 3,208 hierarchy edges in 13 strata with 59,986 annotations covering all 6,607 genes (Supplementary Fig. 10b–d). Coverage is complete for a specific reason: 5,487 annotations with evidence code ND (no biological data) attach genes of unknown function directly to the three namespace roots, so every gene sits in at least two subsystems (median 8, maximum 38) and the roots are the widest subsystems (molecular function: 2,444 direct genes, $L_t = 734$). The network has 61,429 hidden units and 20,613,037 parameters, which equals the `model/params_total` logged by every DCell training run in both wandb projects (15 runs, one value), so the DAG measured here is the one the baseline was trained on.

Against the published ontology. Supplementary Table 11 sets the rebuild beside every quantity the paper states about its ontology and model, each quoted from the paper's text; the two are not the same graph. The paper reports 2,526 subsystems in "12 layers" with 97,181 neurons "ranging from 20 to 1,075 per system"; the rebuild has 2,655 subsystems in 13 strata with 61,429 hidden units from 20 to 734. The paper reports neither hierarchy edges, nor leaves, nor a parameter count, so the 3,208 edges, 1,527 leaves, and 20,613,037 parameters here have no published counterpart. Four causes are identifiable, one of them measured. The GO release differs: the paper names none (it cites the 2016 Consortium paper) and the rebuild uses the 2025-03-16 release, whose retained annotations are dated up to 2023-10-04; under the same three filters, keeping only annotations dated on or before 2017-07-19 leaves 1,408 subsystems rather than 2,655 (Supplementary Table 10), so release drift alone moves the count by more than the gap to 2,526. The containment filter differs in threshold and in population: the paper drops terms "containing fewer than six yeast genes disrupted in the available genotypes", the rebuild terms containing fewer than four of all 6,607 reference genes, a threshold chosen to land near 2,526 on the newer release. The filter semantics and order differ: IGI removal acts on annotations rather than terms, redundancy is judged against a parent rather than the children, and the rebuild applies



Supplementary Figure 10 DCell as implemented in TorchCell. a, The filtered GO DAG that structures the model, drawn whole: 2,655 subsystems as points in 13 strata (stratum 0 is `GO:ROOT`, stratum 1 the three namespace roots BP, MF, CC; the deepest terms at the bottom), colored by namespace, with the 3,208 hierarchy edges (`is_child_of`, child to parent) in gray; within a stratum, terms are ordered by the mean position of their parents. The perturbation enters as data: the bottom row is the gene-state vector s over the 6,607 genes, present = 1, and a strain with deleted set p has $s_i = 0$ for $g_i \in p$; the loader copies the reference's table of 59,986 (term, gene) annotation rows per strain and zeros the rows of the deleted genes, so each strain is a rebuilt input rather than an operator on a shared encoding. One measured strain of the Kuzmin 2018 screen is drawn, the triple deletion YKL191W, YLR172C, YNL051W (record 4 of the 91,050-record Kuzmin 2018-only build, chosen by rule: the first record whose three perturbations are all deletions and whose genes each carry 5 to 12 direct annotations; here 5, 5 and 9): dashed blue, the zeroed gene states entering the subsystems that annotate them (open circles); solid red, every hierarchy edge from those subsystems up to the root, the subsystems the zeroed rows reach. Subsystems run stratum by stratum from the deepest term to the root. Right: the gene-state rule, the subsystem of Eq. 27, the root readout, the per-term auxiliary heads, and the loss of Eq. 28; weights are initialized uniformly in $[-0.001, 0.001]$ as published. **b**, Subsystems per stratum of the filtered DAG; 1,527 of the 2,655 subsystems are leaves. **c**, Genes per subsystem: direct annotations, which set the width L_t, and genes contained by the term or its descendants, the quantity the published model sized subsystems by; the median subsystem has 6 direct and 10 contained genes. **d**, Number of subsystems each of the 6,607 genes is annotated to; 675 genes are in the hierarchy through ND-evidence annotations to the namespace roots alone. The DAG in a and panels b–d are measured by `experiments/006-kuzmin-tmi/scripts/dcell_model_go_stats.py`; the figure is composed by `dcell_model_compose_figure.py`.

redundancy before containment where the paper lists containment second. The width rule counts direct annotations rather than contained genes, which is why the rebuild has more subsystems but fewer hidden units: 2,521 of the 2,655 subsystems sit at the floor of 20, and `GO:ROOT`, which carries no direct annotation, is 20 wide, whereas the published rule makes the widest subsystem 1,075 neurons, a term containing 3,581 to 3,583 genes by that rule. Whether the published "12 layers" counts the root is not stated. A term-by-term comparison would need the published term list, which the paper does not print; it was not made.

Stage	Terms	Edges	Annotations	Genes covered	Leaves
SGD annotations on GO, three namespaces under `GO:ROOT`	5,660	6,674	66,404	6,607	3,826
drop IGI-evidence annotations (empty terms removed)	5,541	6,540	64,483	6,607	3,763
drop terms whose gene set equals a parent's	5,435	6,433	64,324	6,607	3,763
drop terms containing < 4 genes (the trigenic run)	2,655	3,208	59,986	6,607	1,527
reference only: annotations dated $\leq$ 2017-07-19, then the same three filters	1,408	1,714	22,978	6,409	911

Supplementary Table 10 The GO DAG at each filtering stage of the DCell baseline. Filters are applied in the order listed (`torchcell.graph.filter_go_IGI`, `filter_redundant_terms`, `filter_by_contained_genes`), on GO release 2025-03-16 with SGD gene annotations; a removed term's children are reconnected to its parents. Annotations are direct (term, gene) pairs over the 6,607-gene reference; genes covered are those with at least one such pair; leaves are terms with no child term. The last row is not part of the trigenic run.

	Published DCell	**TorchCell DCell**
GO release	not reported; cites the GO Consortium (ref. 9, *Nucleic Acids Res.* 2016)	2025-03-16 (`go.obo` data-version)
Annotation source	not reported ("gene-to-term annotations")	SGD `go_details` over the 6,607-gene S288C reference
Genes	the genes disrupted in the training genotypes; count not reported	6,607 reference genes, all covered
Evidence filter	terms with evidence code IGI removed	IGI annotations removed (1,921); a term goes only when emptied (119 terms)
Redundancy filter	terms redundant with respect to their children	terms whose gene set equals a parent's (106 terms)
Containment threshold	fewer than six disrupted genes, counted over the term and its descendants	fewer than four of the 6,607 reference genes, counted over the term and its descendants (2,780 terms)
Filter order	IGI, containment, redundancy (as listed)	IGI, redundancy, containment
Removed-term rewiring	children connected to all parents	children connected to all parents
Subsystems	2,526	2,655
Hierarchy edges	not reported	3,208
Depth	12 layers	13 strata (`GO:ROOT` plus 12)
Leaves	not reported	1,527
Width rule	$\max(20, \lceil 0.3 \times$ genes contained by $t \rceil)$	$\max(20, \lceil 0.3\,\lvert\text{genes}(t)\rvert \rceil)$, direct annotations
Root readout	one output neuron; root width not reported	one output; `GO:ROOT` has 0 direct genes, so $L_r = 20$
Hidden units	97,181 (20 to 1,075 per subsystem)	61,429 (20 to 734 per subsystem; 2,521 subsystems at the floor of 20)
Parameters	not reported	20,613,037
Auxiliary loss	$\alpha = 0.3$, summed over $t \neq r$, plus $\lambda \lVert W \rVert_2$ (λ by four-fold CV)	$\alpha = 0.3$, averaged over $t \neq r$; AdamW weight decay
Training data	Costanzo 2010 ($\sim$3 M examples) or Costanzo 2016 ($\sim$8 M), single and double deletions	the trigenic dataset of the CGT experiment (Supplementary Note 11)

Supplementary Table 11 The published DCell ontology and model (Ma et al. 2018) against the TorchCell rebuild. Published values are quoted from the paper's text and Online Methods (the OCR text of the mirrored paper, sha256-pinned in the generating script); "not reported" marks a quantity the paper does not state. TorchCell values are measured on the frozen DAG of Supplementary Table 10; counts in parentheses are the terms or annotations each filter removed. Depth counts strata of the longest path from `GO:ROOT`; the paper's "12 layers" does not say whether the root is counted. Hidden units are the sum of subsystem widths L_t.

Supplementary Table 12 The DCell value in Fig. 2d and the run it comes from. All values are the validation Pearson r logged by the single DCell training run (wandb `eni948by`, SLURM job 1922684, 333 epochs, 30.8 days on 4 GPUs). *Rank* is the value's rank among the 333 per-epoch validation evaluations of that run. The three epochs are evaluations of one run, so their spread is checkpoint-to-checkpoint variation, not replicate variance, and the reported SEM is not a replicate SEM.

Evaluation	Epoch	Wall-clock (d)	Val. Pearson r	Rank
Fig. 2d checkpoint	144	13.4	0.155	2
Fig. 2d checkpoint	148	13.8	0.142	4
Fig. 2d checkpoint	150	14.0	0.173	1
Mean of the three (Fig. 2d)			0.157	
SD over the three (ddof=1)			0.016	
SEM = SD/$\sqrt{3}$ (Fig. 2d error bar)			0.0091	
Maximum over all epochs	150	14.0	0.173	1
Minimum validation MSE (the saved checkpoint)	142	13.2	0.127	20
Final epoch	332	30.8	0.038	231

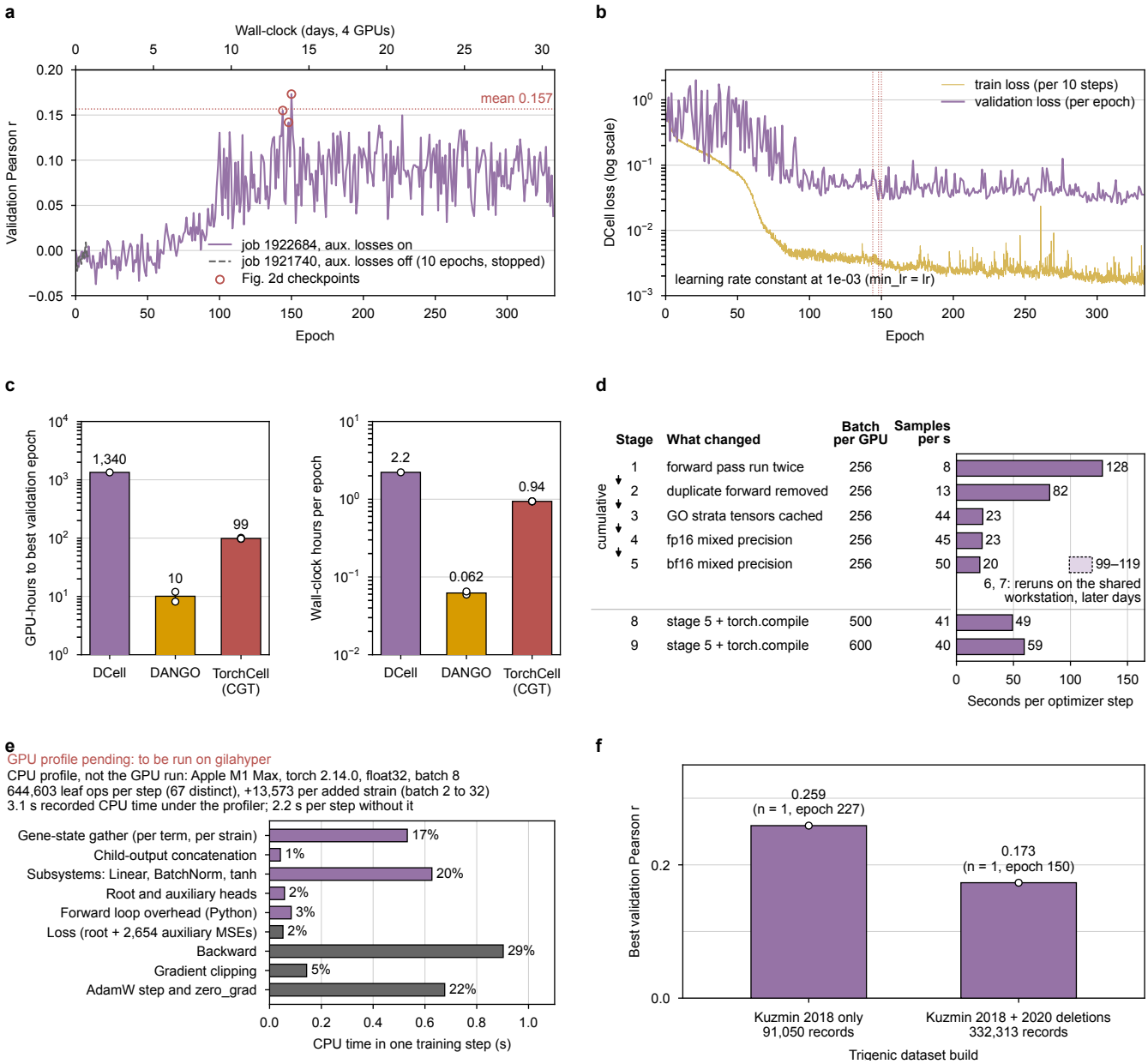
Supplementary Note 11: DCell training cost, instability, and the checkpoint-based baseline ✗

The DCell value in Fig. 2d ($r = 0.157 \pm 0.009$) comes from one training run, not from replicates: the three values behind the mean and error bar are validation evaluations of that run at epochs 144, 148, and 150, so the error bar is not a replicate error bar. DCell in TorchCell (Supplementary Note 10) took 30.8 days on four GPUs to reach epoch 333, passed its best validation epoch after 14 days, was unstable throughout, and was run to completion once within the compute available.

Training run and checkpoint statistic. One full run (wandb `eni948by`, SLURM job 1922684) trained the 2,655-term, 20.6 M-parameter DCell with the DCell loss (auxiliary losses on, $\alpha = 0.3$), AdamW at learning rate 10^{-3} and weight decay 10^{-6}, global batch 2,400 over four GPUs under DDP, bf16 mixed precision, and gradient clipping at 10, on the experiment-006 build (Supplementary Note 9; Supplementary Table 13). The reduce-on-plateau floor equaled the initial rate, so the learning rate never changed (Supplementary Fig. 11b), and the run was stopped after 333 of the configured 1,000 epochs. The three values in Fig. 2d, 0.155, 0.142, and 0.173, include the maximum over all 333 per-epoch evaluations (Supplementary Table 12), so their mean (0.157) is an upward-biased order statistic rather than an expected performance, and their SD (0.016; SEM 0.009) measures epoch-to-epoch variation of one run, not replicate variance. The checkpoint the training script saved, the minimum validation MSE at epoch 142, scores $r = 0.127$; the final epoch scores 0.038. After epoch 100, validation r fluctuated between 0.04 and 0.17 and drifted down while the training loss kept falling by an order of magnitude (Supplementary Fig. 11a, b), overfitting under a constant learning rate; whether a decaying schedule would have stabilized the run is untested.

As for DANGO (Supplementary Note 9), the run on the larger build scored lower: the one DCell run on the Kuzmin 2018-only build peaked at $r = 0.259$ against 0.173 on the experiment-006 build (Supplementary Fig. 11f; one run each, differently split, with auxiliary losses and precision also differing, so the gap is not attributed to data size).

Cost and where the time goes. DCell reached its best validation epoch after 1,340 GPU-hours, against 97–101 GPU-hours for each CGT replicate (best at epoch 24–25) and 8–12 for the two DANGO runs on the same build (Supplementary Table 13; Supplementary Fig. 11c): 33 training samples per second (2.22 h per epoch) against 89 for the CGT and about 1,200 for DANGO, and six times as many epochs as the CGT to peak. Three replicate runs of the same length would cost about 8,900 GPU-hours at the observed rate, which is why one run is reported and labeled as such. The forward pass visits the 2,655 subsystems one at a time in a Python loop ordered by stratum, so every step launches hundreds of thousands of small kernels, more as the batch grows; the speed-up work on a shared four-GPU workstation (Supplementary Fig. 11d) took a step from 128 s to 20 s, and day-to-day reruns of one configuration varied by as much as several of the optimizations. A CPU profile of one step of the same architecture (Supplementary Fig. 11e) counts 644,603 kernel-level ops at batch 8, about 13,600 more per added strain, because the gene-state gather runs once per term and per strain; no GPU profile exists. Hypothesis (untested): the GPU step is bound by those launches rather than by data loading, which the cluster measurements cannot separate; if so, one grouped matrix multiply per stratum and CUDA graphs for the fixed per-stratum shapes would remove most of the launch overhead.



Supplementary Figure 11 DCell training on the trigenic interaction task: one run, its checkpoint statistic, and its cost. **a**, Validation Pearson r per epoch for the single full DCell run (wandb `eni948by`, job 1922684, four GPUs; top axis, wall-clock) and the stopped run without auxiliary losses (job 1921740, 10 epochs; partial). Open circles mark the three evaluations averaged in Fig. 2d; the dotted line is their mean. **b**, Training loss (every 10 steps) and validation loss (per epoch) of the same run; dotted verticals mark the three evaluations; the learning rate was constant. **c**, GPU-hours to the best validation epoch (left) and wall-clock hours per epoch (right, median over epochs) for DCell (four GPUs), DANGO (two runs, two GPUs each) and CGT (three replicates, four GPUs each); bar, mean; open circles, runs; whisker, SEM where $n > 1$ (Supplementary Table 13). **d**, Speed-up stages on a shared four-GPU workstation: what each stage changed, batch per GPU, samples per second, and seconds per optimizer step as bars. Arrows mark the cumulative chain 1 to 5, measured in one sitting; stages 8 and 9 add `torch.compile` at batch 500 and 600; rows 6 and 7 (stage 5 rerun on later days, 99 and 119 s) are the dashed range, because the shared machine drifts from day to day by more than several optimizations and only same-sitting stages compare. **e**, Where one training step spends its time on a CPU (Apple M1 Max, fp32, batch 8), a stand-in until the pending GPU profile on gilahyper replaces it. The step issues 644,603 leaf ops (67 distinct), each a kernel launch on a GPU, growing by about 13,600 per added strain; the per-term gene-state gather and 2,655 BatchNorm calls cost more than the matrix multiplies. **f**, Best validation Pearson r (maximum over epochs) of the same DCell on the Kuzmin 2018-only build (91,050 records; `biucpv7p`, best at epoch 227) and the experiment-006 build (332,313; `eni948by`, best at epoch 150), one run each, splits and training settings differing; the only measured data-size comparison for DCell.

Supplementary Table 13 Training cost of the three learned models compared in Fig. 2d, from the wandb histories of the runs listed in `results/dcell_training/runs.csv`. Samples per epoch is optimizer steps per epoch times the global batch. *Best* is the epoch of the maximum validation Pearson r over the run; the wall-clock and GPU-hours to reach it are the logged run time at that epoch. CGT rows are the three replicate training jobs whose best-Pearson checkpoints give the Fig. 2d value; DANGO rows are the two 1,000-epoch training runs on the same build as DCell. DCell and DANGO trained on the experiment-006 build (332,313 records); the CGT replicates trained on the experiment-010 build (376,732 records).

Model	Run	Build	GPUs	Samples/epoch	h/epoch	Samples/s	Best epoch	Best r	h to best	GPU-h to best
DCell	`eni948by`	006	4	266,393	2.22	33	150	0.173	334.9	1,340
CGT	`lzs9pcj3`	010	4	302,064	0.94	90	24	0.452	24.3	97
CGT	`yv4r30bi`	010	4	302,064	0.94	89	25	0.447	25.3	101
CGT	`c7671wgj`	010	4	302,059	0.94	89	24	0.462	24.5	98
DANGO	`014mprap`	006	2	265,856	0.06	1,244	98	0.368	6.0	12
DANGO	`9jpfy547`	006	2	265,856	0.07	1,134	60	0.354	4.1	8

Supplementary Note 12: The Yeast9 flux-balance baseline for trigenic interactions ✗

The Yeast9 value in Fig. 2d is a mechanistic baseline with no fitted parameter: each deletion set of the trigenic screens is imposed on the yeast-GEM 9.0.2 metabolic reconstruction, growth is computed by flux-balance analysis (FBA), and the interaction of Fig. 2a is formed from the predicted growth rates exactly as it is from the measured colony sizes (Supplementary Fig. 12a). The run was made once, on 2025-09-15, and every number here is recomputed from its frozen output files and the screens' raw tables.

Model and medium. The model is yeast-GEM 9.0.2 as distributed (`yeast-GEM.xml`: 4,131 reactions, 2,806 metabolites, 1,161 genes, 271 exchange reactions), loaded through `torchcell.metabolism.yeast_GEM.YeastGEM`. The medium is the model's default, `model.medium`, which the run never modified: 16 exchange reactions open for uptake, glucose at 1 mmol $\mathrm{gDW}^{-1}\,\mathrm{h}^{-1}$ as the sole carbon source, ammonium as the sole nitrogen source, oxygen and the inorganic ions unbounded, and no amino acid, vitamin, or nucleobase (Supplementary Table 14). The objective is the growth pseudoreaction `r_2111`, non-growth-associated maintenance is fixed at 0.7 mmol ATP $\mathrm{gDW}^{-1}\,\mathrm{h}^{-1}$ (`r_4046`), and the wild-type optimum is $\mu_{\mathrm{WT}} = 0.0858\ \mathrm{h}^{-1}$, which reloading the same SBML file reproduces to 5×10^{-8}. Each deletion set was solved as one linear program with GLPK (cobrapy's default solver, through optlang) under a 60 s limit, in 128 worker processes; the 4,036 singles, 651,181 doubles, and 332,313 triples took 10 min in all, every solve returned `optimal`, and none hit the limit. The cobrapy version at run time was not recorded.

Deletion simulation. A deletion set p sets the state $s_g = 0$ for each of its genes and leaves every other gene at 1. cobrapy's `Gene.knock_out` then evaluates each reaction's gene-reaction rule, a Boolean expression in which isozymes are joined by OR and complex subunits by AND, and sets the flux bounds of every reaction whose rule evaluates false to zero; single, double, and triple deletions differ only in how many genes are set to zero before the rules are evaluated. A gene absent from the model changes no rule, so its deletion leaves growth at the wild-type value. The deletion sets are the 332,313 triples of the dataset build, the trigenic records of Kuzmin 2018 (91,050) and Kuzmin 2020 (241,746; 483 in both), and all of their sub-collections (651,181 doubles, 4,036 singles). Yeast9 covers 736 of the 4,036 genes, 18,565 of the doubles, and 7,419 triples (2.2%) in full; 220,392 triples (66.3%) contain no model gene (Supplementary Fig. 12e).

From growth to interaction. Fitness is the growth ratio $f = \mu/\mu_{\mathrm{WT}}$, with a non-optimal or timed-out solve counted as $f = 0$. Digenic and trigenic interactions are then the definitions of Fig. 2a, $\varepsilon_{ij} = f_{ij} - f_i f_j$ and $\tau_{ijk} = f_{ijk} - f_i f_j f_k - \varepsilon_{ij} f_k - \varepsilon_{ik} f_j - \varepsilon_{jk} f_i$, computed from the predicted fitness of the triple and of its singles and doubles. The predicted τ_{ijk} of each triple is compared by Pearson correlation with the measured τ_{ijk} of the same gene set, taken from the screens' raw tables as the mean adjusted score over the gene set's records (the dataset's deduplication rule); no split, threshold, or fit enters.

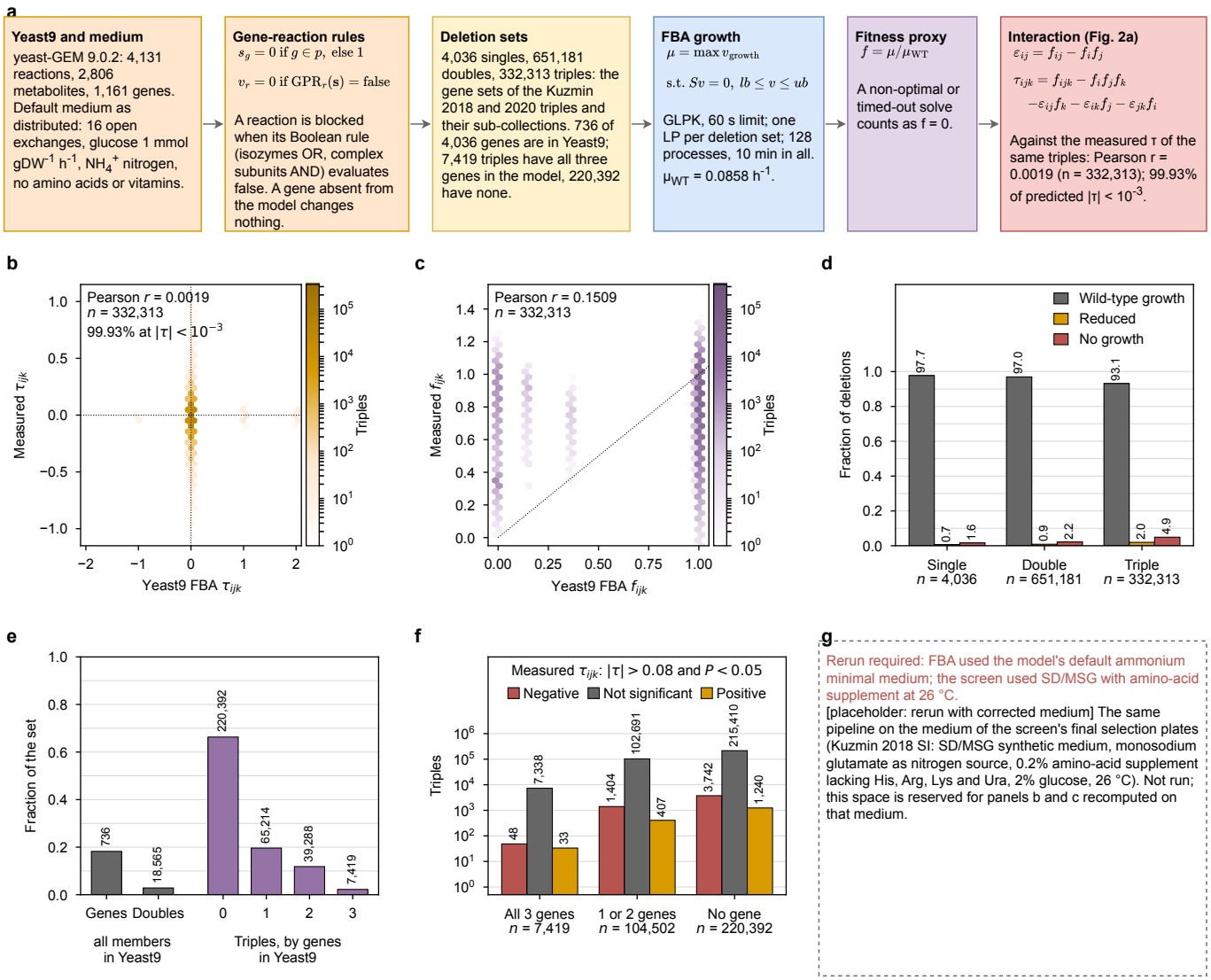
Result. Predicted against measured τ_{ijk} gives $r = 0.0019$ ($n = 332{,}313$, $P = 0.28$; Spearman $\rho = 0.003$; Supplementary Fig. 12b), and predicted against measured triple-mutant fitness $r = 0.151$ ($n = 332{,}313$; 0.254 within the 111,921 triples that contain a model gene; Supplementary Fig. 12c). The value printed in Fig. 2d, 0.0006, was transcribed from the pipeline's output at the time and does not reproduce; whether it came from a different subset or a different run is not recorded, and the scripted value is the one to carry. The pipeline's own matched file cannot be used for either number: its `experimental` column is the true label set in the wrong row order (the sorted stored and raw values coincide for 99.98% of triples, row by row they agree for 1.2%), which left the interaction correlation unchanged, since the prediction is nearly constant, but reported the fitness correlation as −0.005. The interaction correlation is near zero because the prediction is nearly constant: 97.6% of the predicted τ_{ijk} lie within 10^{-6} of zero and 99.93% within 10^{-3}, leaving 231 triples with a nonzero prediction, almost all +1 (122), +2 (62), or −1 (19). The constancy has two measured sources. First, the deletions rarely change the optimum: 97.7% of single, 97.0% of double, and 93.1% of triple deletions grow at the wild-type rate to within 10^{-3}, and the rest fall almost entirely into no growth (1.6%, 2.2%, 4.9%) or one of a few discrete bands (predicted triple-mutant fitness takes 1, 0, and values near 0.15, 0.37, and 0.98–0.99; Supplementary Fig. 12c,d), so f_{ijk} almost always equals $f_i f_j f_k$ exactly. Second, two-thirds of the triples contain no model gene and so have $\tau_{ijk} = 0$ by construction. The nonzero predictions are mostly artifacts: 164 of the 231 arise in triples with only one gene in the model, where τ_{ijk} must be zero, and tracing them shows 253 of 632,616 checkable doubles and 52 of 285,606 checkable triples whose growth contradicts their own single-deletion result (246 of the doubles contain `YGR144W`, whose single deletion grows at the wild-type rate while these doubles return zero growth with status `optimal`), together with 173 triples that grow faster than the least of their three doubles, which FBA cannot produce. The cause of these solver failures was not determined. Within the 7,419 fully covered triples, 48 have a nonzero prediction, and the measured landscape there is as sparse as elsewhere: by the screens' convention ($|\tau| > 0.08$, $P < 0.05$) 48 are negative and 33 positive (1.1%), against 1,404 and 407 among the partly covered and 3,742 and 1,240 among the uncovered triples (2.1% overall; Supplementary Fig. 12f).

Caveats. The medium is the principal open question. The FBA ran on the model's default ammonium minimal medium, whereas the screen's final triple-mutant selection plates were, from the Kuzmin 2018 Supplementary Materials, "$\mathrm{SD}_{\mathrm{MSG}}$ - His/Arg/Lys/Ura + canavanine/thialysine/G418/clonNAT (0.17% yeast nitrogen base without amino acids

Supplementary Table 14 The medium of the Yeast9 flux-balance baseline: the exchange reactions open for uptake in yeast-GEM 9.0.2 as distributed (`model.medium`), which the run used unchanged. Every other exchange reaction is closed to uptake. Uptake bounds are in mmol gDW^{-1} h^{-1}; 1000 is the model's unbounded value. Glucose is the sole carbon source at 1 mmol gDW^{-1} h^{-1} and ammonium the sole nitrogen source; no amino acid, vitamin, or nucleobase is supplied. The objective is the growth pseudoreaction `r_2111` and non-growth-associated maintenance is fixed at 0.7 mmol ATP gDW^{-1} h^{-1} (`r_4046`). Wild-type growth under this medium is 0.0858 h^{-1}.

Reaction	Metabolite	Uptake bound
r_1654	ammonium	1000
r_1714	D-glucose	1
r_1832	H+	1000
r_1861	iron(2+)	1000
r_1992	oxygen	1000
r_2005	phosphate	1000
r_2020	potassium	1000
r_2049	sodium	1000
r_2060	sulphate	1000
r_2100	H2O	1000
r_4593	chloride	1000
r_4594	Cu2(+)	1000
r_4595	Mn(2+)	1000
r_4596	Zn(2+)	1000
r_4597	Mg(2+)	1000
r_4600	Ca(2+)	1000

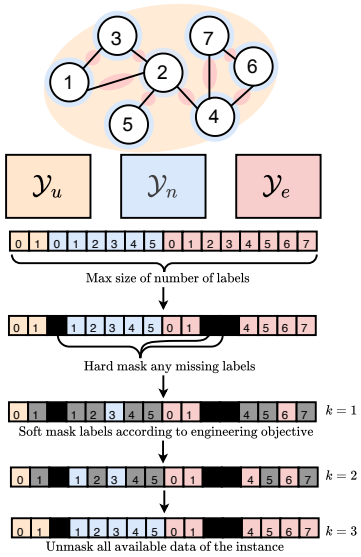
and ammonium sulfate, 0.1% monosodium glutamic acid, 0.2% amino acid supplement, 2% agar, 2% glucose, 50 μg/mL of each analog …)", with "The incubation temperature for all the selection steps was 26°C" (the 2020 screen's medium was not checked). The screen therefore supplied glutamate as the nitrogen source and an amino-acid supplement, neither of which the simulated medium contains, and was grown at 26°C rather than the 30°C the model's biomass equation is parameterized for. Whether these differences change the prediction is untested: the baseline has not been rerun on the screen's medium (Supplementary Fig. 12g), and a later attempt to approximate rich media by opening vitamin and amino-acid exchanges at 5% of the glucose uptake rate (`setup_media_conditions.py`) produced no committed result and is not reported. Two further discrepancies are visible in the code rather than in the data. The knowledge-graph records of these experiments carry the environment YEPD at 30°C, the filter of the query that built the dataset (`queries/001_small_build.cql`), which does not match the selection medium quoted above and should be corrected at the source. And the row misalignment of the matched file (`match_fba_to_experiments.py` pairs `dataset[i]` with `label_df.iloc[i]`) affects any other analysis built on that file. The prediction's constancy, not the medium, sets the ceiling here: with 93% of triple deletions at wild-type growth and 66% of triples outside the model's gene set, no medium change can raise the correlation far unless it also spreads the predicted growth rates.



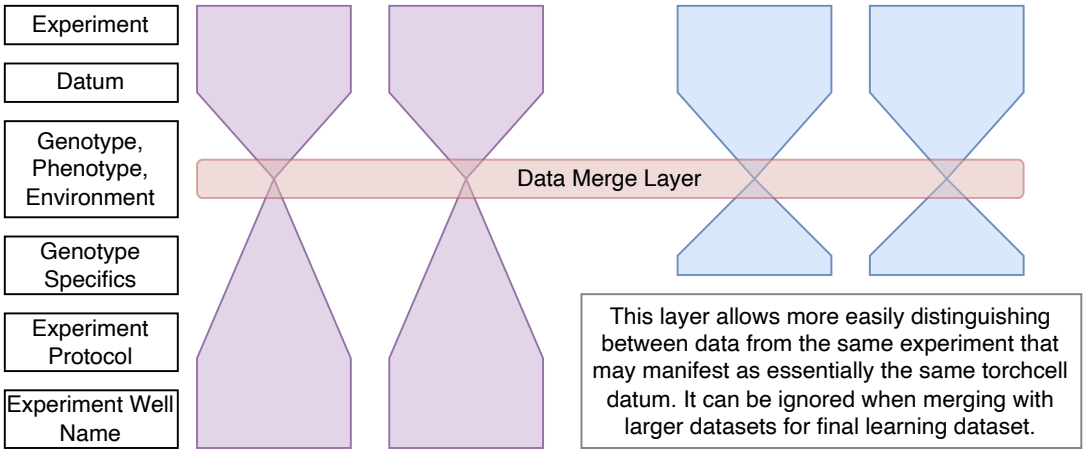
Supplementary Figure 12 The Yeast9 flux-balance baseline of Fig. 2d. a, The pipeline: yeast-GEM 9.0.2 on its default ammonium minimal medium (Supplementary Table 14); a deletion set zeroes the state of its genes and blocks every reaction whose gene-reaction rule evaluates false; the deletion sets are the Kuzmin 2018 and 2020 triples and their sub-collections; FBA maximizes the growth pseudoreaction (GLPK, 60 s limit); fitness is the growth ratio to the wild type; the interaction is that of Fig. 2a, compared with the measured value of the same triple by Pearson correlation. Every number in the boxes is read from `stats.json`. **b**, Predicted against measured τ_{ijk} for all 332,313 triples, as hexagonal bins with a logarithmic count scale; the vertical band at zero holds the 99.93% of triples whose predicted $|\tau_{ijk}|$ is below 10^{-3}, and the 231 others sit almost entirely at $+1$, $+2$, and -1. Measured values are the mean adjusted score over the gene set's raw records. **c**, Predicted against measured triple-mutant fitness for the same triples, same binning; the dotted line is $y = x$. The prediction takes a handful of values (1, 0, and bands near 0.15 and 0.37). **d**, Fraction of the single, double, and triple deletion sets whose predicted growth equals the wild type's to within 10^{-3} (gray), is reduced (orange), or is zero (fitness below 0.01, the pipeline's lethality cut; red); numbers are percentages. **e**, How much of the screens Yeast9 can score: the fraction of the 4,036 genes and of the 651,181 doubles with every member in the model (gray), and the 332,313 triples by how many of their three genes are in the model (purple), with counts; a triple with at most one model gene has $\tau_{ijk} = 0$ by construction, so 7,419 triples can be scored in full. **f**, The measured interactions of the same triples, grouped as in **e** (all three, one or two, or no gene in the model), split by the screens' convention into significant negative ($\tau < -0.08$, $P < 0.05$; red), not significant (gray), and significant positive ($\tau > 0.08$, $P < 0.05$; orange), with counts on a logarithmic axis. P is the smallest over the gene set's raw records. **g**, Placeholder for the rerun on the screen's own selection medium (SD/MSG synthetic medium with glutamate nitrogen and an amino-acid supplement, 26°C), which has not been run.

Supplementary Table 15 Phenotype datasets integrated into the TorchCell knowledge graph. Genotypes are labeled instances; Envs. is the number of distinct media/condition contexts; Label type is the graph object that carries the label (*global* whole-cell scalar, *node* gene-level, *edge* digenic, *hyperedge* trigenic). smf/dmf/tmf: single/double/triple-mutant fitness; dmi/tmi: di-/trigenic interaction.

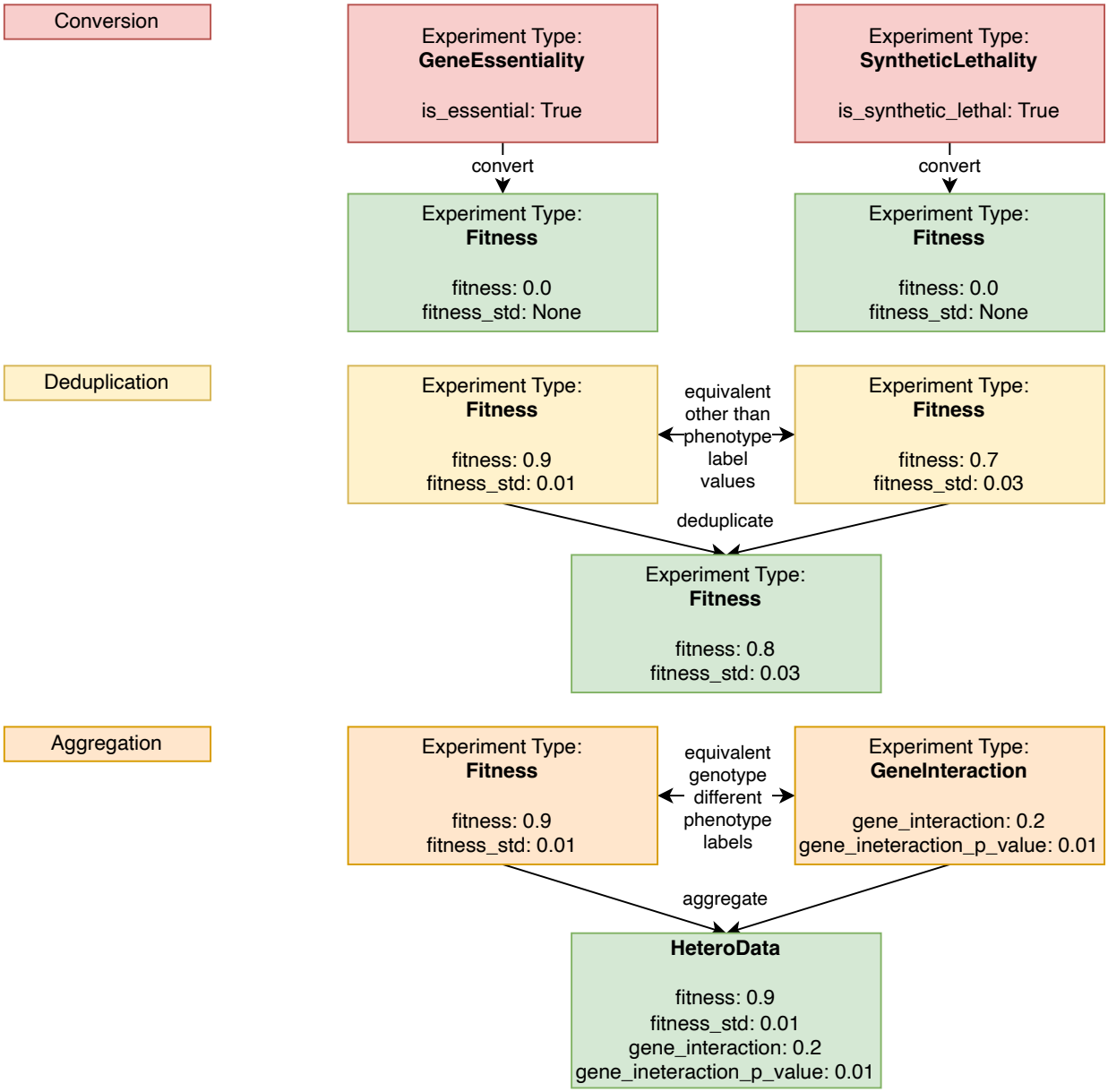
Dataset	Genotypes	Envs.	Phenotype	Label type	Description	Status
Baryshnikova_2010	6,022	1	smf	global	growth rate	✓
Costanzo_2016_smf	21,718	2	smf	global	growth rate	✓
Costanzo_2016_dmf	20,705,612	2	dmf	global	growth rate	✓
Costanzo_2016_dmi	20,705,612	2	dmi	edge	gene interaction	✓
Kuzmin_2018_smf	1,539	1	smf	global	growth rate	✓
Kuzmin_2018_dmf	410,399	1	dmf	global	growth rate	✓
Kuzmin_2018_tmf	91,111	1	tmf	global	growth rate	✓
Kuzmin_2018_dmi	410,399	1	dmi	edge	gene interaction	✓
Kuzmin_2018_tmi	91,111	1	tmi	hyperedge	gene interaction	✓
Kuzmin_2020_smf	480	1	smf	global	growth rate	✓
Kuzmin_2020_dmf	256,862	1	dmf	global	growth rate	✓
Kuzmin_2020_tmf	537,911	1	tmf	global	growth rate	✓
Kuzmin_2020_dmi	256,862	1	dmi	edge	gene interaction	✓
Kuzmin_2020_tmi	537,911	1	tmi	hyperedge	gene interaction	✓
SGD_essential	1,101	1	viable	global	viability	✓
SynthLethalYeast	1,400	–	viable	global	viability	✓
SynthRescueYeast	6,948	–	viable	global	viability	✓
scmd2_2005	4,718	1	morphology	global	cell morphology	✓
scmd2_2018	1,112	1	morphology	global	cell morphology	✓
scmd2_2022	1,982	1	morphology	global	cell morphology	✓
ODuibhir_2014	1,312	1	expr	global, node	sm microarray expression	✓
Kemmeren_2014	1,484	1	expr	global, node	sm microarray expression	✓
Sameith_2015	72	1	expr	global, node	dm microarray expression	✓
Zelezniak_2018	97	1	prot., met.	global, node	sm kinase deletion mutants	✓
Wildenhain_2015	195	4,915	smf	global	drug tolerance	in prog.
Lian_2017	18,000	1	AID	global	furfural tolerance	in prog.
FitDb	∼6,000	1,144	smf	global	growth rate	in prog.



Supplementary Figure 13 TorchCell: ontology-unified knowledge graph and the perturbation-operator cell representation. Panels **a**–**d**.



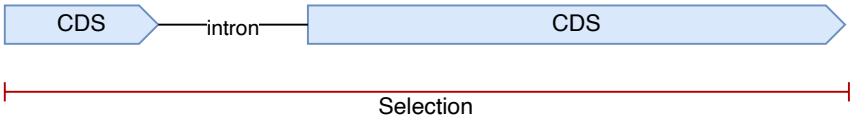
Supplementary Figure 14 Ontology data model (hourglass). Each experiment funnels genotype, phenotype, environment, and experiment-specific metadata through a shared Data Merge Layer that distinguishes records from the same experiment; the layer can be ignored when merging into larger datasets.



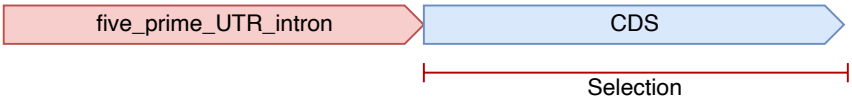
Supplementary Figure 15 Knowledge-graph normalization in three stages. Conversion maps phenotype experiment types (GeneEssentiality, SyntheticLethality) to a common Fitness type; deduplication collapses records equivalent other than their phenotype-label values; aggregation combines equivalent-genotype records with different phenotype labels (Fitness, GeneInteraction) into a single HeteroData instance.

DNA Selection
To comply with the standard format using in DNA language models we want to select the DNA sequence that has the highest probability of having the outermost start and stop codons for the CDS of the the gene. This means we will ignore five prime UTR introns if they are not internal to the gene. Below we outline how we handle intron cases:

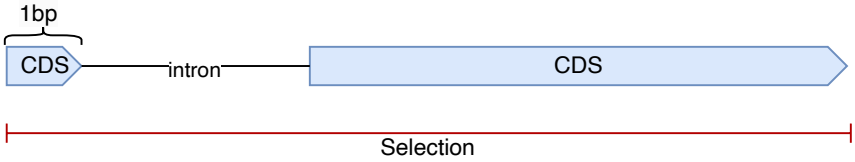
YFL039C - Internal Intron
Selectin - Entire gene



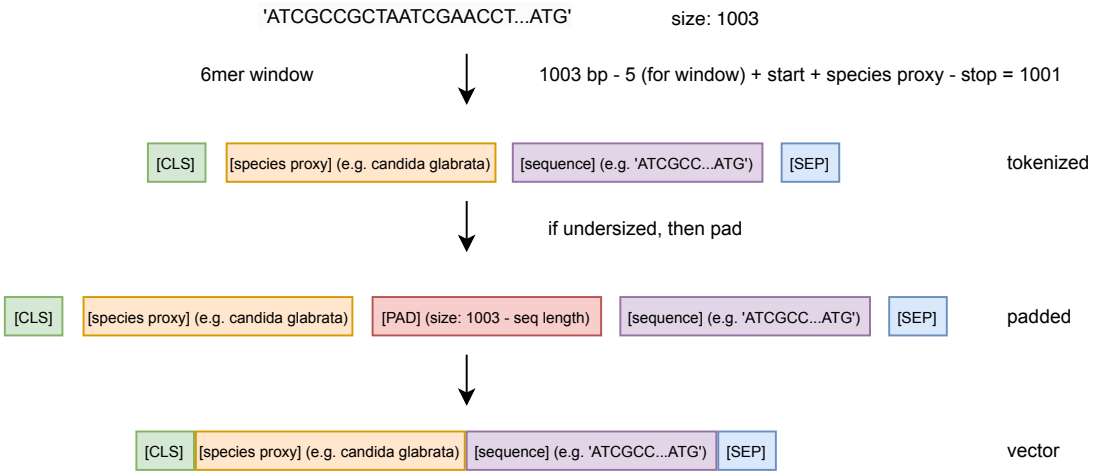
YBL072C - Upstream five prime UTR intron
Selection - Gene



YIL111W - 1bp CDS
Selection - Gene (The only case with no start codon)



Supplementary Figure 16 DNA sequence selection for the sequence model. The coding sequence is taken from the outermost start to stop codon, ignoring five-prime UTR introns unless internal to the gene. Cases: internal intron (YFL039C), 5' UTR intron (YBL072C), and a 1 bp CDS lacking a start codon (YIL111W).



Supplementary Figure 17 DNA tokenization. A 1003 bp window is encoded as [CLS], species proxy, 6-mer sequence, [SEP]; undersized sequences are padded with [PAD] to a fixed length before vectorization.

[Suppl. Fig. – expression & multitask diagnostics]

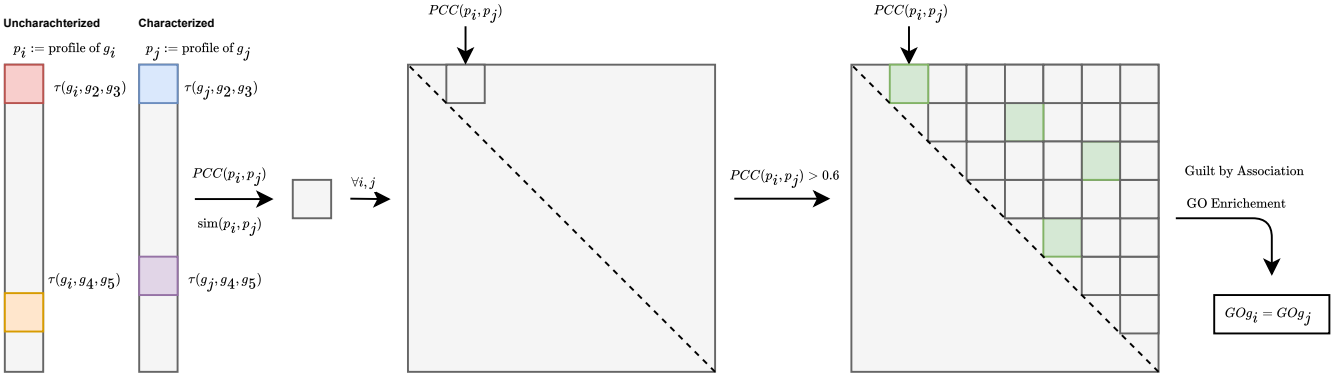
Knockout-transcriptome prediction diagnostics (per-gene calibration, top-gene predicted vs. actual) and cross-phenotype transfer from the shared latent embedding.

Supplementary Figure 18 Expression and multitask prediction diagnostics.

[Suppl. Fig. – inference at scale]

Large-scale inference pipeline over $\sim 10^6$ candidate perturbations (metabolic/kinase gene panels), with resource usage and the ranked strain-design output.

Supplementary Figure 19 Inference at scale for strain design.



Supplementary Figure 20 Guilt-by-association annotation of uncharacterized genes. **a**–**e** Triple-interaction profiles p are correlated (Pearson, PCC) across all gene pairs; pairs with PCC > 0.6 are linked, and GO enrichment over the resulting network transfers annotations ($\text{GO}_{g_i} = \text{GO}_{g_j}$). **f**, Total quantification of triple interactions across the remaining yeast genome (coverage of uncharacterized genes). **g**, The resulting new annotations transferred to previously uncharacterized genes.

References

[1] Stephanopoulos, G., Aristidou, A. A. & Nielsen, J. *Metabolic Engineering: Principles and Methodologies* (Elsevier, 1998).

[2] Bordbar, A., Monk, J. M., King, Z. A. & Palsson, B. O. Constraint-based models predict metabolic and associated cellular functions. *Nature Reviews Genetics* **15**, 107–120 (2014).

[3] Costanzo, M. *et al.* A global genetic interaction network maps a wiring diagram of cellular function. *Science* **353**, aaf1420–aaf1420 (2016).

[4] Kuzmin, E. *et al.* Systematic analysis of complex genetic interactions. *Science* **360**, eaao1729 (2018).

[5] Sapoval, N. *et al.* Current progress and open challenges for applying deep learning across the biosciences. *Nature Communications* **13**, 1728 (2022).

[6] Volk, M. J. *et al.* Metabolic Engineering: Methodologies and Applications. *Chemical Reviews* acs.chemrev.2c00403 (2022).

[7] Presnell, K. V. & Alper, H. S. Systems Metabolic Engineering Meets Machine Learning: A New Era for Data-Driven Metabolic Engineering. *Biotechnology Journal* **14**, 1800416 (2019).

[8] Culley, C., Vijayakumar, S., Zampieri, G. & Angione, C. A mechanism-aware and multiomic machine-learning pipeline characterizes yeast cell growth. *Proceedings of the National Academy of Sciences* **117**, 18869–18879 (2020).

[9] Cui, H. *et al.* scGPT: Toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods* **21**, 1470–1480 (2024).

[10] Kalfon, J., Samaran, J., Peyré, G. & Cantini, L. scPRINT: Pre-training on 50 million cells allows robust gene network predictions. *Nature Communications* **16**, 3607 (2025).

[11] Merzbacher, C., Mac Aodha, O. & Oyarzún, D. A. Accurate prediction of gene deletion phenotypes with Flux Cone Learning. *Nature Communications* **16**, 8492 (2025).

[12] Ahlmann-Eltze, C., Huber, W. & Anders, S. Deep-learning-based gene perturbation effect prediction does not yet outperform simple linear baselines. *Nature Methods* **22**, 1657–1661 (2025).

[13] Stewart, A. J. *et al.* TorchGeo: Deep learning with geospatial data. *Proceedings of the 30th International Conference on Advances in Geographic Information Systems* 1–12 (2022).

[14] Roohani, Y., Huang, K. & Leskovec, J. Predicting transcriptional outcomes of novel multigene perturbations with GEARS. *Nature Biotechnology* **42**, 927–935 (2024). [ZOTERO-PENDING – manual entry, reconcile before submission].

[15] Chao, K.-H. *et al.* Predicting dynamic expression patterns in budding yeast with a fungal DNA language model (2025).

[16] Karollus, A. *et al.* Species-aware DNA language models capture regulatory elements and their evolution. *Genome Biology* **25**, 83 (2024). [ZOTERO-PENDING – manual entry, reconcile before submission].

[17] Rosen, Y. *et al.* Universal cell embedding provides a foundation model for cell biology. *Nature* (2026). [ZOTERO-PENDING – manual entry, reconcile before submission; Nature, online ahead of print (2026-07-08), no volume/pages assigned yet].

Draft word-budget report (body prose vs `\cb` budget; figures/captions excluded; auto-generated at build)

Section	Words	Budget	%
Abstract	140	150	93
Introductionintrotent	362	570	64
Resultsresultstodo	679	1900	36
A shared ontology and knowledge graph unify metabolic-engineering datar1todo	457	380	120
The Cell Graph Transformer predicts trigenic gene-gene interactions at state of the artr2todo	53	380	14
One embedding, many phenotypes: expression, morphology, and fitnessr3todo	20	380	5
Natural genetic variation versus model-designed perturbationsr4todo	21	380	6
Drug exposure and robustness of the learned cell representationr-robusttodo	42	380	11
Extending to metabolism and strain designr5todo	33	380	9
Discussiondiscussiontodo	170	570	30
Methodsmethodstodo	2479	—	—